

Analisis Data & Machine Learning Dasar

untuk SMK dan umum



2025

Kelas 10 Semester 2

Analisis Data & Machine Learning Dasar

Aolia Ikhwanudin
Onno W. Purbo

Institut Teknologi Tangerang Selatan (ITTS)
OnnoCenter
2025

Lisensi & Catatan Karya

Karya ini dilisensikan di bawah lisensi **Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)**.

Artinya, **siapa pun bebas untuk menggunakan, menyalin, membagikan, dan mengadaptasi** materi ini, **dengan syarat**:

- **Memberikan atribusi yang sesuai** (menyebut sumber asli),
- **Menyertakan lisensi yang sama** jika dimodifikasi atau dikembangkan lebih lanjut.

Lihat detail lisensi di sini <https://creativecommons.org/licenses/by-nd/4.0/legalcode.id>

Desain Cover: Irwan Siswanto

Disclaimer

Materi pembelajaran ini dibuat dengan **dana swadaya masyarakat Indonesia** dan **kontribusi sukarela dari para dosen di Institut Teknologi Tangerang Selatan (ITTS)**.

Karya ini didistribusikan secara bebas untuk mendukung pendidikan digital dan kecerdasan buatan di kalangan pelajar Indonesia.

Kami **memohon maaf** jika terdapat kekurangan dalam isi maupun penyajian materi ini. Kritik dan saran sangat kami harapkan untuk terus menyempurnakan karya serupa di masa depan.

Kata Pengantar

Buku ini disusun sebagai wujud nyata kepedulian sivitas akademika Institut Teknologi Tangerang Selatan (ITTS) terhadap perkembangan pendidikan kejuruan di Indonesia, khususnya bagi siswa Sekolah Menengah Kejuruan (SMK). Di tengah kemajuan teknologi yang begitu pesat, kami terdorong untuk menghadirkan materi pembelajaran **yang menantang, aplikatif, dan relevan dengan dunia industri digital**, agar siswa dapat mencapai **kemampuan maksimalnya** dalam memahami dan membangun sistem *Artificial Intelligence (AI)*.

Perlu kami sampaikan bahwa **materi dalam buku ini menuntut tingkat pemahaman teknis yang cukup tinggi**, terutama karena pembahasannya **sangat kental dengan praktik *coding* menggunakan bahasa Python** untuk membuat dan melatih model AI secara mandiri. Oleh karena itu, **tidak semua siswa, sekolah, maupun guru mungkin dapat langsung menyerap seluruh isi buku ini secara optimal**.

Bagi pembaca yang masih merasa kesulitan mengikuti langkah-langkah teknis yang disajikan, **kami sangat menyarankan untuk terlebih dahulu mempelajari buku “AI untuk SMA”**, yang disusun dengan pendekatan yang lebih **sederhana dan bertahap** sebelum melanjutkan ke tingkat SMK ini.

Penyusunan dan penerbitan buku ini dilakukan secara mandiri dengan semangat berbagi pengetahuan terbuka. Kami mengucapkan terima kasih kepada berbagai pihak yang telah mendukung dan memberi masukan dalam proses penulisan. Kritik dan saran yang membangun sangat kami harapkan agar edisi berikutnya dapat semakin baik dan mudah diakses oleh seluruh siswa Indonesia.

Kami berharap buku ini dapat menjadi **jembatan antara teori dan praktik AI**, sekaligus menginspirasi siswa SMK untuk **berpikir kreatif, logis, dan bertanggung jawab dalam menggunakan teknologi**. Dengan rendah hati, kami mohon doa dan dukungan agar karya sederhana ini dapat memberi manfaat luas dan menjadi amal jariyah bagi semua pihak yang terlibat.

Tangerang Selatan, Oktober 2025

Penulis

Daftar Isi

Disclaimer.....	2
Kata Pengantar.....	3
Daftar Isi.....	4
BAB 1: Pembersihan & Preprocessing Data dengan Pandas.....	8
Tujuan Pembelajaran.....	8
Peta Konsep Sederhana.....	8
Apersepsi: Kamu Main Game Berantakan!.....	8
Penjelasan Teori.....	9
Kenapa Harus Bersihkan Data?.....	9
Teori Dasar.....	9
Analogi Game Minecraft.....	9
Kasus Ilustrasi.....	10
Soal Latihan & Pembahasan.....	11
Fun Facts!.....	14
Tips Belajar & Menghafal Cepat.....	14
Aktivitas Siswa.....	14
Rangkuman.....	17
Latihan Soal.....	17
Tugas Proyek (5 Soal).....	21
BAB 2: Konsep Machine Learning (Supervised vs Unsupervised).....	24
Tujuan Pembelajaran.....	24
Peta Konsep.....	24
Apersepsi: Seru-Seruan Bareng Bot!.....	24
Penjelasan Teori.....	25
Apa Itu Machine Learning?.....	25
Dua Jenis Belajar di Machine Learning.....	25
Analoginya Biar Lebih Jelas.....	26
Contoh Nyata.....	26
Kenapa Penting untuk Kalian?.....	26
Catatan.....	26
Kasus Ilustrasi.....	27
Soal Latihan & Pembahasan.....	29
Fun Facts!.....	30
Tips Belajar & Menghafal Cepat.....	30
Aktivitas Siswa.....	31
Aktivitas 2: Kelompokkan Lagu Favorit (Unsupervised).....	31
Rangkuman.....	34
Latihan Soal.....	35
Tugas Proyek (5 Soal).....	39
BAB 3: Eksperimen dengan Orange Data Mining (Klasifikasi & Clustering).....	44
Tujuan Pembelajaran.....	44

Peta Konsep Sederhana.....	44
Apersepsi: Kamu Mau Prediksi Pemenang Turnamen Game!.....	44
Penjelasan Teori.....	45
Konsep Dasar (Teori Ringan).....	45
Kenapa Penting?.....	45
Alat yang Dipakai di Orange.....	45
Rangkuman.....	45
Kasus Ilustrasi.....	46
Soal Latihan & Pembahasan.....	47
Fun Facts!.....	49
Tips Belajar & Menghafal Cepat.....	50
Aktivitas Siswa.....	50
Rangkuman.....	53
Latihan Soal.....	54
Tugas Proyek (5 Soal).....	57
BAB 4: Supervised Learning dengan scikit-learn (Regresi Linier & Decision Tree).....	61
Tujuan Pembelajaran.....	61
Peta Konsep Sederhana.....	61
Apersepsi: Kamu Mau Prediksi Harga Jual HP Bekas!.....	62
Penjelasan Teori.....	62
Apa Itu Supervised Learning?.....	62
Dua Algoritma Andalan.....	62
Ilustrasi Sederhana.....	63
Kenapa Keren Buat SMK?.....	63
Kasus Ilustrasi.....	63
Soal Latihan & Pembahasan.....	65
Fun Facts!.....	67
Tips Belajar & Menghafal Cepat.....	67
Aktivitas Siswa.....	68
Rangkuman Singkat.....	73
Latihan Soal.....	73
Tugas Proyek.....	77
BAB 5: Evaluasi Model (Akurasi, Presisi, Recall).....	81
Tujuan Pembelajaran.....	81
Peta Konsep Sederhana.....	81
Apersepsi: Kamu Punya Bot Detektor Kucing!.....	81
Penjelasan Teori.....	82
Akurasi (Accuracy).....	82
Presisi (Precision).....	82
Recall.....	82
Analogi Seru.....	82
Rangkuman.....	83
Kasus Ilustrasi.....	83

Soal Latihan & Pembahasan.....	85
Soal 1: Hitung Akurasi.....	85
Soal 4: Pilih Metrik yang Tepat.....	86
Ringkasan.....	86
Fun Facts!.....	87
Tips Belajar & Menghafal Cepat.....	87
Aktivitas Siswa.....	87
Rangkuman.....	90
Latihan Soal.....	91
Tugas Proyek.....	94
Proyek 5: Klasifikasi Jenis Bunga.....	99
BAB 6: Visualisasi Hasil Machine Learning (ML).....	101
Tujuan Pembelajaran.....	101
Peta Konsep Sederhana.....	101
Apersepsi: Kamu Mau Pamerin Game ke Teman!.....	102
Penjelasan Teori.....	102
Kenapa Visualisasi Itu Penting?.....	102
Jenis-Jenis Visualisasi Populer.....	102
Analogi Seru.....	103
Kasus Ilustrasi.....	103
Soal Latihan & Pembahasan.....	105
Fun Facts!.....	108
Tips Belajar atau Menghafal Cepat.....	108
Aktivitas Siswa.....	108
Rangkuman.....	113
Latihan Soal.....	113
Tugas Proyek.....	117
BAB 7: Etika Data & AI (Bias, Privasi, Fairness).....	123
Tujuan Pembelajaran.....	123
Peta Konsep Sederhana.....	123
Apersepsi: Filter Foto yang Bikin Kamu Bingung!.....	123
Penjelasan Teori.....	123
Konsep Pendek & Jelas.....	123
5 Aturan Emas Etika.....	124
Checklist Etika.....	124
Praktik Aman dengan Python + Pandas (open source).....	124
Mini-Lab scikit-learn: uji akurasi per kelompok.....	125
Praktik Keras (neural network open source).....	126
Orange Data Mining (tanpa koding).....	126
Rangkuman (super singkat).....	127
Kasus Ilustrasi.....	127
Soal Latihan & Pembahasan.....	129
Fun Facts!.....	132
Tips Belajar atau Menghafal Cepat.....	133

Aktivitas Siswa.....	133
Rangkuman.....	137
Latihan Soal.....	137
Tugas Proyek.....	141
BAB 8: Proyek – Model Prediksi Sederhana.....	146
Tujuan Pembelajaran.....	146
Peta Konsep Sederhana.....	146
Apersepsi: Kamu Mau Prediksi Penjualan Kebab di Sekolahmu!.....	146
Penjelasan Teori.....	147
Apa itu Prediksi?.....	147
Dataset = Tabel Cerita.....	147
Preprocessing (Beresin Data) — pakai Pandas.....	147
Beda Tugas: Regression vs Classification.....	148
Bagi Data: Train–Test Split.....	148
Pipeline (Rangkaian Otomatis di scikit-learn).....	148
Model Regresi: Linear Regression.....	148
Model Klasifikasi: Decision Tree.....	149
Evaluasi yang Jujur.....	149
Visualisasi (Biar WOW).....	149
Neural Network — pakai Keras.....	149
Etika Data (Selalu!).....	150
Checklist Proyek (dari nol → rilis).....	150
Catatan Open-Source (Wajib!).....	150
Ringkas Super Singkat.....	150
Mini Contoh Alur Teknis.....	150
Kasus Ilustrasi.....	151
Soal Latihan & Pembahasan.....	155
Fun Facts!.....	160
Tips Belajar atau Menghafal Cepat.....	160
Aktivitas Siswa.....	161
Kesimpulan.....	165
Rangkuman.....	165
Latihan Soal.....	166
Tugas Proyek.....	170
Catatan Umum (Berlaku untuk 5 Proyek).....	177
Institut Teknologi Tangerang Selatan (ITTS) dan Penulis.....	179

BAB 1: Pembersihan & Preprocessing Data dengan Pandas

Siapa jadi detektif data? Kali ini kita mau belajar satu hal penting banget: **membersihkan dan merapikan data**. Kenapa ini penting? Karena data itu mirip kayak bahan makanan yang baru dibeli dari pasar. Kadang ada yang kotor, ada yang busuk, atau ada yang hilang. Nah, kita perlu **Pandas** buat membersihkan semuanya biar siap

Tujuan Pembelajaran

Setelah sesi ini, kalian bakal bisa:

- **Mengidentifikasi** data yang kotor (hilang, salah, atau duplikat).
- **Menghapus** atau **mengisi** data yang hilang dengan cara yang benar.
- **Mengganti** data yang salah.
- **Menghilangkan** data yang kembar.
- **Mengubah** format data agar seragam.

Peta Konsep Sederhana

Bayangin kita lagi main game Minecraft dan mau bikin rumah keren. Tapi, bahan-bahan kita berantakan.

- **Bahan Mentah (Data Mentah)**: Ini kayu, batu, dan pasir yang berserakan. Ada yang hancur, ada yang kurang.
- **Pembersihan (Preprocessing)**: Kita pakai **Pandas** buat:
 - **Pilih** bahan yang bagus.
 - **Ganti** bahan yang rusak.
 - **Tambahin** bahan yang kurang.
- **Bahan Siap Pakai (Data Bersih)**: Semua bahan sudah rapi, siap dibangun jadi rumah impian!

Apersepsi: Kamu Main Game Berantakan!

Kamu lagi main game RPG. Ada data pemain, item, dan status. Tapi datanya berantakan banget:

- Ada nama item yang ketikannya salah (misal: "PedangApi" dan "Pedang api").
- Ada data yang kosong, misalnya di kolom "level".
- Ada data pemain yang kembar.

Kalau datanya kayak gini, game-nya bisa error atau ngaco. Jadi, kita butuh "sihir" Pandas buat merapikan semua data itu.

Penjelasan Teori

Kenapa Harus Bersihkan Data?

Bayangin kamu lagi main **game RPG**. Semua item di *inventory* berantakan:

- Ada **pedang kembar** (duplikat).
- Ada **ramuan tanpa label** (data kosong).
- Ada **armor salah eja**: “Armor Baja” vs “Amor Baja”.

Kalau game-nya dibiarkan, bisa **error**.

Di dunia nyata, data kita juga gitu—sering berantakan.

Nah, disinilah kita butuh **alat ajaib**: *Pandas*!

Pandas adalah *library open source* di Python yang super jago rapihin data.

Teori Dasar

- **Data kotor** = bahan makanan dari pasar, ada yang busuk/hilang.
- **Preprocessing** = cuci sayur sebelum dimasak.
- **Data bersih** = hasil analisis akurat & siap dipakai buat *machine learning* atau *neural network*.

Langkah umum preprocessing dengan *Pandas*:

1. **Cek data kosong** → `isnull().sum()`
2. **Hapus data kosong** → `dropna()`
3. **Isi data kosong** → `fillna()`
4. **Hapus data kembar** → `drop_duplicates()`
5. **Ubah format data** (teks → angka, tanggal, dsb) → `astype()`, `to_datetime()`
6. **Seragamkan ejaan** → `.str.lower()`, `.str.capitalize()`

Analogi Game Minecraft

- **Data Mentah** = kayu, batu, pasir yang masih berantakan.
- **Pembersihan (Preprocessing)** = pilih yang bagus, buang yang rusak, isi yang kurang.
- **Data Bersih** = bahan siap dipakai buat bangun rumah impian.

Kasus Ilustrasi

Kasus 1: Jadwal Piket Kelas

Masalah: Ada data "Senin", "senin", "senin.". Komputer baca ini beda semua.

Solusi: Seragamkan jadi "Senin".

Pseudocode:

```
FOR setiap data di kolom 'hari':  
    ubah huruf awal jadi kapital  
    hapus tanda titik  
END
```

Kasus 2: Survei Kantin Sekolah

Masalah: Ada siswa yang isi survei tapi kolom "makanan_favorit" kosong.

Solusi: Isi kosong dengan "Tidak tahu".

Pseudocode:

```
FOR setiap baris di kolom 'makanan_favorit':  
    IF data kosong:  
        isi dengan "Tidak tahu"  
    ENDIF  
END
```

Kasus 3: Data Penjualan Jajanan

Masalah: Ada harga yang ditulis "Rp. 5000". Komputer anggap itu teks, bukan angka.

Solusi: Buang "Rp." biar bisa dihitung.

Pseudocode:

```
FOR setiap data di kolom 'harga':  
    hapus tulisan "Rp. "  
    ubah jadi angka  
END
```

Kasus 4: Daftar Anggota Ekskul

Masalah: Ada nama yang sama muncul dua kali, misalnya "Budi".

Solusi: Hapus duplikat, biar tiap nama unik.

Pseudocode:

```
hapus semua baris yang kembar  
simpan hanya satu data unik
```

Kasus 5: Data Peminjaman Buku

Masalah: Ada tanggal dengan format beda: "01-09-2025" dan "2025/09/01".

Solusi: Samakan format jadi "2025-09-01".

Pseudocode:

```
FOR setiap data di kolom 'tanggal':  
    ubah format tanggal jadi YYYY-MM-DD  
END
```

Kenapa Ini Penting?

- Data bersih = hasil analisis akurat.
- Pandas bikin hidup lebih mudah.
- Ini dasar sebelum masuk ke *machine learning*, *keras*, atau *neural network*.

Kalau datanya kotor, model *AI* bisa salah.

Kalau datanya bersih, hasil *AI* bisa lebih pintar.

Tantangan Mini

Coba kalian buat data sederhana tentang:

- Absensi kelas.
- Daftar nilai ujian.
- Penjualan di kantin.

Lalu, rapikan datanya pakai aturan di atas.

Soal Latihan & Pembahasan

Soal 1: Cek Data Kosong

Kamu punya tabel `df_survey`. Beberapa kolom ada nilai kosong.

Bagaimana cara tahu berapa banyak data yang hilang di tiap kolom?

Pseudocode:

```
FOR setiap kolom di df_survey:  
    hitung jumlah data kosong  
END
```

Jawaban:

```
df_survey.isnull().sum()
```

Pembahasan:

Data kosong itu bahaya, karena bisa bikin hasil analisis salah. Dengan `.isnull().sum()`, kita tahu kolom mana yang bolong. Ibaratnya kayak cek daftar piket, siapa yang bolos.

Soal 2: Hapus Data Kosong

Ada baris data di `df_survey` yang kolomnya kosong. Bagaimana cara hapus baris itu?

Pseudocode:

```
FOR setiap baris di df_survey:
    IF ada kolom yang kosong:
        hapus baris itu
    ENDIF
END
```

Jawaban:

```
df_survey.dropna()
```

Pembahasan:

Kalau datanya kosong total, lebih aman dibuang. Bayangin bikin laporan nilai, ada siswa tanpa nilai sama sekali → mending hapus supaya rapi.

Soal 3: Ubah Tipe Data

Di kolom `harga`, data disimpan sebagai teks `"5000"`. Bagaimana cara ubah jadi angka?

Pseudocode:

```
FOR setiap data di kolom 'harga':
    ubah tipe data jadi integer
END
```

Jawaban:

```
df_survey['harga'] = df_survey['harga'].astype(int)
```

Pembahasan:

Kalau masih teks, komputer nggak bisa hitung. Setelah diubah ke angka, kita bisa menjumlahkan atau menghitung rata-rata. Ibaratnya, angka "100" beda jauh dengan teks "seratus".

Soal 4: Hapus Data Duplikat

Ada siswa yang tercatat dua kali di `df_survey`.
Bagaimana cara hapus data ganda?

Pseudocode:

```
hapus semua baris yang identik  
simpan hanya satu versi unik
```

Jawaban:

```
df_survey.drop_duplicates()
```

Pembahasan:

Data kembar bisa bikin analisis salah.

Misalnya, nama “Budi” muncul dua kali → jumlah siswa jadi kelihatan lebih banyak dari aslinya.

Soal 5: Merapikan Huruf

Di kolom `jenis_makanan`, ada data "bakso", "Bakso", "BAKSO".
Bagaimana cara bikin semuanya seragam jadi “Bakso”?

Pseudocode:

```
FOR setiap data di kolom 'jenis_makanan':  
    ubah huruf awal jadi kapital  
    ubah huruf lain jadi kecil  
END
```

Jawaban:

```
df_survey['jenis_makanan'] =  
df_survey['jenis_makanan'].str.capitalize()
```

Pembahasan:

Data yang beda huruf tapi sama arti harus diseragamkan.

Kalau tidak, komputer anggap itu tiga makanan berbeda.

Padahal semuanya cuma **Bakso**.

Kesimpulan

- Data kosong → cek pakai `.isnull().sum()` atau hapus dengan `.dropna()`.
- Data teks ke angka → ubah dengan `.astype(int)`.
- Data kembar → hapus dengan `.drop_duplicates()`.
- Data ejaan → seragamkan pakai `.str.capitalize()`.

Semua jurus ini bisa dilakukan dengan **Python + Pandas** (open source).

Dengan data bersih, kita siap lanjut ke tahap **machine learning** dan bahkan bikin **neural network** pakai Keras!

Fun Facts!

- **Pandas** sering banget dipakai buat bersihin data. Sampai-sampai, 80% waktu data scientist itu dihabiskan untuk **data cleaning**!
- **Ollama** dan **OpenWebUI** itu bisa kalian pakai buat bikin asisten AI di komputer sendiri. Kalian bisa tanya "bagaimana cara bersihkan data yang kembar di Pandas?" dan dia bakal kasih jawabannya.
- **RAG (Retrieval-Augmented Generation)** lokal itu bikin AI kalian bisa 'belajar' dari materi yang kalian punya. Jadi, AI kalian bisa lebih pintar dari buku cetak.

Tips Belajar & Menghafal Cepat

- **Praktik Langsung!** Cari dataset yang berantakan di internet (misalnya di Kaggle) dan coba bersihkan.
- **Buat Skema Proyek Kecil:** Buat data fiktif tentang hobi kalian (misalnya: data rekor game atau daftar film favorit) lalu coba rapikan.
- **Gunakan Komentar di Kode:** Setiap kali kalian mengetik kode, tulis di sampingnya fungsinya buat apa.

Aktivitas Siswa

Aktivitas 1: Detektif Data Kosong

Kamu jadi detektif! Cari tahu kolom mana yang ada data kosong.

Langkah-langkah:

1. Buat DataFrame dengan data yang ada kosongnya (*np.nan*).
2. Cek berapa jumlah data kosong tiap kolom.
3. Catat hasilnya.

Pseudocode:

```
buat tabel dengan beberapa nilai kosong
untuk setiap kolom:
    hitung jumlah data kosong
```

Python Script:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "nama": ["Andi", "Budi", None, "Citra"],
    "nilai": [80, None, 70, 90]
})

print(df.isnull().sum())
```

Aktivitas 2: Rapikan Ejaan

Kolom **warna** berisi “merah”, “Merah”, “MERAH”. Rapikan supaya semua jadi sama.

Langkah-langkah:

1. Buat DataFrame dengan data acak huruf besar-kecil.
2. Ubah semua jadi huruf awal kapital.
3. Lihat hasilnya.

Pseudocode:

```
untuk setiap data di kolom warna:  
    ubah huruf awal jadi kapital
```

Python Script:

```
df = pd.DataFrame({"warna": ["merah", "Merah", "MERAH",  
                             "meRah"]})  
df["warna"] = df["warna"].str.capitalize()  
print(df)
```

Aktivitas 3: Isi Data Kosong

Ada kolom **nilai** yang kosong. Isi dengan angka 0 biar lengkap.

Langkah-langkah:

1. Buat DataFrame dengan data kosong.
2. Isi nilai kosong dengan 0.
3. Tampilkan hasilnya.

Pseudocode:

```
untuk setiap data kosong di kolom nilai:  
    isi dengan 0
```

Python Script:

```
df = pd.DataFrame({  
    "nama": ["Andi", "Budi", "Citra"],  
    "nilai": [80, None, 70]  
})  
  
df["nilai"] = df["nilai"].fillna(0)  
print(df)
```

Aktivitas 4: Hapus Data Kembar

Ada siswa yang tercatat dua kali. Hapus yang ganda.

Langkah-langkah:

1. Buat DataFrame dengan data ganda.
2. Hapus data duplikat.
3. Simpan hanya data unik.

Pseudocode:

```
hapus semua baris kembar  
simpan hanya satu
```

Python Script:

```
df = pd.DataFrame({  
    "nama": ["Andi", "Budi", "Budi", "Citra"],  
    "kelas": ["X", "XI", "XI", "XII"]  
})  
  
df = df.drop_duplicates()  
print(df)
```

Aktivitas 5: Ubah Tipe Data

Kolom **harga** ditulis sebagai teks "5000". Ubah jadi angka biar bisa dihitung.

Langkah-langkah:

1. Buat DataFrame dengan harga dalam bentuk teks.
2. Ubah tipe data jadi integer.
3. Hitung total harga.

Pseudocode:

```
untuk setiap data di kolom harga:  
    ubah teks jadi integer  
jumlahkan semua harga
```

Python Script:

```
df = pd.DataFrame({  
    "barang": ["Pensil", "Buku", "Penggaris"],  
    "harga": ["5000", "10000", "7000"]  
})  
  
df["harga"] = df["harga"].astype(int)  
print("Total Harga:", df["harga"].sum())
```

Kesimpulan

- Data kosong bisa dicari dengan `.isnull().sum()`.
- Ejaan bisa dirapikan dengan `.str.capitalize()`.

- Data kosong bisa diisi dengan `.fillna()`.
- Data ganda bisa dihapus dengan `.drop_duplicates()`.
- Data teks bisa diubah jadi angka dengan `.astype(int)`.

Dengan latihan ini, kalian sudah mulai jadi **detektif data**.

Langkah ini penting banget sebelum masuk ke *machine learning*, *keras*, dan *neural network*.

Rangkuman

Dalam bab ini kita belajar bahwa **data mentah tidak selalu rapi**. Sama seperti kamar yang berantakan, data juga bisa ada yang **kosong, salah, atau kembar**. Kalau dipakai langsung, hasil analisis bisa **error**. Karena itu kita butuh *data preprocessing* dengan **Pandas**. Dengan Pandas, kita bisa **menghapus data kosong**, **mengisi nilai hilang**, **mengubah teks jadi angka**, dan **menghapus data ganda**.

Pembersihan data itu ibarat **cuci sayur sebelum dimasak**. Kalau bahannya bersih, masakan pasti enak. Kalau datanya bersih, analisis juga jadi **akurat**. Inilah yang bikin *data preprocessing* jadi **langkah paling penting** sebelum kita masuk ke *machine learning*, *keras*, atau *neural network*. Ingat, **AI yang pintar hanya bisa lahir dari data yang bersih**.

Skill ini bukan cuma untuk tugas sekolah. **Di dunia kerja**, banyak profesi seperti *data scientist* dan *data analyst* menghabiskan 80% waktunya untuk membersihkan data. Jadi, dengan menguasai Pandas, kalian sudah punya **bekal emas** untuk karir di masa depan. Jangan lupa, **data kotor = hasil salah**, tapi **data bersih = keputusan tepat**.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. Data kotor bisa membuat hasil analisis salah.
Jawaban: Benar
2. `dropna()` dipakai untuk mengisi data kosong dengan nilai rata-rata.
Jawaban: Salah
3. `fillna()` dapat digunakan untuk mengganti nilai kosong.
Jawaban: Benar
4. `drop_duplicates()` berfungsi menghapus baris yang ganda.
Jawaban: Benar

5. Mengubah teks "5000" ke angka cukup dengan `astype(str)`.

Jawaban: Salah

B. Pilihan Ganda (10 Soal)

1. Perintah untuk membaca file CSV:

A. `pd.read_txt()` B. `pd.read_csv()` C. `pd.csv_read()` D. `pd.readfile()`

Jawaban: B

2. Mengecek jumlah nilai kosong per kolom:

A. `df.info()` B. `df.count()` C. `df.isnull().sum()` D. `df.describe()`

Jawaban: C

3. Menghapus baris yang memiliki nilai kosong:

A. `df.fillna()` B. `df.dropna()` C. `df.clearnna()` D. `df.removenana()`

Jawaban: B

4. Menghapus baris duplikat:

A. `df.drop_duplicates()` B. `df.unique()` C. `df.remove_dupes()` D. `df.drop_dupes()`

Jawaban: A

5. Mengubah tipe kolom `harga` (teks) menjadi integer:

A. `df["harga"] = df["harga"].astype(int)`

B. `df["harga"] = int(df["harga"])`

C. `df.astype(df["harga"], int)`

D. `int.astype(df["harga"])`

Jawaban: A

6. Menyeragamkan ejaan "senin", "Senin." menjadi "Senin":

A. `df["hari"] = df["hari"].str.upper()`

B. `df["hari"] = df["hari"].str.capitalize().str.replace(".", "", regex=False)`

C. `df["hari"] = df["hari"].str.lower()`

D. `df["hari"] = df["hari"].astype("Senin")`

Jawaban: B

7. Membersihkan "Rp. 5000" menjadi 5000:

A. `df["harga"] = df["harga"].str.replace("Rp.", "").astype(int)`

B. `df["harga"] = df["harga"].astype(int)`

C. `df["harga"] = int(df["harga"].replace("Rp.", ""))`

D. `df["harga"] = df["harga"].str.drop("Rp.").astype(int)`

Jawaban: A

8. Menstandarkan format tanggal campur-campur:

- A. `df["tgl"] = df["tgl"].astype("date")`
- B. `df["tgl"] = pd.to_datetime(df["tgl"])`
- C. `df["tgl"] = df["tgl"].str.datetime()`
- D. `df["tgl"] = date(df["tgl"])`

Jawaban: B

9. Mengisi nilai kosong kolom `nilai` dengan 0:

- A. `df["nilai"] = df["nilai"].dropna(0)`
- B. `df["nilai"] = df["nilai"].fillna(0)`
- C. `df["nilai"] = 0.fillna(df["nilai"])`
- D. `df.fillna(df["nilai"]=0)`

Jawaban: B

10. Mengonversi kolom angka yang tersimpan sebagai teks dengan spasi ekstra:

- A. `df["angka"] = df["angka"].str.strip().astype(int)`
- B. `df["angka"] = df["angka"].strip().int()`
- C. `df["angka"] = int.strip(df["angka"])`
- D. `df["angka"] = df.strip("angka").astype(int)`

Jawaban: A

C. Isian Singkat (5 Soal)

1. Perintah untuk melihat 5 baris awal DataFrame: _____.

Jawaban: `df.head()`

2. Fungsi untuk menghapus baris duplikat: _____.

Jawaban: `drop_duplicates()`

3. Fungsi untuk mengubah tipe data kolom: _____.

Jawaban: `astype()`

4. Fungsi untuk mengisi nilai kosong: _____.

Jawaban: `fillna()`

5. Fungsi untuk mengubah string tanggal ke tipe datetime: _____.

Jawaban: `pd.to_datetime()`

D. Soal Explorasi (3 Soal)

Soal 1:

Kamu mendapat `absensi.csv` dengan kolom `nama`, `status`. Nilai `status` beragam: "hadir", "Hadir", "HADIR", ada yang kosong. Tugasmu: seragamkan jadi

“Hadir” atau “Tidak Hadir”, isi kosong sebagai “Tidak Hadir”, dan hitung total yang “Hadir”.

Kunci Jawaban (contoh):

```
import pandas as pd
df = pd.read_csv("absensi.csv")
df["status"] = df["status"].fillna("Tidak Hadir")
df["status"] = df["status"].str.strip().str.capitalize()
total_hadir = (df["status"] == "Hadir").sum()
print(total_hadir)
```

Soal 2:

Ada `kantin.csv` berisi `barang`, `harga`, `jumlah`, di mana `harga` masih “Rp. 5000” dsb. Rapikan `harga` → angka, buat kolom `total = harga * jumlah`, lalu tampilkan total penjualan keseluruhan.

Kunci Jawaban (contoh):

```
import pandas as pd
df = pd.read_csv("kantin.csv")
df["harga"] = df["harga"].str.replace("Rp.", "",
regex=False).str.replace(" ", "", regex=False).astype(int)
df["total"] = df["harga"] * df["jumlah"]
print(df["total"].sum())
```

Soal 3:

File `pustaka.csv` memiliki `nama`, `tanggal_pinjam` dengan format tanggal campur: “01-09-2025”, “2025/09/01”, “2025-09-01”. Standarkan ke `YYYY-MM-DD`, urutkan dari tanggal paling baru, dan tampilkan 3 baris teratas.

Kunci Jawaban (contoh):

```
import pandas as pd
df = pd.read_csv("pustaka.csv")
df["tanggal_pinjam"] = pd.to_datetime(df["tanggal_pinjam"])
df = df.sort_values("tanggal_pinjam", ascending=False)
print(df.head(3))
```

Catatan singkat (opsional untuk siswa)

- Cek kosong: `df.isnull().sum()`
- Bersih kosong: `df.dropna()` / isi: `df.fillna(...)`
- Seragam ejaan: `.str.lower()` / `.str.capitalize()`
- Hapus duplikat: `df.drop_duplicates()`
- Ubah tipe: `.astype(int)`
- Tanggal rapi: `pd.to_datetime()`

Tugas Proyek (5 Soal)

Proyek 1: Data Kehadiran OSIS

Bayangkan kamu sekretaris OSIS. Kamu punya data absensi rapat. Tapi, ada yang kosong di kolom **status**.

Tugas: Isi data kosong dengan “Tidak Hadir” dan rapikan ejaan status agar seragam.

Pseudocode:

```
UNTUK setiap baris di kolom 'status':  
    JIKA kosong:  
        isi dengan "Tidak Hadir"  
        ubah huruf awal jadi kapital  
SELESAI
```

Python Code:

```
import pandas as pd  
  
df = pd.DataFrame({  
    "nama": ["Andi", "Budi", "Citra", "Dina"],  
    "status": ["hadir", None, "Hadir", "tidak hadir"]  
})  
  
df["status"] = df["status"].str.capitalize()  
df["status"] = df["status"].fillna("Tidak hadir")  
print(df)
```

Proyek 2: Data Nilai Ujian

Ada tabel nilai ujian, tapi kolom **nilai** sebagian berbentuk teks, misalnya "80".

Tugas: Ubah semua nilai jadi angka (*integer*) lalu hitung rata-rata kelas.

Pseudocode:

```
UNTUK setiap data di kolom 'nilai':  
    ubah teks jadi integer  
hitung rata-rata
```

Python Code:

```
df = pd.DataFrame({  
    "nama": ["Andi", "Budi", "Citra"],  
    "nilai": ["80", "75", "90"]  
})  
  
df["nilai"] = df["nilai"].astype(int)  
print("Rata-rata kelas:", df["nilai"].mean())
```

Proyek 3: Data Penjualan Kantin

Kamu punya data penjualan kantin. Di kolom **harga**, ada format "**Rp. 5000**".

Tugas: Bersihkan teks "Rp.", ubah jadi angka, lalu hitung total penjualan.

Pseudocode:

```
UNTUK setiap data di kolom 'harga':  
    hapus "Rp. "  
    ubah jadi integer  
total = harga * jumlah
```

Python Code:

```
df = pd.DataFrame({  
    "barang": ["Bakso", "Es Teh", "Mie Goreng"],  
    "harga": ["Rp. 5000", "Rp. 3000", "Rp. 7000"],  
    "jumlah": [10, 20, 5]  
})  
  
df["harga"] = df["harga"].str.replace("Rp. ", "",  
    regex=False).astype(int)  
df["total"] = df["harga"] * df["jumlah"]  
print(df)  
print("Total Penjualan:", df["total"].sum())
```

Proyek 4: Data Peminjaman Perpustakaan

Data tanggal peminjaman ada dua format: "**01-09-2025**" dan "**2025/09/01**".

Tugas: Samakan format tanggal jadi **YYYY-MM-DD**.

Pseudocode:

```
UNTUK setiap data di kolom 'tanggal':  
    ubah ke format YYYY-MM-DD
```

Python Code:

```
df = pd.DataFrame({  
    "nama": ["Andi", "Budi", "Citra"],  
    "tanggal": ["01-09-2025", "2025/09/01", "2025-09-01"]  
})  
  
df["tanggal"] = pd.to_datetime(df["tanggal"])  
print(df)
```

Proyek 5: Data Anggota Ekskul

Daftar anggota ekskul punya nama yang ganda.

Tugas: Hapus duplikat sehingga tiap siswa tercatat hanya sekali.

Pseudocode:

```
hapus baris yang kembar  
simpan data unik saja
```

Python Code:

```
df = pd.DataFrame({  
    "nama": ["Andi", "Budi", "Budi", "Citra"],  
    "ekskul": ["Futsal", "Basket", "Basket", "Musik"]  
})  
  
df = df.drop_duplicates()  
print(df)
```

Kesimpulan

- **Proyek ini melatih logika sederhana:** cek, ubah, isi, hapus, dan rapikan data.
- Semua bisa dilakukan dengan **Python + Pandas** (*open source*).
- Setelah paham, kalian siap lanjut ke dunia **machine learning, keras, dan neural network**.

BAB 2: Konsep Machine Learning (Supervised vs Unsupervised)

Siap-siap, karena kita mau kenalan sama sesuatu yang super keren dan ada di mana-mana: **Machine Learning**! Kalian pasti pernah dengar soal AI (kecerdasan buatan), kan? Nah, Machine Learning itu adalah “otak” di baliknya. Intinya, kita bakal ngajarin komputer buat belajar sendiri dari data. Mirip kayak kamu ngajarin kucingmu buat duduk atau salaman.

Tujuan Pembelajaran

Setelah sesi ini, kalian bakal bisa:

- Memahami apa itu **Machine Learning** dan kenapa dia penting.
- Mengenal dua tipe Machine Learning yang paling sering dipakai: **Supervised Learning** dan **Unsupervised Learning**.
- Membedakan kapan harus pakai Supervised Learning dan kapan harus pakai Unsupervised Learning.
- Menghubungkan konsep ini ke dunia kerja dan hobi kalian.

Peta Konsep

Machine Learning (ML)

- **Supervised Learning** (*Belajar dengan Guru*)
 - Ada **label/jawaban**
 - Tujuan: **prediksi & klasifikasi**
 - Contoh: spam email, harga rumah
- **Unsupervised Learning** (*Belajar Mandiri*)
 - **Tanpa label/jawaban**
 - Tujuan: **cari pola & kelompok**
 - Contoh: rekomendasi musik, clustering pelanggan

Ringkasnya:

- **Supervised** = Ada guru, ada jawaban.
- **Unsupervised** = Mandiri, cari pola sendiri.

Apersepsi: Seru-Seruan Bareng Bot!

Bayangin kamu lagi main **game online**. Tiba-tiba kamu kepikiran, “Asik juga kalau aku punya bot pintar yang bisa bantu farming atau nyelamatin aku pas sekarat.”

Nah, sekarang ada dua pilihan cara ngajarin si bot:

- **Tipe 1 – Supervised Learning**

- Kamu jadi guru.
- Kamu kasih contoh jelas:
 - “Kalau musuh datang → tembak!”
 - “Kalau HP tinggal 20% → lari cari *health pack*.”
- Bot nurut karena tahu aturan + jawabannya.
- **Tipe 2 – *Unsupervised Learning***
 - Kamu suruh bot cari tahu sendiri.
 - Bot lihat semua item dalam game, lalu mengelompokkan sendiri:
 - “Oh, item yang bikin lari cepat satu grup.”
 - “Item yang bikin darah nambah grup lain.”

Jadi, si bot bisa belajar **dengan guru** atau belajar **mandiri**.

Dua-duanya keren, dan dua-duanya dipakai di dunia nyata.

Pertanyaan buat kamu sebelum masuk teori:

Kalau kamu bikin bot Mobile Legends biar jago farming sendiri, menurutmu lebih enak pakai cara **Supervised** atau **Unsupervised**?

Penjelasan Teori

Apa Itu Machine Learning?

Bayangin kamu punya komputer. Biasanya, komputer itu cuma nurut kalau kita kasih perintah: “Buka folder,” “Jalankan game,” atau “Print dokumen.”

Tapi, dengan *Machine Learning* (ML), komputer bisa **belajar sendiri dari data**. Jadi bukan cuma nurut, tapi juga pintar bikin keputusan.

Analogi gampangnya:

- Kamu mengajari kucing untuk duduk. Awalnya kamu kasih contoh + hadiah. Lama-lama kucing bisa ngerti sendiri kapan harus duduk.
- Nah, komputer juga gitu: kita kasih data, lalu dia belajar dari data itu.

Dua Jenis Belajar di Machine Learning

Ada dua “gaya belajar” yang paling penting:

1. **Supervised Learning (Belajar dengan Guru)**
 - Komputer dapat data + jawaban.
 - Contoh:
 - Kamu kasih 1000 foto kucing & anjing. Setiap foto sudah ada label: “ini kucing,” “ini anjing.”
 - Komputer belajar dari contoh, lalu bisa menebak foto baru.
2. **Unsupervised Learning (Belajar Mandiri)**
 - Komputer cuma dapat **data tanpa jawaban**.

- Contoh:
 - Kamu kasih 1000 foto hewan campur aduk, tapi tanpa label.
 - Komputer bakal cari pola sendiri: “Oh, yang bulat berbulu ini sekelompok, yang kuping panjang sekelompok lain.”

Analoginya Biar Lebih Jelas

- **Supervised = Ada Guru**
Kayak belajar matematika di kelas. Ada soal, ada kunci jawaban. Kamu tinggal latihan sampai paham.
- **Unsupervised = Eksperimen Sendiri**
Kayak kamu masuk lab tanpa instruksi. Kamu pegang bahan kimia, coba-coba, dan menemukan kelompok cairan yang reaksinya mirip.

Contoh Nyata

- **Supervised Learning**
 - *Prediksi Harga Rumah*: berdasarkan luas tanah & lokasi.
 - *Deteksi Spam*: email ditandai “spam” atau “aman.”
- **Unsupervised Learning**
 - *Clustering Pelanggan*: toko online mengelompokkan pembeli berdasarkan hobi belanja.
 - *Rekomendasi Musik*: Spotify bikin playlist otomatis dari lagu-lagu yang kamu suka.

Kenapa Penting untuk Kalian?

Kalian tiap hari pakai ML tanpa sadar:

- Algoritma TikTok/YouTube → rekomendasi video.
- Google Maps → prediksi macet.
- Tokopedia/Shopee → rekomendasi barang.
- Game Online → bot musuh belajar cara nyerang lebih pintar.

Dan semua ini bisa kalian pelajari **dengan Python, Pandas, Keras, dan tools open source lain.**

Catatan

- **Machine Learning** = Komputer belajar dari data.
- **Supervised Learning** = ada guru, ada jawaban.
- **Unsupervised Learning** = mandiri, tanpa jawaban.
- Contoh di sekitar kita banyak banget: dari email, game, sampai belanja online.

Kasus Ilustrasi

Kasus 1: Membantu Guru Menilai Otomatis (Supervised Learning)

Bayangin guru punya data nilai tugas dan ujian. Biasanya, guru harus hitung manual. Capek kan?

Dengan *Machine Learning*, komputer bisa belajar dari data lama, lalu memprediksi nilai akhir siswa baru.

Ilustrasi:

- Input: nilai_tugas, nilai_ujian
- Output: nilai_akhir

Pseudocode:

```
data = load("nilai_siswa.csv")
model = train_supervised(data, label="nilai_akhir")
prediksi = model.predict([nilai_tugas, nilai_ujian])
print(prediksi)
```

Kasus 2: Filter Komentar di Media Sosial (Supervised Learning)

Pernah lihat komentar kasar di Instagram? Nah, ML bisa jadi “penjaga gerbang” komentar.

Komputer belajar dari contoh komentar sopan vs komentar kasar. Nanti kalau ada komentar baru, bot bisa memutuskan komentar itu harus tampil atau diblokir.

Ilustrasi:

- Input: teks komentar
- Output: “aman” atau “kasar”

Pseudocode:

```
data = load("komentar.csv")
model = train_supervised(data, label="kategori")
cek = model.predict(["Dasar noob!"])
print(cek)    # Output: kasar
```

Kasus 3: Rekomendasi Film di Netflix (Unsupervised Learning)

Netflix nggak nanya kamu suka genre apa satu per satu.

Dia cukup lihat film apa saja yang kamu tonton, lalu mengelompokkan dengan orang-orang yang punya selera mirip. Dari situ, muncul rekomendasi: “Karena kamu nonton A, coba juga nonton B.”

Ilustrasi:

- Input: riwayat tontonan
- Output: kelompok penonton

Pseudocode:

```
data = load("riwayat_tonton.csv")
grup = cluster_unsupervised(data, method="kmeans")
print(grup[user])
```

Kasus 4: Mengelompokkan Siswa untuk Proyek (Unsupervised Learning)

Guru punya data hobi dan pelajaran favorit siswa.

Daripada pilih kelompok asal-asalan, ML bisa otomatis mengelompokkan siswa dengan kesukaan mirip. Jadi proyek jadi lebih seru dan efektif.

Ilustrasi:

- Input: hobi, pelajaran_favorit
- Output: kelompok siswa

Pseudocode:

```
data = load("data_siswa.csv")
grup = cluster_unsupervised(data, method="kmeans")
print("Kelompok 1:", grup[1])
print("Kelompok 2:", grup[2])
```

Kasus 5: Deteksi Penipuan Kartu Kredit (Supervised Learning)

Bank harus hati-hati dengan transaksi aneh.

ML bisa belajar dari data transaksi lama: mana transaksi normal, mana yang mencurigakan.

Kalau ada pembelian aneh, sistem langsung kasih alert.

Ilustrasi:

- Input: jumlah_transaksi, lokasi, waktu
- Output: normal atau penipuan

Pseudocode:

```
data = load("transaksi.csv")
model = train_supervised(data, label="status")
cek = model.predict([jumlah, lokasi, waktu])
print(cek)    # Output: penipuan
```

Soal Latihan & Pembahasan

Soal 1: Foto Kucing & Anjing

Kamu punya 1000 foto kucing dan anjing. Setiap foto sudah diberi label: “*kucing*” atau “*anjing*”.

Tugasmu: ajari komputer supaya bisa menebak foto baru itu kucing atau anjing.

Pseudocode:

```
data = load("foto_kucing_dan_anjing.csv")
model = train_supervised(data)
hasil = model.predict("foto_baru.jpg")
```

Jawaban & Pembahasan:

Ini termasuk **Supervised Learning**, karena data sudah punya label. Komputer belajar dari contoh yang ada, lalu bisa memprediksi foto baru.

Soal 2: Playlist Lagu Favorit

Bayangkan kamu punya 500 lagu di HP, tapi tidak ada label genre (pop, rock, jazz). Kamu ingin komputer mengelompokkan lagu berdasarkan pola.

Pseudocode:

```
data = load("lagu_favorit.csv")
kelompok = cluster_unsupervised(data, k=3)
print(kelompok)
```

Jawaban & Pembahasan:

Ini **Unsupervised Learning**, karena datanya tidak ada label. Komputer harus cari pola sendiri, lalu membagi lagu ke dalam grup.

Soal 3: Prediksi Nilai Siswa

Guru punya data nilai ujian matematika dan IPA. Ada juga kolom “Lulus” atau “Tidak”. Guru ingin komputer bisa menebak apakah siswa baru lulus atau tidak.

Pseudocode:

```
data = load("nilai_siswa.csv")
model = train_supervised(data, label="status_lulus")
hasil = model.predict({"matematika": 80, "ipa": 75})
```

Jawaban & Pembahasan:

Ini **Supervised Learning**, karena ada data berlabel “*lulus/tidak lulus*”. Tujuannya untuk prediksi.

Soal 4: Rekomendasi Game Online

Sebuah game online punya data tentang kebiasaan main pemain: berapa lama mereka main, senjata favorit, dan level kesulitan yang sering dipilih. Tidak ada label. Komputer diminta mengelompokkan pemain yang mirip.

Pseudocode:

```
data = load("data_pemain.csv")
kelompok = cluster_unsupervised(data, k=4)
print(kelompok)
```

Jawaban & Pembahasan:

Ini **Unsupervised Learning**, karena tidak ada label. Komputer membuat kelompok otomatis agar game bisa kasih rekomendasi senjata atau mode permainan.

Soal 5: Deteksi Spam Email

Kamu punya ribuan email. Beberapa sudah ditandai “spam”, sisanya “aman”. Kamu ingin komputer otomatis menandai email baru.

Pseudocode:

```
data = load("email_data.csv")
model = train_supervised(data, label="kategori")
hasil = model.predict("email_baru.txt")
```

Jawaban & Pembahasan:

Ini **Supervised Learning**, karena data sudah punya label “spam/aman”. Tujuannya klasifikasi.

Fun Facts!

- **Ollama** itu keren banget, karena kalian bisa pakai AI kelas ChatGPT di laptop kalian sendiri, tanpa internet!
- **RAG (Retrieval-Augmented Generation)** itu bikin AI bisa "membaca" dokumen di komputermu. Jadi, kalian bisa kasih AI dokumen materi SMK, dan dia bisa jawab pertanyaan tentang materi itu!
- **Orange Data Mining** punya fitur yang bikin kalian bisa coba Machine Learning tanpa perlu nulis kode sama sekali, tinggal klik dan seret!

Tips Belajar & Menghafal Cepat

- **Pikirkan Analogi:** Kalau bingung, ingat analogi "Guru vs Mandiri" atau "Gambar Berlabel vs Gambar Tanpa Label".

- **Buat Mind Map:** Gambar peta konsep sederhana di buku catatan kalian. Tarik garis dan kasih contoh.
- **Cari Contoh di Sekitar:** Coba pikirkan, "Algoritma di TikTok ini pakai Supervised atau Unsupervised, ya?"

Aktivitas Siswa

Aktivitas 1: Bedakan Kucing & Anjing (Supervised)

Tugas:

Latih komputer membedakan gambar kucing 🐱 dan anjing 🐶 dengan data berlabel.

Langkah:

1. Siapkan dataset sederhana (`data_kucing_anjing.csv`) berisi nama file dan label (*kucing / anjing*).
2. Gunakan *Python Pandas* untuk membaca data.
3. Latih model *Decision Tree* sederhana.
4. Prediksi gambar baru.

Pseudocode:

```
data = load("data_kucing_anjing.csv")
model = train_supervised(data)
prediksi = model.predict("foto_baru.jpg")
```

Contoh Python:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

# 1. Baca data
data = pd.DataFrame({
    "ukuran": [10, 20, 15, 30],
    "suara": [1, 0, 1, 0], # 1 = meong, 0 = guk
    "label": ["kucing", "anjing", "kucing", "anjing"]
})

# 2. Latih model
X = data[["ukuran", "suara"]]
y = data["label"]
model = DecisionTreeClassifier().fit(X, y)

# 3. Prediksi data baru
print(model.predict([[12, 1]])) # prediksi: kucing
```

Aktivitas 2: Kelompokkan Lagu Favorit (Unsupervised)

Tugas:

Pisahkan lagu berdasarkan pola tanpa label genre.

Langkah:

1. Buat data sederhana tentang tempo dan durasi lagu.
2. Gunakan *K-Means clustering*.
3. Lihat hasil pengelompokan.

Pseudocode:

```
data = load("playlist.csv")
kelompok = cluster_unsupervised(data, k=3)
show(kelompok)
```

Contoh Python:

```
import pandas as pd
from sklearn.cluster import KMeans

# 1. Data sederhana
lagu = pd.DataFrame({
    "tempo": [120, 140, 90, 100, 150],
    "durasi": [3, 4, 5, 3, 4]
})

# 2. Kelompokkan
kmeans = KMeans(n_clusters=2, random_state=0).fit(lagu)

# 3. Lihat cluster
print(kmeans.labels_)
```

Aktivitas 3: Prediksi Kelulusan (Supervised)**Tugas:**

Prediksi apakah siswa lulus berdasarkan nilai matematika & IPA.

Langkah:

1. Siapkan dataset `nilai_siswa.csv` dengan kolom: `matematika`, `ipa`, `lulus`.
2. Gunakan *Logistic Regression*.
3. Prediksi data siswa baru.

Pseudocode:

```
data = load("nilai_siswa.csv")
model = train_supervised(data, label="lulus")
hasil = model.predict({"matematika":80, "ipa":75})
```

Contoh Python:

```

import pandas as pd
from sklearn.linear_model import LogisticRegression

# 1. Data contoh
data = pd.DataFrame({
    "matematika": [80, 60, 90, 50],
    "ipa": [85, 55, 88, 40],
    "lulus": [1, 0, 1, 0]
})

# 2. Latih model
X = data[["matematika", "ipa"]]
y = data["lulus"]
model = LogisticRegression().fit(X, y)

# 3. Prediksi siswa baru
print(model.predict([[75, 70]])) # 1 = lulus

```

Aktivitas 4: Kelompokkan Siswa Berdasarkan Hobi (Unsupervised)

Tugas:

Pisahkan siswa berdasarkan kesukaan (gaming, musik, olahraga).

Langkah:

1. Buat dataset sederhana **hobi.csv**.
2. Gunakan *K-Means* untuk mengelompokkan.
3. Lihat hasil cluster.

Pseudocode:

```

data = load("hobi_siswa.csv")
kelompok = cluster_unsupervised(data, k=3)
show(kelompok)

```

Contoh Python:

```

import pandas as pd
from sklearn.cluster import KMeans

# 1. Data sederhana
siswa = pd.DataFrame({
    "gaming": [1, 0, 1, 0, 1],
    "musik": [0, 1, 1, 0, 1],
    "olahraga": [0, 1, 0, 1, 0]
})

# 2. Kelompokkan
kmeans = KMeans(n_clusters=2, random_state=0).fit(siswa)

# 3. Lihat hasil cluster

```

```
print(kmeans.labels_)
```

Aktivitas 5: Deteksi Spam Email (Supervised)

Tugas:

Latih model untuk memisahkan email spam dan aman.

Langkah:

1. Buat dataset `email.csv` dengan kolom: `panjang_kata`, `ada_link`, `label`.
2. Gunakan *Naive Bayes*.
3. Prediksi email baru.

Pseudocode:

```
data = load("email.csv")
model = train_supervised(data, label="spam")
hasil = model.predict("email_baru.txt")
```

Contoh Python:

```
import pandas as pd
from sklearn.naive_bayes import GaussianNB

# 1. Data contoh
email = pd.DataFrame({
    "panjang_kata": [100, 20, 300, 50],
    "ada_link": [0, 1, 1, 0],
    "label": ["aman", "spam", "spam", "aman"]
})

# 2. Latih model
X = email[["panjang_kata", "ada_link"]]
y = email["label"]
model = GaussianNB().fit(X, y)

# 3. Prediksi email baru
print(model.predict([[40, 1]])) # kemungkinan spam
```

Rangkuman

Berangkat dari hal paling penting dulu: **Machine Learning** adalah cara komputer **belajar dari data**. Bukan hanya nurut perintah. Ia **mengamati contoh**, lalu **membuat keputusan**. Kalian sering memakainya tanpa sadar: rekomendasi video, deteksi **spam**, hingga rute **Google Maps**. Ingat kata kunci: **data** → **belajar** → **keputusan**. Dalam ML ada banyak *istilah asing* seperti *model*, *fit*, *prediksi*, tapi tenang—kita pakai contoh nyata dan kalimat sederhana.

Dua gaya belajar utama itu **Supervised Learning** dan **Unsupervised Learning**. **Supervised** = ada **label** dan ada “guru”. Cocok untuk **klasifikasi** (kucing/anjing, spam/aman) atau **prediksi** (harga rumah). **Unsupervised** = **tanpa label**, komputer mencari **pola** dan **kelompok** sendiri. Cocok untuk **clustering** pelanggan atau **rekomendasi** musik. Rumus cepatnya: **Supervised** = **ada jawaban**, **Unsupervised** = **cari pola**. Pilih sesuai tujuan: mau **menebak** sesuatu? Pakai **Supervised**. Mau **mengelompokkan** data? Pakai **Unsupervised**.

Semua ini bisa kalian praktekan dengan alat *open source*: **Python**, **Pandas**, **Keras**, dan *neural network*. Alurnya simpel: **siapkan data** (bersih dan rapi), **latih model** (*train*), lalu **uji** (*test*) dan **pakai** untuk keputusan. Mulai dari kecil dulu: dataset mini, fitur sedikit, kode pendek. Lalu bertahap tambah data, tambah fitur, dan evaluasi metrik. Kalian siap jadi pembuat solusi, bukan sekadar pengguna. Kuncinya: **coba**, **lihat hasil**, **perbaiki**—ulang terus sampai mantap.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. Machine Learning membuat komputer belajar dari data.
Jawaban: Benar
2. *Unsupervised learning* membutuhkan label pada setiap data.
Jawaban: Salah
3. *Supervised learning* cocok untuk klasifikasi spam atau tidak spam.
Jawaban: Benar
4. Pandas dipakai untuk membuat *neural network*.
Jawaban: Salah
5. Keras adalah library *open source* untuk membangun *neural network*.
Jawaban: Benar

B. Pilihan Ganda (10 Soal)

1. Data berlabel biasanya dipakai di:
A. *Unsupervised*
B. *Supervised*
C. *Reinforcement*
D. *Rule-based*
Jawaban: B
2. Tujuan utama *unsupervised learning* adalah:
A. Prediksi nilai ujian
B. Menentukan label yang benar

- C. Mencari pola/kelompok
 - D. Menghapus data kosong
- Jawaban: C

3. Library Python untuk olah data tabel:

- A. NumPy
- B. Pandas
- C. Matplotlib
- D. Flask

Jawaban: B

4. Contoh tugas klasifikasi:

- A. Mengelompokkan pelanggan tanpa label
- B. Menebak genre musik tanpa label
- C. Menandai email spam atau aman
- D. Menghitung rata-rata nilai

Jawaban: C

5. Baris kode Pandas untuk membaca CSV:

- A. `pd.read_txt("a.csv")`
- B. `pandas.readcsv("a.csv")`
- C. `pd.read_csv("a.csv")`
- D. `pd.csv_read("a.csv")`

Jawaban: C

6. Model yang sering dipakai untuk klasifikasi sederhana:

- A. Linear Regression
- B. K-Means
- C. Decision Tree
- D. PCA

Jawaban: C

7. *Neural network* biasanya dibangun dengan:

- A. Keras
- B. Requests
- C. OS
- D. CSV

Jawaban: A

8. K-Means termasuk jenis:

- A. *Supervised*
- B. *Unsupervised*
- C. *Reinforcement*
- D. *Semi-supervised*

Jawaban: B

9. Untuk memecah data menjadi fitur X dan label y pada Pandas:
- A. `X = df.drop("label", axis=1); y = df["label"]`
 - B. `X = df["label"]; y = df.drop("label", axis=1)`
 - C. `X = df; y = df`
 - D. `X = label; y = fitur`
- Jawaban: A

10. Contoh output klasifikasi biner:
- A. 3 cluster
 - B. Harga rumah
 - C. 0 atau 1
 - D. Gambar berwarna
- Jawaban: C

C. Isian Singkat (5 Soal)

1. ML dengan data berlabel disebut _____.
Jawaban: Supervised learning
2. Algoritma untuk mengelompokkan data tanpa label: _____.
Jawaban: K-Means
3. Library untuk membangun *neural network* di Python: _____.
Jawaban: Keras
4. Perintah Pandas untuk menampilkan lima baris awal: _____.
Jawaban: `df.head()`
5. Vektor fitur disimpan dalam variabel ___, label disimpan dalam variabel ___ (konvensi umum).
Jawaban: X, y

D. Soal Explorasi (3 Soal)

Soal 1:

Buat skrip kecil yang membaca `nilai.csv` berisi kolom `matematika`, `ipa`, `lulus` lalu latih *Decision Tree* untuk memprediksi `lulus`. Tampilkan prediksi untuk data baru `matematika=80`, `ipa=70`.

Kunci Jawaban (contoh):

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

df = pd.read_csv("nilai.csv")
```

```
X = df.drop("lulus", axis=1)
y = df["lulus"]
model = DecisionTreeClassifier().fit(X, y)
print(model.predict([[80, 70]]))
```

Soal 2:

Kamu punya **playlist.csv** berisi **tempo**, **durasi**. Lakukan *clustering* 2 kelompok dengan K-Means, lalu tulis satu kalimat ringkas yang menjelaskan pola tiap cluster (misal: cepat/pendek vs lambat/panjang).

Kunci Jawaban (contoh):

```
import pandas as pd
from sklearn.cluster import KMeans

df = pd.read_csv("playlist.csv")
km = KMeans(n_clusters=2, random_state=0).fit(df)
df["cluster"] = km.labels_
# Contoh kalimat ringkas (sesuaikan hasil):
# Cluster 0: tempo cepat, durasi pendek; Cluster 1: tempo lebih
lambat, durasi lebih panjang.
```

Soal 3:

Rancang *baseline neural network* sederhana dengan Keras untuk klasifikasi biner dari **fitur1**, **fitur2**, **label** pada **data.csv**. Buat arsitektur minimal (Dense 8, Dense 1 sigmoid), latih singkat, dan tampilkan akurasi.

Kunci Jawaban (contoh):

```
import pandas as pd
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers

df = pd.read_csv("data.csv")
X = df[["fitur1", "fitur2"]].values
y = df["label"].values

model = keras.Sequential([
    layers.Dense(8, activation="relu", input_shape=(2,)),
    layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="adam", loss="binary_crossentropy",
metrics=["accuracy"])
history = model.fit(X, y, epochs=10, batch_size=8, verbose=0)
print(model.evaluate(X, y, verbose=0)[1])
```

Catatan Mini (opsional dipakai siswa saat latihan)

- *Supervised* → pakai label; *Unsupervised* → tanpa label.
- Pandas: **read_csv**, **head**, **drop**.

- Model cepat: *Decision Tree* (klasifikasi), K-Means (*clustering*), Keras untuk *neural network*.

Tugas Proyek (5 Soal)

Proyek 1: Prediksi Cuaca Sederhana

Tugas:

Buat program untuk memprediksi apakah besok **hujan** atau **tidak** berdasarkan suhu dan kelembaban hari ini.

Langkah:

1. Siapkan dataset **cuaca.csv** (suhu, kelembaban, hujan/tdk).
2. Gunakan *Supervised Learning* dengan **Decision Tree**.
3. Prediksi data baru: suhu 28°C, kelembaban 80%.

Pseudocode:

```
data = load("cuaca.csv")
model = train_supervised(data, label="hujan")
prediksi = model.predict({"suhu":28, "kelembaban":80})
```

Contoh Python:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

data = pd.DataFrame({
    "suhu": [25, 30, 28, 22],
    "kelembaban": [70, 60, 80, 90],
    "hujan": ["ya", "tidak", "ya", "ya"]
})

X = data[["suhu", "kelembaban"]]
y = data["hujan"]
model = DecisionTreeClassifier().fit(X, y)

print(model.predict([[28, 80]])) # Output: ['ya']
```

Output:

```
['ya']
```

Proyek 2: Kelompokkan Film Favorit

Tugas:

Kamu punya data film berdasarkan durasi dan rating. Kelompokkan film jadi 2 grup: “Ringan” dan “Serius”.

Langkah:

1. Siapkan dataset `film.csv` (durasi, rating).
2. Gunakan *Unsupervised Learning* dengan **K-Means**.
3. Lihat hasil pengelompokan.

Pseudocode:

```
data = load("film.csv")
kelompok = cluster_unsupervised(data, k=2)
print(kelompok)
```

Contoh Python:

```
import pandas as pd
from sklearn.cluster import KMeans

film = pd.DataFrame({
    "durasi": [90, 120, 150, 80],
    "rating": [7, 9, 8, 6]
})

kmeans = KMeans(n_clusters=2, random_state=0).fit(film)
print(kmeans.labels_)
```

Output:

```
[1 0 0 1]
```

Proyek 3: Prediksi Nilai Olahraga

Tugas:

Prediksi apakah siswa akan dapat nilai **A** atau **B** di pelajaran olahraga, berdasarkan *lama latihan* dan *waktu tidur*.

Langkah:

1. Buat dataset `olahraga.csv` (latihan, tidur, nilai).
2. Gunakan *Logistic Regression*.
3. Prediksi siswa baru: latihan 5 jam, tidur 6 jam.

Pseudocode:

```
data = load("olahraga.csv")
model = train_supervised(data, label="nilai")
prediksi = model.predict({"latihan":5, "tidur":6})
```

Contoh Python:

```
import pandas as pd
from sklearn.linear_model import LogisticRegression

data = pd.DataFrame({
    "latihan": [2, 4, 6, 5],
    "tidur": [5, 7, 8, 6],
    "nilai": ["B", "A", "A", "B"]
})

X = data[["latihan", "tidur"]]
y = data["nilai"]
model = LogisticRegression().fit(X, y)

print(model.predict([[5, 6]])) # Output: ['B']
```

Output:

```
['B']
```

Proyek 4: Kelompokkan Warna Favorit

Tugas:

Kamu punya data tentang preferensi warna siswa (merah, biru, hijau). Buat clustering siapa yang selera warnanya mirip.

Langkah:

1. Buat dataset **warna.csv** dengan nilai biner (0/1).
2. Gunakan *K-Means* untuk kelompokkan.
3. Tampilkan hasil cluster.

Pseudocode:

```
data = load("warna.csv")
kelompok = cluster_unsupervised(data, k=3)
print(kelompok)
```

Contoh Python:

```
import pandas as pd
from sklearn.cluster import KMeans

warna = pd.DataFrame({
    "merah": [1, 0, 1, 0, 1],
    "biru": [0, 1, 1, 0, 0],
})
```

```

        "hijau": [0, 1, 0, 1, 0]
    })

    kmeans = KMeans(n_clusters=3, random_state=0).fit(warna)
    print(kmeans.labels_)

```

Output:

```
[0 2 0 1 0]
```

Proyek 5: Prediksi Pola Tidur

Tugas:

Prediksi apakah siswa akan **segar** atau **ngantuk** di kelas, berdasarkan jam tidur dan jam bangun.

Langkah:

1. Buat dataset **tidur.csv** (jam_tidur, jam_bangun, kondisi).
2. Gunakan *Decision Tree*.
3. Prediksi siswa baru: tidur jam 23.00, bangun jam 05.00.

Pseudocode:

```

data = load("tidur.csv")
model = train_supervised(data, label="kondisi")
prediksi = model.predict({"tidur":23, "bangun":5})

```

Contoh Python:

```

import pandas as pd
from sklearn.tree import DecisionTreeClassifier

data = pd.DataFrame({
    "tidur": [22, 23, 0, 21],
    "bangun": [6, 5, 7, 4],
    "kondisi": ["segar", "ngantuk", "ngantuk", "segar"]
})

X = data[["tidur", "bangun"]]
y = data["kondisi"]
model = DecisionTreeClassifier().fit(X, y)

print(model.predict([[23, 5]])) # Output: ['ngantuk']

```

Output:

```
['ngantuk']
```

Catatan Akhir

- **Supervised = ada label** → prediksi atau klasifikasi.
- **Unsupervised = tanpa label** → kelompokkan pola.
- Semua proyek bisa dijalankan dengan **Python + Pandas + Scikit-learn**. Gratis, *open source*, dan fun buat dicoba di lab SMK.

BAB 3: Eksperimen dengan Orange Data Mining (Klasifikasi & Clustering)

Halo, teman-teman SMK! Sekarang, kita akan coba langsung praktekkan ilmu **Machine Learning** yang sudah kita pelajari di Orange Data Mining. Kali ini, kita bakal jadi ahli detektif yang bisa mengelompokkan data dan memprediksi sesuatu. Siap?

Tujuan Pembelajaran

Setelah sesi ini, kalian akan bisa:

- Menggunakan **Orange Data Mining** untuk membuat model **Klasifikasi** dan **Clustering**.
- Membedakan hasil dari kedua metode tersebut.
- Menerapkan kedua metode ini pada dataset nyata.
- Menginterpretasikan hasil visualisasinya.

Peta Konsep Sederhana

Bayangin kita lagi mengelompokkan komik dan buku di perpustakaan.

- **Klasifikasi (Supervised)**: Kamu punya rak komik dan buku yang sudah diberi label. Tugasmu cuma memastikan semua komik masuk ke rak komik dan semua buku ke rak buku. Ini kayak punya **tujuan jelas**.
- **Clustering (Unsupervised)**: Kamu punya tumpukan buku yang belum diapa-apain. Tugasmu adalah mengelompokkan buku-buku yang mirip (misal: semua buku horor jadi satu, semua buku komedi jadi satu), tanpa ada label sebelumnya. Ini seperti mencari **pola tersembunyi**.

Apersepsi: Kamu Mau Prediksi Pemenang Turnamen Game!

Ada turnamen game Mobile Legends di sekolahmu. Kamu mau tahu, tim mana yang bakal jadi juara.

- **Dengan Klasifikasi**: Kamu punya data historis pertandingan (jumlah menang, jumlah kalah, KDA). Kamu bisa mengajari model untuk memprediksi, "Tim yang punya KDA di atas 50% akan menang."
- **Dengan Clustering**: Kamu punya data 10 tim. Kamu nggak tahu siapa yang bakal menang. Kamu cuma mau tahu, tim mana aja yang punya gaya main mirip. Hasilnya, mungkin akan ada 3 kelompok: kelompok **agresif**, **defensif**, dan **campuran**.

Keren kan? Kita bisa pakai Orange buat melakukan itu semua!

Penjelasan Teori

Teman-teman SMK, siap jadi *detektif data*?

Kali ini kita akan main dengan **Orange Data Mining**, sebuah software open source yang super keren.

Dengan Orange, kalian bisa belajar *Machine Learning* tanpa harus nulis kode panjang-panjang. Tinggal tarik, lepas, sambung, dan... hasil analisisnya keluar!

Nah, dua hal utama yang kita pelajari di sini adalah:

- **Klasifikasi (Classification)** → memprediksi kategori berdasarkan data berlabel.
- **Clustering (Pengelompokan)** → mencari pola tersembunyi dalam data yang belum berlabel.

Konsep Dasar (Teori Ringan)

Bayangin kalian lagi di perpustakaan sekolah:

- **Klasifikasi** → Semua buku sudah ada labelnya: “Komik” atau “Novel.” Tugasmu hanya memastikan buku baru masuk ke rak yang benar.
- **Clustering** → Ada tumpukan buku tanpa label. Kamu diminta mengelompokkan berdasarkan kesamaan: horor, komedi, atau romantis.

Kata kunci gampang:

- Klasifikasi → *prediksi kategori* (butuh label).
- Clustering → *cari pola mirip* (tanpa label).

Kenapa Penting?

- Dunia kerja butuh prediksi: contoh apakah pelanggan akan beli atau tidak (klasifikasi).
- Dunia nyata penuh pola tersembunyi: contoh kelompok siswa berdasarkan hobi (clustering).
- Dengan Orange, kita bisa praktek *Machine Learning* secepat main *drag & drop*.

Alat yang Dipakai di Orange

- **Klasifikasi:** Widget File, Data Table, Test & Score, dan model (Logistic Regression, Tree, Naive Bayes).
- **Clustering:** Widget File, Data Table, K-Means, dan Scatter Plot.

Rangkuman

- **Klasifikasi = prediksi kategori (butuh label).**
- **Clustering = cari pola mirip (tanpa label).**
- Orange bikin semuanya mudah: tarik, sambung, lihat hasil.

Jadi, bukan hanya teori. Langsung praktek, seru, dan bikin kalian makin dekat jadi **data scientist masa depan!**

Kasus Ilustrasi

Kasus 1: Klasifikasi Genre Film

Bayangin kamu punya data film. Ada jumlah adegan aksi, jumlah adegan komedi, dan label genre.

Model akan belajar dari data lama. Lalu, kalau ada film baru, dia bisa nebak genrenya.

Pseudocode:

```
data = load_csv("film.csv")
model = train_classifier(data.features, data.label)
prediksi = model.predict(film_baru)
```

Hasil: film baru bisa ditebak apakah **Action**, **Komedi**, atau **Romantis**.

Kasus 2: Clustering Pelanggan Kantin

Di kantin sekolah, ada data pembelian siswa: nasi, jajan, dan minuman. Tapi datanya nggak ada label. Jadi kita pakai *clustering* untuk cari pola.

Pseudocode:

```
data = load_csv("kantin.csv")
clusters = kmeans(data, k=3)
show(clusters)
```

Hasil: siswa terbagi jadi 3 kelompok → **Pembeli Nasi**, **Pembeli Jajan**, **Pembeli Minuman**.

Kasus 3: Klasifikasi Hasil Ujian

Kamu punya data siswa: nilai_tugas, nilai_ujian, dan label lulus/tidak. Model bisa belajar dari data lama, lalu prediksi siswa baru.

Pseudocode:

```
data = load_csv("nilai.csv")
model = train_classifier(data.features, data.label)
prediksi = model.predict(siswa_baru)
```

Hasil: model bisa bilang apakah siswa baru itu **Lulus**  atau **Tidak Lulus** .

Kasus 4: Clustering Hobi Siswa

Data berisi hobi siswa: `suka_musik`, `suka_olahraga`, `suka_game`.
Kita ingin mengelompokkan siswa yang mirip hobinya.

Pseudocode:

```
data = load_csv("hobi.csv")
clusters = kmeans(data, k=3)
show(clusters)
```

Hasil: ada grup **anak musik**, **anak olahraga**, dan **anak gamer**.

Kasus 5: Klasifikasi Tipe Makanan

Data makanan punya kolom rasa: `pedas`, `manis`, `asin`, dan label jenis makanan.
Model bisa belajar, lalu prediksi makanan baru.

Pseudocode:

```
data = load_csv("makanan.csv")
model = train_classifier(data.features, data.label)
prediksi = model.predict(makanan_baru)
```

Hasil: model bisa tebak apakah makanan baru cocok untuk pecinta **pedas**, **manis**, atau **asin**.

Kesimpulan

- **Klasifikasi** = prediksi kategori dengan *label*.
- **Clustering** = mengelompokkan data tanpa *label*.
- Di Orange Data Mining, semua ini bisa dilakukan hanya dengan **tarik-seret widget**.
- Ilustrasi sehari-hari bikin konsepnya gampang dipahami: film, kantin, ujian, hobi, dan makanan.

Soal Latihan & Pembahasan

Soal 1: Email Spam atau Bukan Spam

Kamu punya dataset email dengan label: *spam* atau *bukan spam*.
Tugas: memprediksi email baru.

Pseudocode:

```
data = load_csv("email.csv")
model = train_classifier(data.features, data.label)
prediksi = model.predict(email_baru)
```

Jawaban:

Metode yang dipakai = **Klasifikasi**.

Pembahasan:

Karena datanya sudah ada label (spam atau bukan spam), maka model belajar dari contoh lama. Ini seperti guru yang sudah tahu mana PR benar atau salah, lalu bisa menilai PR murid baru.

Soal 2: Mengelompokkan Buku di Perpustakaan

Ada data rating buku dari 1–5, tapi tidak ada label genre.
Kamu ingin dikelompokkan jadi 3 grup berdasarkan kemiripan.

Pseudocode:

```
data = load_csv("buku.csv")
clusters = kmeans(data, k=3)
show(clusters)
```

Jawaban:

Metode yang dipakai = **Clustering**.

Pembahasan:

Karena tidak ada label (genre buku), maka model hanya mencari pola mirip. Hasilnya bisa saja: kelompok buku serius, buku ringan, dan buku populer.

Soal 3: Prediksi Kelulusan Siswa

Ada data siswa dengan kolom: nilai_tugas, nilai_ujian, label lulus/tidak.
Kamu ingin memprediksi siswa baru.

Pseudocode:

```
data = load_csv("nilai.csv")
model = train_classifier(data.features, data.label)
prediksi = model.predict(siswa_baru)
```

Jawaban:

Metode yang dipakai = **Klasifikasi**.

Pembahasan:

Karena ada label *lulus* atau *tidak*, ini jelas klasifikasi. Model belajar dari siswa sebelumnya, lalu bisa menilai siswa baru.

Soal 4: Mengelompokkan Hobi Siswa

Data siswa: suka_musik, suka_olahraga, suka_game. Tidak ada label.
Kamu ingin cari kelompok siswa dengan hobi mirip.

Pseudocode:

```
data = load_csv("hobi.csv")
clusters = kmeans(data, k=3)
show(clusters)
```

Jawaban:

Metode yang dipakai = **Clustering**.

Pembahasan:

Tidak ada label. Model hanya melihat pola. Hasilnya mungkin terbentuk kelompok: pecinta musik, pecinta olahraga, pecinta game.

Soal 5: Prediksi Tipe Makanan

Data makanan punya kolom: rasa_pedas, rasa_manis, rasa_asin, label selera. Kamu ingin memprediksi makanan baru.

Pseudocode:

```
data = load_csv("makanan.csv")
model = train_classifier(data.features, data.label)
prediksi = model.predict(makanan_baru)
```

Jawaban:

Metode yang dipakai = **Klasifikasi**.

Pembahasan:

Karena ada label selera (pedas, manis, asin), maka ini klasifikasi. Model akan belajar dari data lama lalu memprediksi makanan baru cocoknya ke mana.

Kesimpulan

- **Klasifikasi** dipakai jika ada **label** → contoh: spam/bukan spam, lulus/tidak, selera makanan.
- **Clustering** dipakai jika **tidak ada label** → contoh: kelompok buku, kelompok hobi.
- Di Orange, semua ini mudah banget. Tinggal tarik-seret widget, lalu lihat hasil visualisasi.

Fun Facts!

- **Orange Data Mining** punya lebih dari 100 widget yang berbeda-beda. Jadi, kalian bisa bikin alur kerja yang super kompleks.
- **Ollama** bisa diajarkan untuk bikin kode Python atau Orange. Jadi kalau kalian bingung, tinggal tanya aja.
- **OpenWebUI** adalah 'rumah' buat AI-mu. Dia bikin AI-mu gampang dipakai, seperti pakai ChatGPT.

Tips Belajar & Menghafal Cepat

- **Pakai Analoginya:** Selalu ingat analogi "klasifikasi itu memprediksi kategori," "clustering itu mengelompokkan."
- **Coba-Coba Sendiri:** Jangan takut salah! Coba ubah-ubah widget dan lihat hasilnya. Belajar paling efektif itu dari kesalahan.
- **Tonton Video Tutorial:** Banyak video tutorial Orange di YouTube yang bisa kalian tonton.

Aktivitas Siswa

Aktivitas 1: Klasifikasi Bunga dengan Dataset Iris

Tujuan: Melatih model untuk memprediksi jenis bunga dari ukuran kelopak & sepal.

Langkah-langkah:

1. Buka **Orange Data Mining**.
2. Tarik widget **File**, pilih dataset *iris.csv* (sudah bawaan Orange).
3. Sambungkan ke **Logistic Regression** lalu ke **Test & Score**.
4. Lihat hasil akurasi prediksi.

Pseudocode:

```
data = load_csv("iris.csv")
model = train_classifier(data.features, data.label)
prediksi = model.predict(data_baru)
```

Python Script:

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data,
                                                    iris.target, test_size=0.2)

model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
print("Akurasi:", model.score(X_test, y_test))
```

Output:

```
Akurasi: 0.9666666666666667
```

Aktivitas 2: Clustering Data Iris dengan K-Means

Tujuan: Mengelompokkan bunga tanpa label.

Langkah-langkah:

1. Tarik widget **File**, pilih *iris.csv*.
2. Hubungkan ke **K-Means**. Atur jumlah cluster = 3.
3. Sambungkan ke **Scatter Plot**.
4. Lihat hasil pengelompokan dengan warna berbeda.

Pseudocode:

```
data = load_csv("iris.csv")
clusters = kmeans(data, k=3)
show(clusters)
```

Python Script:

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

iris = load_iris()
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(iris.data)
print("Label Cluster:", kmeans.labels )
```

Output:

[illegible]

Aktivitas 3: Membuat Dataset Sendiri untuk Klasifikasi Nilai

Tujuan: Prediksi apakah siswa lulus berdasarkan nilai tugas & ujian.

Langkah-langkah:

1. Buat file Excel/CSV dengan kolom: `nilai_tugas`, `nilai_ujian`, `lulus`.
2. Simpan sebagai **nilai.csv**.
3. Impor ke Orange → hubungkan ke **Tree** dan **Test & Score**.
4. Lihat hasil prediksi.

Pseudocode:

```
data = load_csv("nilai.csv")
model = train_classifier(data.features, data.label)
hasil = model.predict(siswa_baru)
```

Python Script:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

data = pd.read_csv("nilai.csv")
X = data[['nilai_tugas', 'nilai_ujian']]
y = data['lulus']

model = DecisionTreeClassifier()
model.fit(X, y)
print("Prediksi siswa baru:", model.predict([[80, 75]]))
```

Aktivitas 4: Clustering Data Berat & Tinggi Badan

Tujuan: Mengelompokkan siswa berdasarkan berat & tinggi badan.

Langkah-langkah:

1. Buat file CSV dengan kolom: **berat**, **tinggi**.
2. Impor ke Orange → hubungkan ke **K-Means**.
3. Pilih jumlah cluster = 3.
4. Lihat hasil kelompok di **Data Table** atau **Scatter Plot**.

Pseudocode:

```
data = load_csv("siswa.csv")
clusters = kmeans(data, k=3)
print(clusters)
```

Python Script:

```
import pandas as pd
from sklearn.cluster import KMeans

data = pd.read_csv("siswa.csv")
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(data[['berat', 'tinggi']])
data['cluster'] = kmeans.labels_
print(data)
```

Aktivitas 5: Analisis Sederhana dengan Pandas

Tujuan: Belajar membersihkan & melihat data sebelum dipakai Orange.

Langkah-langkah:

1. Buat file CSV dengan data makanan (**pedas**, **manis**, **asin**, **label**).
2. Buka Python → gunakan Pandas untuk cek data.
3. Bersihkan data kosong (kalau ada).

4. Simpan kembali CSV untuk dipakai di Orange.

Pseudocode:

```
data = load_csv("makanan.csv")
clean_data = drop_missing(data)
save_csv(clean_data, "makanan_baru.csv")
```

Python Script:

```
import pandas as pd

data = pd.read_csv("makanan.csv")
print("Data Awal:")
print(data.head())

data = data.dropna()
print("Data Bersih:")
print(data.head())

data.to_csv("makanan_baru.csv", index=False)
```

Kesimpulan

- **Aktivitas ini melatih kalian langsung dengan data nyata.**
- Ada yang pakai Orange (tarik-seret widget), ada juga yang pakai Python.
- Semua berbasis open source: Python, Pandas, scikit-learn, Orange Data Mining.
- Dengan latihan ini, kalian bisa jadi **mini data scientist** di sekolah!

Rangkuman

Berani jadi detektif data? Di BAB ini kamu belajar dua hal inti: **klasifikasi** dan **clustering** di **Orange Data Mining**. **Klasifikasi** itu memprediksi **kategori** pakai data yang sudah **berlabel**. **Clustering** itu **mengelompokkan** data **tanpa label** untuk mencari pola mirip. Semua dilakukan dengan *drag-and-drop workflow*: **File** → **Model** → **Test & Score** untuk klasifikasi, dan **File** → **K-Means** → **Scatter Plot** untuk clustering. Visualnya jelas. Warna membantu kamu membaca pola. Ini cara cepat memahami *Machine Learning* tanpa pusing kode dulu.

Kamu tetap bisa pakai ekosistem **open source**. Data dibersihkan dengan **Python + Pandas**. Model bisa dibangun di Orange, lalu diuji lebih lanjut di Python. Kalau butuh model lebih canggih, kamu bisa coba *library* **scikit-learn** atau *framework* **Keras** untuk *neural network*—semuanya **gratis** dan **terbuka**. Intinya: **Orange** buat belajar konsep dan visual, **Python** buat eksperimen mendalam. Kombinasinya bikin kamu **paham konsep** sekaligus **terampil praktik**.

Ingat inti pelajaran: **Klasifikasi = prediksi kategori (butuh label)**, **Clustering = cari pola (tanpa label)**. Mulai dari contoh dekat: **email spam**, **kelulusan siswa**, **kelompok hobi**, atau **pembeli kantin**. Kerjakan langkah kecil: **siapkan data** → **bersihkan** → **pilih metode** → **jalankan** → **baca grafik**. Dengan kebiasaan mencoba (*trial and error*), kamu akan makin jago membaca pola dan mengambil keputusan berbasis data. Itulah skill penting di dunia industri AI/ML masa kini.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. Klasifikasi membutuhkan data yang sudah memiliki label.
Jawaban: Benar
2. Clustering digunakan untuk memprediksi kategori yang sudah diketahui sebelumnya.
Jawaban: Salah
3. Orange Data Mining bisa digunakan tanpa menulis kode.
Jawaban: Benar
4. Widget *K-Means* digunakan untuk membuat model klasifikasi.
Jawaban: Salah
5. Data nilai ujian siswa bisa dipakai untuk klasifikasi apakah lulus atau tidak.
Jawaban: Benar

B. Pilihan Ganda (10 Soal)

1. Metode yang dipakai untuk mengelompokkan data tanpa label adalah ...
 - a. Klasifikasi
 - b. Clustering
 - c. Regresi
 - d. Sorting**Jawaban: b. Clustering**
2. Untuk memprediksi apakah email spam atau bukan spam digunakan metode ...
 - a. Clustering
 - b. Sorting
 - c. Klasifikasi
 - d. Indexing**Jawaban: c. Klasifikasi**
3. Widget yang digunakan untuk evaluasi model klasifikasi di Orange adalah ...
 - a. File

- b. Data Table
- c. Test & Score
- d. Scatter Plot

Jawaban: c. Test & Score

4. Apa yang dilakukan *Scatter Plot*?
- a. Menghapus data
 - b. Menampilkan hasil dalam bentuk grafik
 - c. Membuat label
 - d. Melatih model

Jawaban: b. Menampilkan hasil dalam bentuk grafik

5. Contoh data yang cocok untuk clustering adalah ...
- a. Email spam/tidak spam
 - b. Hobi siswa
 - c. Label ujian lulus/tidak
 - d. Genre film sudah diketahui

Jawaban: b. Hobi siswa

6. Python library yang umum dipakai untuk membaca data tabel adalah ...
- a. NumPy
 - b. Pandas
 - c. Orange3
 - d. TensorFlow

Jawaban: b. Pandas

7. Pseudocode untuk klasifikasi:

```
model = train_classifier(data.features, data.label)
```

Fungsi `train_classifier` dipakai untuk ...

- a. Membersihkan data
- b. Melatih model dengan data berlabel
- c. Menampilkan visualisasi
- d. Menyimpan file CSV

Jawaban: b. Melatih model dengan data berlabel

8. Dalam clustering, jika kita menentukan `k=3`, artinya ...
- a. Ada 3 dataset
 - b. Ada 3 kelompok hasil clustering
 - c. Ada 3 label sudah diketahui
 - d. Ada 3 model berbeda

Jawaban: b. Ada 3 kelompok hasil clustering

9. Framework open source untuk *deep learning* adalah ...
- a. Keras
 - b. MS Excel
 - c. SPSS

d. Tableau

Jawaban: a. Keras

10. Neural network dalam AI dipakai untuk ...

- a. Mengelompokkan file komputer
- b. Meniru cara kerja otak manusia
- c. Membuat tabel data
- d. Menyortir dokumen

Jawaban: b. Meniru cara kerja otak manusia

C. Isian Singkat (5 Soal)

1. Sebutkan dua metode utama yang dipelajari di Orange Data Mining!

Jawaban: Klasifikasi dan Clustering

2. Tuliskan satu contoh kasus nyata penggunaan klasifikasi!

Jawaban: Prediksi email spam atau bukan spam

3. Apa nama widget di Orange yang digunakan untuk menampilkan data dalam bentuk tabel?

Jawaban: Data Table

4. Library Python yang biasa digunakan untuk membuat model klasifikasi seperti Logistic Regression adalah ...

Jawaban: scikit-learn

5. Apa output utama dari algoritma K-Means?

Jawaban: Kelompok (cluster) data berdasarkan kesamaan

D. Soal Explorasi (3 Soal)

Soal 1: Eksperimen Klasifikasi Film

Buat dataset sederhana berisi `jumlah_action`, `jumlah_komedi`, dan `genre`.
Gunakan Orange atau Python untuk membuat model klasifikasi yang bisa memprediksi genre film baru.

Contoh jawaban siswa:

- Buat file `film.csv`
- Impor ke Orange → Logistic Regression → Test & Score
- Prediksi film baru: `jumlah_action=5`, `jumlah_komedi=2` → hasilnya "Action"

Soal 2: Clustering Gaya Main Game

Kamu punya data 10 tim dengan kolom **kill**, **assist**, **death**. Gunakan K-Means dengan **k=3** untuk mengelompokkan gaya main tim.

Contoh jawaban siswa:

- Buat file **game.csv**
- Impor ke Orange → K-Means → Scatter Plot
- Hasil: tim terbagi jadi 3 kelompok → agresif, defensif, campuran

Soal 3: Klasifikasi Kelulusan Siswa

Buat dataset **nilai_tugas**, **nilai_ujian**, **lulus** (0 = tidak, 1 = ya). Bangun model untuk memprediksi siswa baru.

Contoh jawaban siswa:

- Buat file **siswa.csv**
- Impor ke Orange → Decision Tree → Test & Score
- Uji data baru: tugas=80, ujian=75 → prediksi: "Lulus"

Tugas Proyek (5 Soal)

Proyek 1: Klasifikasi Cuaca

Tantangan:

Buat dataset sederhana dengan kolom: **suhu**, **kelembapan**, **cuaca** (label: cerah / hujan).

Lalu buat model klasifikasi untuk memprediksi cuaca.

Langkah:

1. Buat file **cuaca.csv**.
2. Masukkan data suhu (angka), kelembapan (%), dan label cuaca.
3. Impor ke Orange. Sambungkan ke Logistic Regression → Test & Score.
4. Lihat hasil akurasi.

Pseudocode:

```
data = load_csv("cuaca.csv")
model = train_classifier(data.features, data.label)
prediksi = model.predict(cuaca_baru)
```

Python Script:

```
import pandas as pd
from sklearn.linear_model import LogisticRegression

data = pd.read_csv("cuaca.csv")
X = data[['suhu', 'kelembapan']]
y = data['cuaca']
```

```

model = LogisticRegression()
model.fit(X, y)
print("Prediksi:", model.predict([[30, 70]])) # suhu=30,
kelembapan=70%

```

Proyek 2: Clustering Waktu Belajar Siswa

Tantangan:

Kamu punya data **jam_belajar** dan **nilai**. Kelompokkan siswa jadi 3 cluster.

Langkah:

1. Buat file **belajar.csv**.
2. Masukkan data: jam_belajar/hari dan nilai ujian.
3. Impor ke Orange → K-Means → Scatter Plot.
4. Lihat hasil grouping.

Pseudocode:

```

data = load_csv("belajar.csv")
clusters = kmeans(data, k=3)
show(clusters)

```

Python Script:

```

import pandas as pd
from sklearn.cluster import KMeans

data = pd.read_csv("belajar.csv")
kmeans = KMeans(n_clusters=3, random_state=42)
data['cluster'] =
kmeans.fit_predict(data[['jam_belajar', 'nilai']])
print(data)

```

Proyek 3: Klasifikasi Status Gizi

Tantangan:

Buat dataset **berat**, **tinggi**, dan **status_gizi** (kurus / normal / gemuk).
Gunakan klasifikasi untuk prediksi status gizi siswa baru.

Langkah:

1. Buat file **gizi.csv**.
2. Masukkan data contoh 10–15 siswa.
3. Impor ke Orange → Decision Tree → Test & Score.
4. Uji dengan data baru.

Pseudocode:

```
data = load_csv("gizi.csv")
model = train_classifier(data.features, data.label)
prediksi = model.predict(siswa_baru)
```

Python Script:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

data = pd.read_csv("gizi.csv")
X = data[['berat', 'tinggi']]
y = data['status_gizi']

model = DecisionTreeClassifier()
model.fit(X, y)
print("Prediksi:", model.predict([[50, 160]])) # berat=50,
tinggi=160
```

Proyek 4: Clustering Pola Tidur

Tantangan:

Buat dataset **jam_tidur** dan **jam_bangun**. Kelompokkan siswa jadi tipe tidur: rajin, normal, begadang.

Langkah:

1. Buat file **tidur.csv**.
2. Masukkan jam_tidur (dalam 24 jam) dan jam_bangun.
3. Impor ke Orange → K-Means → Data Table.
4. Lihat cluster.

Pseudocode:

```
data = load_csv("tidur.csv")
clusters = kmeans(data, k=3)
print(clusters)
```

Proyek 5: Klasifikasi Mood Musik

Tantangan:

Buat dataset **tempo**, **energi**, **mood** (happy / sad / chill).
Gunakan klasifikasi untuk memprediksi mood lagu baru.

Langkah:

1. Buat file **musik.csv**.
2. Masukkan data tempo (BPM), energi (0–100), dan mood.
3. Impor ke Orange → Naive Bayes → Test & Score.
4. Coba prediksi lagu baru.

Pseudocode:

```
data = load_csv("musik.csv")
model = train_classifier(data.features, data.label)
prediksi = model.predict(lagu_baru)
```

Python Script:

```
import pandas as pd
from sklearn.naive_bayes import GaussianNB

data = pd.read_csv("musik.csv")
X = data[['tempo', 'energi']]
y = data['mood']

model = GaussianNB()
model.fit(X, y)
print("Prediksi:", model.predict([[120, 85]])) # tempo=120,
energi=85
```

Kesimpulan

- Proyek ini melatih kalian berpikir **seperti ilmuwan data**.
- **Klasifikasi** dipakai untuk *prediksi kategori*, **Clustering** untuk *mencari pola*.
- Semua bisa dikerjakan pakai **Orange Data Mining** atau **Python + Pandas + scikit-learn**.
- Semakin sering coba, semakin cepat paham cara kerja *Machine Learning*!

BAB 4: Supervised Learning dengan scikit-learn (Regresi Linier & Decision Tree)

Halo, teman-teman SMK! 💡 Kembali lagi, sekarang kita akan masuk ke materi paling seru: **membuat prediksi dengan Supervised Learning!** Kita bakal pakai **Python** dan **scikit-learn**, library super canggih buat bikin model cerdas.

Tujuan Pembelajaran

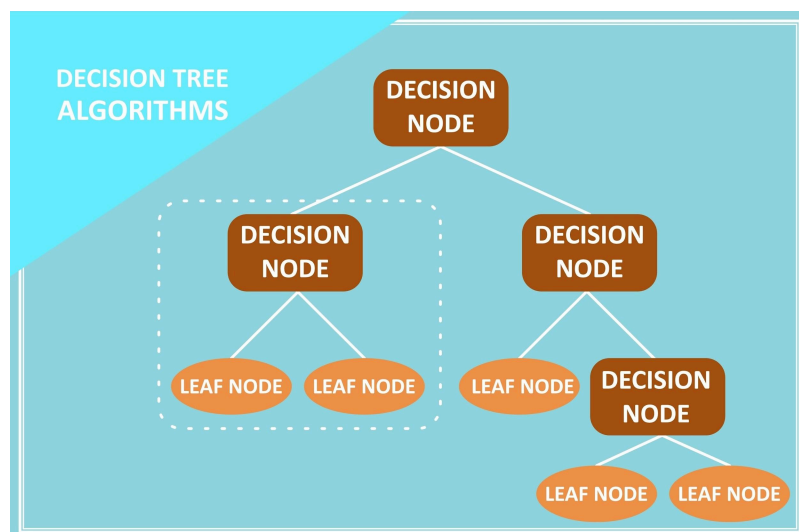
Setelah sesi ini, kalian bakal bisa:

- Memahami konsep **Supervised Learning** lebih dalam.
- Mengenal dua algoritma andalan: **Regresi Linier** dan **Decision Tree**.
- Membedakan kapan harus pakai Regresi Linier dan kapan pakai Decision Tree.
- Membuat model prediksi sederhana dengan **scikit-learn** dan **Python**.

Peta Konsep Sederhana

Bayangin kita mau memprediksi tinggi badan teman kita di masa depan. 🤖

- **Regresi Linier:** Kita pakai rumus matematika sederhana. $\text{Tinggi_masa_depan} = (\text{Tinggi_sekarang} \times 1.5) + 5$. Ini cuma tebakan berdasarkan garis lurus. Mirip kayak menebak nilai UTS dari nilai ulangan harianmu.
- **Decision Tree:** Kita pakai bagan keputusan. "Jika tingginya di atas 170cm, kemungkinan akan jadi pemain basket. Jika di bawah 170cm, cek lagi, apakah suka main game? Jika ya, kemungkinan akan jadi gamer." Ini kayak pohon percabangan yang membuat keputusan.



Dilisensikan oleh Google

Apersepsi: Kamu Mau Prediksi Harga Jual HP Bekas!

Kamu mau jual HP bekasmu. Tapi bingung, harganya berapa ya?

- **Cara Kuno:** Kamu tanya-tanya teman atau cek harga di forum. Hasilnya nggak pasti.
- **Dengan Regresi Linier:** Kamu kumpulkan data: harga jual HP bekas, usia HP, dan kondisi baterai. Kamu bisa buat model prediksi yang bisa menebak harganya. "HP-mu yang usianya 2 tahun dengan kondisi baterai 80%, harganya Rp2 juta."
- **Dengan Decision Tree:** Kamu buat bagan: "Jika kondisi baterai di atas 80% **DAN** layar tidak ada baret, **MAKA** harganya Rp2 juta. **JIKA TIDAK**, cek lagi, apakah ada dus?..."

Tuh, ada dua cara keren buat memprediksi!

Penjelasan Teori

Hari ini kita masuk ke bab yang seru banget: **membuat prediksi dengan komputer**. Iya, komputer bisa jadi seperti *ramalan cuaca* atau *kalkulator cerdas*. Semua itu ada di bidang yang disebut **Supervised Learning**. Kita akan belajar bareng pakai *Python* dan *scikit-learn* — dua *tool open source* yang gratis, keren, dan dipakai di seluruh dunia.

Apa Itu Supervised Learning?

Bayangin kamu punya guru les matematika. Guru itu selalu kasih soal dan jawabannya. Lama-lama, kamu hafal polanya, jadi bisa menjawab soal baru sendiri. Nah, komputer juga bisa begitu. Caranya:

- **Kasih data yang sudah ada jawabannya (data berlabel).**
- Komputer belajar dari pola data itu.
- Setelah belajar, komputer bisa **menebak jawaban** untuk data baru yang belum pernah dia lihat.

Jadi singkatnya: **Supervised Learning = Komputer belajar dari contoh.**

Dua Algoritma Andalan

Ada dua algoritma keren yang bakal kita pakai:

1. **Regresi Linier (Linear Regression)**
 - Fokus: memprediksi **angka** (nilai numerik).
 - Contoh: memprediksi harga rumah, berat badan, atau nilai ujian.
 - Cara kerja: mencari **garis lurus terbaik** yang pas dengan data.
 - Bayangin grafik dengan titik-titik data, lalu ditarik garis yang paling mewakili semuanya.
2. **Decision Tree (Pohon Keputusan)**
 - Fokus: memprediksi **kategori** (ya/tidak, lulus/tidak lulus) atau juga angka.

- Contoh: apakah siswa ikut futsal atau tidak, apakah pelanggan beli produk atau tidak.
- Cara kerja: membuat **pertanyaan bercabang** seperti pohon.
 - “Apakah baterai HP > 80%? Ya → harga Rp2 juta, Tidak → cek kondisi layar.”

Ilustrasi Sederhana

- **Regresi Linier**

Misal, kamu ingin tahu **nilai Ujian Akhir** dari rata-rata nilai tugas dan ulangan. Komputer bikin garis lurus yang bisa dipakai buat prediksi nilai.

- **Decision Tree**

Kamu mau tahu temanmu akan pilih **game RPG atau strategi**. Pohon keputusan bisa tanya:

- “Apakah dia suka tantangan sulit?”
- “Apakah dia suka cerita panjang?”

Kenapa Keren Buat SMK?

- Bisa dipakai buat prediksi bisnis kecil (contoh: prediksi penjualan kebab harian 🍴).
- Bisa bantu cari strategi marketing sederhana.
- Bisa dipakai buat belajar AI dengan *Python* tanpa perlu komputer super mahal.

Kasus Ilustrasi

Kasus 1: Prediksi Nilai Ujian

Bayangin kamu mau tahu nilai ujian akhir dari data nilai tugas dan ulangan harian. Ini cocok pakai **Regresi Linier**, karena hasil akhirnya berupa **angka**.

Ilustrasi:

- Input: Nilai tugas = 80, Nilai ulangan = 75
- Output: Prediksi nilai ujian akhir ≈ 78

Pseudocode:

```
data = [ (nilai_tugas, nilai_ulangan, nilai_ujian) ]
model = LinearRegression()
model.train(data)
prediksi = model.predict( (80, 75) )
```

Kasus 2: Prediksi Pelanggan Setia

Kamu punya data belanja pelanggan di kantin sekolah.
Mau tahu apakah mereka bakal balik belanja lagi atau tidak.
Ini cocok pakai **Decision Tree**, karena hasilnya berupa **kategori (Ya/Tidak)**.

Ilustrasi:

- Input: Jumlah belanja = Rp50.000, frekuensi = 3 kali seminggu
- Output: “Pelanggan ini Setia”

Pseudocode:

```
data = [ (jumlah_belanja, frekuensi, setia_atau_tidak) ]  
model = DecisionTreeClassifier()  
model.train(data)  
prediksi = model.predict( (50000, 3) )
```

Kasus 3: Prediksi Biaya Makanan

Kamu mau masak untuk kelas, tapi harus tahu berapa total biaya bahan.
Ini cocok pakai **Regresi Linier**, karena hasilnya berupa **angka (biaya)**.

Ilustrasi:

- Input: Jumlah bahan (5 item), harga rata-rata per item = Rp10.000
- Output: Prediksi total biaya ≈ Rp50.000

Pseudocode:

```
data = [ (jumlah_bahan, harga_rata, total_biaya) ]  
model = LinearRegression()  
model.train(data)  
prediksi = model.predict( (5, 10000) )
```

Kasus 4: Rekomendasi Game

Kamu ingin tahu game apa yang cocok buat temanmu berdasarkan kesukaan.
Ini cocok pakai **Decision Tree**, karena hasilnya berupa **kategori (genre game)**.

Ilustrasi:

- Input: Suka tantangan = Ya, Suka cerita panjang = Ya
- Output: “Rekomendasi: Game RPG”

Pseudocode:

```
data = [ (suka_tantangan, suka_cerita, genre_game) ]  
model = DecisionTreeClassifier()  
model.train(data)  
prediksi = model.predict( (1, 1) )
```

Kasus 5: Prediksi Waktu Perjalanan

Kamu mau tahu berapa lama waktu tempuh ke sekolah dengan sepeda.
Ini cocok pakai **Regresi Linier**, karena hasilnya berupa **angka (menit)**.

Ilustrasi:

- Input: Jarak = 5 km, Kecepatan rata-rata = 15 km/jam
- Output: Waktu perjalanan $\approx$ 20 menit

Pseudocode:

```
data = [ (jarak, kecepatan, waktu_tempuh) ]  
model = LinearRegression()  
model.train(data)  
prediksi = model.predict( (5, 15) )
```

Kesimpulan

- Kalau yang diprediksi **angka** → **Regresi Linier**.
- Kalau yang diprediksi **kategori** → **Decision Tree**.
- Semua bisa dicoba dengan *Python*, *Pandas*, dan *scikit-learn*.
- Gratis, open source, dan seru buat eksperimen di sekolah!

Soal Latihan & Pembahasan

Soal 1

Kamu ingin memprediksi **nilai Ujian Nasional** berdasarkan **nilai ulangan harian**.
Algoritma apa yang paling cocok?

Jawaban: Regresi Linier

Pembahasan:

Karena yang diprediksi berupa **angka** (nilai ujian), maka lebih cocok menggunakan *Linear Regression*.

Pseudocode:

```
data = [ (ulangan, ujian) ]  
model = LinearRegression()  
model.train(data)  
prediksi = model.predict(ulangan=80)
```

Soal 2

Seorang guru ingin tahu apakah siswanya **lulus atau tidak lulus** berdasarkan data kehadiran dan nilai tugas. Algoritma apa yang paling tepat?

Jawaban: Decision Tree

Pembahasan:

Karena hasilnya berupa **kategori** (Lulus/Tidak), maka *Decision Tree* lebih sesuai.

Pseudocode:

```
data = [ (kehadiran, nilai_tugas, status_lulus) ]
model = DecisionTreeClassifier()
model.train(data)
prediksi = model.predict( (90, 70) )
```

Soal 3

Apa tujuan utama dari algoritma **Regresi Linier**?

Jawaban: Untuk memprediksi nilai **numerik**.

Pembahasan:

Linear Regression selalu mencari **garis lurus terbaik** yang bisa mewakili data. Misalnya memprediksi harga, berat badan, atau suhu.

Pseudocode:

```
model = LinearRegression()
model.predict( data_baru )
```

Soal 4

Apa yang terjadi jika **Decision Tree** terlalu banyak cabangnya (terlalu kompleks)?

Jawaban: Terjadi *overfitting*.

Pembahasan:

Overfitting = model terlalu hafal data lama, tapi gagal menebak data baru.

Bayangin kamu hanya hafal soal latihan, tapi nggak bisa jawab soal ujian yang beda sedikit.

Pseudocode:

```
model = DecisionTreeClassifier(max_depth=100)
# terlalu dalam → overfitting
```

Soal 5

Apa fungsi utama dari library **scikit-learn**?

Jawaban: Menyediakan algoritma *Machine Learning* siap pakai.

Pembahasan:

Dengan scikit-learn, kita tidak perlu bikin model dari nol. Tinggal import, latih, lalu prediksi.

Pseudocode:

```
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeClassifier
```

Kesimpulan

- Kalau prediksi **angka** → pakai *Regresi Linier*.
- Kalau prediksi **kategori** → pakai *Decision Tree*.
- Semua bisa dilakukan dengan *Python*, *Pandas*, dan *scikit-learn* yang gratis & open source.
- Latihan dengan soal-soal ini bisa bikin kalian makin jago di dunia *Machine Learning*!

Fun Facts!

- **scikit-learn** itu dibangun di atas **Pandas** dan **Numpy**. Jadi, kalau kalian jago Pandas, kalian sudah punya modal buat belajar scikit-learn!
- **Ollama** dan **OpenWebUI** bisa bantu kalian bikin model machine learning. Kalian bisa tanya, "Gimana cara bikin model Decision Tree di Python buat dataset ini?" dan dia bakal kasih kodenya.
- Di dunia kerja, **Regresi Linier** dan **Decision Tree** adalah dua model yang paling sering dipakai karena simpel dan gampang dijelaskan.

Tips Belajar & Menghafal Cepat

- **Fokus pada Tipe Output:** Selalu tanya, "Outputnya mau angka atau kategori?" Itu kunci untuk memilih antara **Regresi** dan **Klasifikasi**.
- **Visualisasi!** Bikin sketsa pohon keputusan di buku catatan. Itu akan sangat membantu.
- **Praktik di Google Colab:** Nggak perlu instal Python di laptop, kalian bisa pakai Google Colab yang gratis.

Aktivitas Siswa

Aktivitas 1: Prediksi Ukuran Baju dengan Regresi Linier

Tujuan: Memahami bagaimana *Linear Regression* memprediksi angka.

Langkah-langkah:

1. Buat data fiktif: tinggi badan dan ukuran baju.
2. Latih model dengan *Linear Regression*.
3. Prediksi ukuran baju baru berdasarkan tinggi badan.

Pseudocode:

```
data = (tinggi_badan, ukuran_baju)
model = LinearRegression()
model.train(data)
hasil = model.predict(tinggi_baru)
```

Contoh Python:

```
from sklearn.linear_model import LinearRegression
import pandas as pd

# Data fiktif
df = pd.DataFrame({
    "tinggi_badan": [150, 160, 170, 180],
    "ukuran_baju": [38, 40, 42, 44]
})

X = df[["tinggi_badan"]]
y = df["ukuran_baju"]

model = LinearRegression()
model.fit(X, y)

print(model.predict([[175]])) # Prediksi ukuran baju utk tinggi
175cm
```

Output:

```
[43.]
```

Aktivitas 2: Apakah Suka Makanan Pedas Manis?

Tujuan: Memahami *Decision Tree* untuk klasifikasi (Ya/Tidak).

Langkah-langkah:

1. Buat data fiktif: suka pedas, suka manis → suka pedas manis (0/1).
2. Latih model *DecisionTreeClassifier*.

3. Prediksi untuk data baru.

Pseudocode:

```
data = (suka_pedas, suka_manis, suka_pedas_manis)
model = DecisionTreeClassifier()
model.train(data)
hasil = model.predict((1, 0))
```

Contoh Python:

```
from sklearn.tree import DecisionTreeClassifier
import pandas as pd

# Data fiktif
df = pd.DataFrame({
    "suka_pedas": [1, 0, 1, 0],
    "suka_manis": [1, 1, 0, 0],
    "suka_pedas_manis": [1, 1, 0, 0]
})

X = df[["suka_pedas", "suka_manis"]]
y = df["suka_pedas_manis"]

model = DecisionTreeClassifier()
model.fit(X, y)

print(model.predict([[1, 0]])) # Prediksi utk suka pedas tapi
                                tidak suka manis
```

Output:

```
[0]
```

Aktivitas 3: Visualisasi Pohon Keputusan

Tujuan: Melihat cara *Decision Tree* mengambil keputusan.

Langkah-langkah:

1. Gunakan data dari Aktivitas 2.
2. Latih model *Decision Tree*.
3. Visualisasikan dengan *graphviz*.

Pseudocode:

```
buat_model_decision_tree()
tampilkan_grafik_pohon()
```

Contoh Python:

```
# 1. Install library yang dibutuhkan
```

```

!pip install graphviz scikit-learn

# 2. Import semua library
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_graphviz
import graphviz

# 3. Buat data contoh (misal: memprediksi suka jengkol)
# Fitur: suka_pedas (1=Ya, 0=Tidak), suka_manis (1=Ya, 0=Tidak)
# Target: suka_jengkol (1=Ya, 0=Tidak)
data = {
    'suka_pedas': [1, 1, 0, 0, 1, 0],
    'suka_manis': [0, 1, 1, 0, 0, 1],
    'suka_jengkol': [1, 0, 0, 0, 1, 0] # Target
}
df = pd.DataFrame(data)

X = df[['suka_pedas', 'suka_manis']]
y = df['suka_jengkol']

# 4. Buat dan latih model Decision Tree
model = DecisionTreeClassifier(max_depth=2) # Dibatasi
kedalamannya agar mudah dibaca
model.fit(X, y)

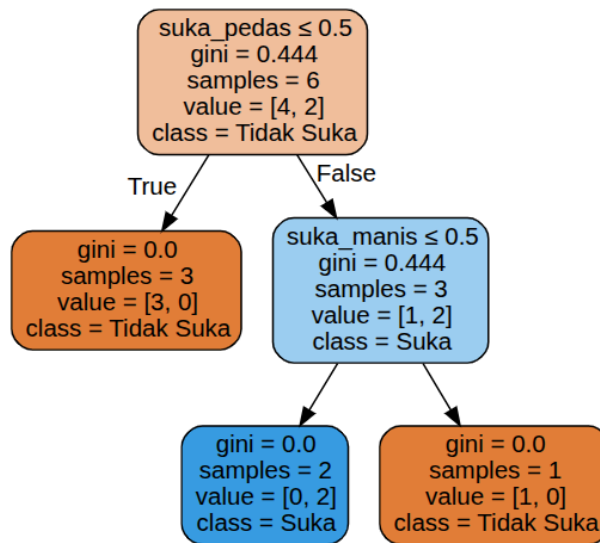
# 5. Buat visualisasi graph
dot_data = export_graphviz(
    model,
    out_file=None,
    feature_names=["suka_pedas", "suka_manis"],
    class_names=["Tidak Suka", "Suka"], # Label untuk kelas 0
    dan 1
    filled=True,
    rounded=True,
    special_characters=True
)

graph = graphviz.Source(dot_data)

# 6. Tampilkan gambar di output Colab
graph

```

Output:



Aktivitas 4: Prediksi Harga HP Bekas

Tujuan: Menggunakan *Linear Regression* untuk kasus nyata.

Langkah-langkah:

1. Buat data fiktif: usia HP, kondisi baterai, harga.
2. Latih model *Linear Regression*.
3. Prediksi harga HP dengan data baru.

Pseudocode:

```
data = (usia_hp, baterai, harga)
model = LinearRegression()
model.train(data)
prediksi = model.predict((2, 80))
```

Contoh Python:

```
df = pd.DataFrame({
    "usia_hp": [1, 2, 3, 4],
    "baterai": [90, 80, 70, 60],
    "harga": [4000000, 3000000, 2000000, 1000000]
})

X = df[["usia_hp", "baterai"]]
y = df["harga"]

model = LinearRegression()
model.fit(X, y)
```

```
print(model.predict([[2, 75]])) # Prediksi harga utk HP 2
tahun, baterai 75%
```

Output:

```
[2504950.4950495]
```

Aktivitas 5: Challenge – Tanya AI Lokal

Tujuan: Latihan mandiri dengan AI open source.

Langkah-langkah:

1. Instal Ollama atau OpenWebUI (AI lokal).
2. Ajukan pertanyaan: *“Berikan kode Python untuk Decision Tree prediksi genre game.”*
3. Jalankan kode yang diberikan di Google Colab.

Pseudocode (tanya AI):

```
ask_AI("contoh kode Decision Tree untuk prediksi genre game")
```

Contoh Python (hasil latihan):

```
df = pd.DataFrame({
    "waktu_main": [2, 5, 10, 20],
    "level_sulit": [1, 2, 3, 4],
    "genre": ["Casual", "Casual", "RPG", "RPG"]
})

X = df[["waktu_main", "level_sulit"]]
y = df["genre"]

model = DecisionTreeClassifier()
model.fit(X, y)

print(model.predict([[8, 3]])) # Prediksi genre utk data baru
```

Rangkuman Aktivitas

- **Aktivitas 1:** Prediksi ukuran baju → Linear Regression.
- **Aktivitas 2:** Apakah suka pedas manis → Decision Tree.
- **Aktivitas 3:** Visualisasi pohon keputusan.
- **Aktivitas 4:** Prediksi harga HP bekas.
- **Aktivitas 5:** Challenge dengan AI lokal.

Belajar jadi lebih seru karena kalian langsung praktek pakai **Python**, **Pandas**, dan **scikit-learn**.

Rangkuman Singkat

Hari ini kita sudah belajar tentang **Supervised Learning**, yaitu cara komputer belajar dari data yang sudah ada jawabannya. Dua algoritma penting yang kita pakai adalah **Regresi Linier** dan **Decision Tree**. *Regresi Linier* dipakai untuk memprediksi **angka** seperti harga, nilai ujian, atau waktu perjalanan. Sedangkan *Decision Tree* dipakai untuk memprediksi **kategori**, misalnya “lulus/tidak lulus” atau “beli/tidak beli”.

Dengan bantuan *Python* dan *scikit-learn* yang bersifat **open source**, kita bisa membuat model pintar dengan langkah sederhana: kumpulkan data → latih model → prediksi data baru. **Regresi Linier** bekerja dengan mencari *garis lurus terbaik* untuk memprediksi angka, sementara **Decision Tree** bekerja dengan membuat *pertanyaan bercabang* untuk mengambil keputusan. Kedua model ini sering dipakai di dunia nyata karena **mudah, cepat, dan bisa dipahami oleh siapa saja**.

Jangan lupa, kunci memilih algoritma adalah: **jika output angka → Regresi Linier, jika output kategori → Decision Tree**. Semakin banyak kalian latihan, semakin jago kalian membuat prediksi. Dunia kerja sekarang butuh orang yang paham *Machine Learning* dengan dasar kuat. Jadi, **latihan, eksperimen, dan praktek langsung** adalah cara terbaik supaya kalian siap bersaing di era digital ini.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. *Supervised Learning* berarti komputer belajar dari data yang sudah ada jawabannya.
Jawaban: Benar
2. *Regresi Linier* digunakan untuk memprediksi kategori, misalnya “Lulus/Tidak Lulus”.
Jawaban: Salah
3. *Decision Tree* bekerja dengan membuat pertanyaan bercabang (ya/tidak).
Jawaban: Benar
4. Library **scikit-learn** bersifat open source dan bisa digunakan gratis.
Jawaban: Benar
5. Jika model terlalu kompleks, bisa terjadi *overfitting*.
Jawaban: Benar

B. Pilihan Ganda (10 Soal)

6. Algoritma yang cocok untuk memprediksi **angka** adalah ...
 - a) Decision Tree
 - b) K-Means
 - c) Regresi Linier

d) Clustering

Jawaban: c

7. Jika outputnya kategori, maka algoritma yang dipilih adalah ...

- a) Regresi Linier
- b) Decision Tree
- c) Linear Algebra
- d) Pandas

Jawaban: b

8. Apa tujuan utama *Decision Tree*?

- a) Membuat grafik garis
- b) Membuat prediksi kategori
- c) Membersihkan data
- d) Menyimpan dataset

Jawaban: b

9. Library **scikit-learn** dibangun di atas ...

- a) SQL dan MongoDB
- b) TensorFlow dan Keras
- c) Pandas dan NumPy
- d) C++ dan Java

Jawaban: c

10. Jika data latih terlalu sedikit, model bisa jadi ...

- a) Sangat akurat
- b) Overfitting
- c) Underfitting
- d) Supervised

Jawaban: c

11. Prediksi “harga rumah” cocok menggunakan ...

- a) Decision Tree
- b) Regresi Linier
- c) CNN
- d) NLP

Jawaban: b

12. Dalam *Python*, fungsi untuk melatih model *Linear Regression* adalah ...

- a) `model.train()`
- b) `model.fit()`
- c) `model.run()`
- d) `model.learn()`

Jawaban: b

13. *Overfitting* berarti ...

- a) Model terlalu hafal data lama tapi jelek pada data baru

- b) Model tidak bisa belajar sama sekali
- c) Model selalu benar
- d) Model sangat sederhana

Jawaban: a

14. Contoh output *Decision Tree* adalah ...

- a) 170 cm
- b) Rp 2.000.000
- c) "Ya/Tidak"
- d) 3.14

Jawaban: c

15. Library *graphviz* dipakai untuk ...

- a) Membuat grafik data
- b) Visualisasi pohon keputusan
- c) Menghitung regresi
- d) Menyimpan file Python

Jawaban: b

C. Isian Singkat (5 Soal)

16. Lengkapi: *Regresi Linier* mencari _____ terbaik untuk data.

Jawaban: garis lurus

17. Jika ingin memprediksi jumlah pengunjung website, algoritma yang dipakai adalah _____.

Jawaban: Regresi Linier

18. Jika ingin memprediksi siswa ikut lomba futsal atau tidak, algoritma yang dipakai adalah _____.

Jawaban: Decision Tree

19. Apa nama fungsi *Python* untuk melakukan prediksi pada model machine learning?

Jawaban: `predict()`

20. Apa istilah untuk kondisi ketika model terlalu kompleks dan terlalu hafal data?

Jawaban: Overfitting

D. Soal Eksplorasi (3 Soal)

Soal 1:

Bayangkan kamu menjadi data scientist di kantin sekolah. Kamu ingin memprediksi **penjualan minuman es teh** berdasarkan jumlah promosi dan cuaca (cerah/hujan).

- Buatlah rencana data sederhana (kolom apa saja yang dibutuhkan).
- Pilih algoritma yang tepat (*Regresi Linier* atau *Decision Tree*).
- Tulis *pseudocode* singkat.

Contoh Jawaban:

- Data: `jumlah_promosi`, `cuaca`, `penjualan`.
- Algoritma: Decision Tree (karena ada kategori cuaca).
- Pseudocode:

```
data = (promosi, cuaca, penjualan)
model = DecisionTreeClassifier()
model.train(data)
prediksi = model.predict((5, "Cerah"))
```

Soal 2:

Seorang guru ingin tahu berapa **nilai ujian akhir** siswanya berdasarkan nilai tugas harian. Kamu diminta membuat model prediksi.

- Sebutkan algoritma yang dipakai.
- Tulis kode *Python* singkat untuk melatih model.

Contoh Jawaban:

- Algoritma: Regresi Linier
- Python:

```
from sklearn.linear_model import LinearRegression
X = [[60], [70], [80]]
y = [65, 75, 85]
model = LinearRegression()
model.fit(X, y)
print(model.predict([[75]]))
```

Soal 3:

Kamu sedang mengembangkan aplikasi rekomendasi game untuk teman sekelas. Data yang dipakai: `jam_main`, `level_sulit`, dan `genre_game` (`RPG/Strategi`).

- Sebutkan algoritma yang tepat.
- Buat *pseudocode* untuk melatih model.
- Apa yang akan kamu lakukan jika hasil prediksi kurang akurat?

Contoh Jawaban:

- Algoritma: Decision Tree
- Pseudocode:

```
data = (jam_main, level_sulit, genre_game)
model = DecisionTreeClassifier()
model.train(data)
prediksi = model.predict((5, 2))
```

- Jika akurasi rendah → tambah data, kurangi overfitting, atau ubah parameter model.

Tugas Proyek

Proyek 1: Prediksi Nilai Rata-rata Kelas

Soal: Buat model *Regresi Linier* untuk memprediksi **nilai rata-rata kelas** berdasarkan jumlah jam belajar per minggu.

Langkah:

1. Buat data fiktif: **jam_belajar** dan **nilai_rata_rata**.
2. Latih model dengan *LinearRegression*.
3. Prediksi nilai rata-rata jika jam belajar = 8.

Pseudocode:

```
data = (jam_belajar, nilai_rata_rata)
model = LinearRegression()
model.train(data)
prediksi = model.predict(jam=8)
```

Python:

```
from sklearn.linear_model import LinearRegression
import pandas as pd

df = pd.DataFrame({
    "jam_belajar": [2, 4, 6, 10],
    "nilai_rata_rata": [60, 70, 75, 90]
})

X = df[["jam_belajar"]]
y = df["nilai_rata_rata"]

model = LinearRegression()
model.fit(X, y)
```

```
print(model.predict([[8]])) # Prediksi nilai utk 8 jam belajar
```

Proyek 2: Apakah Siswa Aktif di Ekstrakurikuler?

Soal: Buat model *Decision Tree* untuk memprediksi apakah siswa **aktif di ekstrakurikuler** berdasarkan jumlah kegiatan dan rata-rata nilai.

Langkah:

1. Buat data fiktif: **jumlah_kegiatan, nilai_rata2, aktif_ekskul (ya/tidak)**.
2. Latih model dengan *DecisionTreeClassifier*.
3. Prediksi apakah siswa dengan 2 kegiatan & nilai 80 akan aktif.

Pseudocode:

```
data = (kegiatan, nilai, aktif/tidak)
model = DecisionTreeClassifier()
model.train(data)
prediksi = model.predict((2, 80))
```

Python:

```
from sklearn.tree import DecisionTreeClassifier
import pandas as pd

df = pd.DataFrame({
    "kegiatan": [0, 1, 2, 3],
    "nilai": [60, 70, 80, 85],
    "aktif": ["Tidak", "Ya", "Ya", "Ya"]
})

X = df[["kegiatan", "nilai"]]
y = df["aktif"]

model = DecisionTreeClassifier()
model.fit(X, y)

print(model.predict([[2, 80]])) # Prediksi siswa dgn 2 kegiatan
& nilai 80
```

Proyek 3: Prediksi Harga Jajanan Kantin

Soal: Gunakan *Regresi Linier* untuk memprediksi harga jajanan berdasarkan jumlah bahan dan kualitas bahan.

Langkah:

1. Buat data fiktif: **jumlah_bahan, kualitas, harga**.
2. Latih model.
3. Prediksi harga untuk 5 bahan dengan kualitas 80.

Pseudocode:

```
data = (jumlah_bahan, kualitas, harga)
model = LinearRegression()
model.train(data)
prediksi = model.predict((5, 80))
```

Proyek 4: Prediksi Genre Film Favorit

Soal: Buat *Decision Tree* untuk menebak genre film favorit siswa berdasarkan durasi film dan tingkat aksi.

Langkah:

1. Buat data fiktif: **durasi, aksi_level, genre (Action/Romance)**.
2. Latih model.
3. Prediksi genre untuk film dengan durasi 120 menit & aksi level 3.

Pseudocode:

```
data = (durasi, aksi_level, genre)
model = DecisionTreeClassifier()
model.train(data)
prediksi = model.predict((120, 3))
```

Proyek 5: Prediksi Jumlah Penonton Event Sekolah

Soal: Gunakan *Regresi Linier* untuk memprediksi jumlah penonton acara sekolah berdasarkan jumlah promosi dan jumlah hari persiapan.

Langkah:

1. Buat data fiktif: **promosi, hari_persiapan, penonton**.
2. Latih model.
3. Prediksi jumlah penonton untuk 5 promosi dan 7 hari persiapan.

Pseudocode:

```
data = (promosi, hari_persiapan, penonton)
model = LinearRegression()
model.train(data)
prediksi = model.predict((5, 7))
```

Kesimpulan

- **Proyek 1 & 3 & 5** → Regresi Linier (prediksi angka).

- **Proyek 2 & 4** → Decision Tree (prediksi kategori).
- Semua proyek bisa kalian coba di **Google Colab** atau laptop dengan **Python + scikit-learn**.
- Semakin sering latihan, semakin jago kalian menguasai *Machine Learning*!

BAB 5: Evaluasi Model (Akurasi, Presisi, Recall)

Setelah kita seru-seruan bikin model prediksi, sekarang saatnya kita jadi **juri**! Kita akan belajar cara menilai seberapa jago model yang sudah kita buat. Di materi ini, kita akan kenalan sama tiga juri andalan: **Akurasi, Presisi, dan Recall**.

Tujuan Pembelajaran

Setelah sesi ini, kalian bakal bisa:

- Memahami apa itu **evaluasi model**.
- Membedakan antara **Akurasi, Presisi, dan Recall**.
- Menilai kinerja model dengan benar.
- Menggunakan matrik evaluasi untuk memilih model yang terbaik.

Peta Konsep Sederhana

Bayangin kita mau ngetes seberapa jago bot di game tembak-menembak.

- **Akurasi**: Bot-nya bilang, "Musuh terdeteksi!" dan ternyata benar ada musuh. Atau bot-nya bilang, "Tidak ada musuh," dan ternyata benar tidak ada musuh. **Akurasi itu perbandingan total tebakan yang benar.**
- **Presisi**: Bot-nya bilang, "Musuh terdeteksi!" Seberapa sering tebakan itu **benar-benar** musuh (bukan teman atau NPC)?
- **Recall**: Ada 10 musuh di lapangan. Bot-nya cuma bilang, "Musuh terdeteksi!" 7 kali. **Recall itu perbandingan total musuh yang terdeteksi.**

Apersepsi: Kamu Punya Bot Detektor Kucing!

Kamu bikin bot yang bisa mendeteksi kucing di foto. Lalu, kamu tes dengan 100 foto:

- **Ada 50 foto kucing**
- **Ada 50 foto anjing**

Bot-mu tebak, "Ada kucing!" 40 kali. Ternyata, 35 tebakan itu benar kucing, tapi 5 lainnya malah anjing. Sebaliknya, bot-mu juga bilang, "Tidak ada kucing!" 60 kali, dan 45 di antaranya memang benar anjing.

Nah, dari sini kita bisa hitung **Akurasi, Presisi, dan Recall**-nya!

- **Akurasi**: $(35 + 45) / 100 = 80\%$. Cukup bagus!
- **Presisi**: $35 / 40 = 87,5\%$. Kalau bot bilang kucing, kemungkinan besar itu benar-benar kucing.
- **Recall**: $35 / 50 = 70\%$. Dari 50 kucing asli, bot-mu cuma bisa menemukan 35.

Penjelasan Teori

Setelah kita bikin model cerdas dengan *machine learning*, sekarang waktunya kita jadi juri. Bayangkan kita punya robot pintar yang suka menebak: “Ini kucing!” atau “Ini bukan kucing!” Nah, kita perlu tahu seberapa pintar robot itu. Di sinilah kita belajar **Evaluasi Model**. Ada tiga juri utama yang akan menilai:

- **Akurasi**
- **Presisi**
- **Recall**

Akurasi (Accuracy)

- Definisi: Seberapa banyak tebakan robot yang benar dibandingkan semua tebakan.
- Rumus:

$$\text{Akurasi} = \frac{\text{Tebakan Benar}}{\text{Total Tebakan}}$$

- Contoh: Dari 100 tebakan, kalau 80 benar → Akurasi = 80%.

Presisi (Precision)

- Definisi: Saat robot bilang “Ini kucing!”, seberapa sering tebakan itu benar-benar kucing?
- Rumus:

$$\text{Presisi} = \frac{\text{Positif Benar}}{\text{Semua Tebakan Positif}}$$

- Contoh: Dari 40 kali robot bilang “kucing”, ternyata 35 benar → Presisi = 87,5%.

Recall

- Definisi: Dari semua kucing yang ada, berapa banyak yang berhasil ditemukan robot?
- Rumus:

$$\text{Recall} = \frac{\text{Positif Benar}}{\text{Semua Data Positif}}$$

- Contoh: Ada 50 kucing asli, robot cuma nemu 35 → Recall = 70%.

Analogi Seru

- **Akurasi:** Nilai ujian matematika kamu. Dari 10 soal, kalau kamu benar 8, akurasi kamu 80%.
- **Presisi:** Bayangkan kamu detektif. Kalau kamu teriak “Ini pencuri!” → presisi mengukur seberapa sering kamu benar-benar menunjuk pencuri, bukan orang biasa.

- **Recall:** Dari 10 pencuri yang berkeliaran, berapa banyak yang berhasil kamu tangkap?

Rangkuman

- **Akurasi** = Seberapa banyak tebakan yang benar.
- **Presisi** = Seberapa sering tebakan “positif” benar-benar positif.
- **Recall** = Seberapa banyak data positif yang berhasil ditemukan.
- Akurasi cocok untuk kasus umum, Presisi penting untuk menghindari *false positive*, Recall penting untuk menghindari *false negative*.

Kasus Ilustrasi

Kasus 1: Deteksi Virus di Komputer

- Tujuan: Jangan sampai ada virus yang lolos!
- Fokus: **Recall** lebih penting.

Pseudocode:

```
data = [virus, virus, normal, virus, normal]
prediksi = [virus, normal, normal, virus, normal]

hitung_recall(data, prediksi)
```

Artinya: model harus bisa menemukan semua virus, walaupun ada beberapa alarm palsu.

Kasus 2: Filter Email Spam

- Tujuan: Email penting jangan nyangkut di folder spam.
- Fokus: **Presisi** lebih penting.

Pseudocode:

```
email_data = [spam, normal, spam, normal, spam]
prediksi = [spam, spam, spam, normal, normal]

hitung_presisi(email_data, prediksi)
```

Artinya: kalau model bilang “spam”, harus benar-benar spam. Jangan sampai salah tebak.

Kasus 3: Prediksi Cuaca Hujan

- Tujuan: Memberi info cuaca yang bisa dipercaya.
- Fokus: **Akurasi**, tapi Recall juga penting kalau soal hujan.

Pseudocode:

```
cuaca_asli = [hujan, hujan, cerah, hujan, cerah]
prediksi =  [hujan, cerah, cerah, hujan, cerah]

hitung_akurasi(cuaca_asli, prediksi)
```

Artinya: model harus benar dalam menebak cuaca, terutama hujan. Salah sedikit bisa bikin orang keuhujan.

Kasus 4: Deteksi Penyakit Langka

- Tujuan: Semua pasien sakit harus terdeteksi.
- Fokus: **Recall** lebih penting.

Pseudocode:

```
pasien = [sakit, sehat, sakit, sehat, sakit]
prediksi = [sakit, sehat, sehat, sehat, sakit]

hitung_recall(pasien, prediksi)
```

Artinya: lebih baik salah alarm daripada ada pasien sakit yang tidak terdeteksi.

Kasus 5: Rekomendasi Konten YouTube

- Tujuan: Hanya menampilkan video yang benar-benar disukai.
- Fokus: **Presisi** lebih penting.

Pseudocode:

```
video_suka = [suka, tidak, suka, tidak, suka]
prediksi =  [suka, suka, suka, tidak, tidak]

hitung_presisi(video_suka, prediksi)
```

Artinya: kalau sistem merekomendasikan video, harus sesuai dengan selera pengguna.

Rangkuman

- **Akurasi:** cocok untuk kasus umum (misalnya prediksi cuaca).
- **Presisi:** penting kalau salah tebak itu bahaya (misalnya spam atau penipuan).
- **Recall:** penting kalau yang positif jangan sampai ada yang terlewat (misalnya virus atau penyakit).

Soal Latihan & Pembahasan

Soal 1: Hitung Akurasi

Kamu punya model yang menebak hasil ujian siswa.
Dari 100 data: 85 tebakan benar, 15 tebakan salah.
Hitung Akurasi!

Pseudocode:

```
benar = 85
total = 100
akurasi = benar / total
```

Jawaban:

Akurasi = $85 / 100 = 85\%$

Pembahasan: Akurasi menunjukkan seberapa banyak tebakan benar dari semua tebakan.

Soal 2: Hitung Presisi

Bot deteksi kucing diuji dengan 50 foto.

- Tebakan bot: "kucing" sebanyak 20 kali.
- Dari 20 tebakan itu, 15 benar-benar kucing, 5 salah (anjing).
Hitung Presisi!

Pseudocode:

```
positif_benar = 15
total_positif_prediksi = 20
presisi = positif_benar / total_positif_prediksi
```

Jawaban:

Presisi = $15 / 20 = 75\%$

Pembahasan: Presisi menunjukkan, kalau bot bilang "kucing", seberapa sering benar.

Soal 3 – Hitung Recall

Masih kasus bot deteksi kucing.

- Ada 30 kucing asli di data.
- Bot hanya berhasil mendeteksi 18 kucing.
Hitung Recall!

Pseudocode:

```
positif_benar = 18
total_positif_asli = 30
```

```
recall = positif_benar / total_positif_asli
```

Jawaban:

Recall = $18 / 30 = 60\%$

Pembahasan: Recall menunjukkan berapa banyak data positif (kucing asli) yang berhasil ditemukan.

Soal 4: Pilih Metrik yang Tepat

Kasus: deteksi penyakit berbahaya.

Menurutmu, metrik mana yang lebih penting: Akurasi, Presisi, atau Recall?

Jawaban: Recall.

Pembahasan: Lebih baik ada salah alarm (sehat dikira sakit) daripada pasien sakit tidak terdeteksi.

Soal 5: Kombinasi Akurasi, Presisi, dan Recall

Model prediksi email spam diuji:

- Ada 100 email.
- Bot menebak 60 email spam. Dari 60 itu, 50 benar-benar spam.
- Total spam asli: 70 email.

Hitung Akurasi, Presisi, dan Recall.

Pseudocode:

```
total_data = 100
tebakan_benar = 50 + (30 - (70-50))    # spam benar + normal benar
akurasi = tebak_benar / total_data

presisi = 50 / 60
recall = 50 / 70
```

Jawaban:

- Akurasi = $(50 + 20) / 100 = 70\%$
- Presisi = $50 / 60 = 83.3\%$
- Recall = $50 / 70 = 71.4\%$

Pembahasan:

- Akurasi: gambaran umum tebakan.
- Presisi: bot cukup bagus, kalau bilang spam, biasanya benar.
- Recall: bot masih melewatkan beberapa spam asli.

Ringkasan

- **Akurasi** → Seberapa banyak tebakan benar.
- **Presisi** → Fokus pada tebakan positif yang benar.
- **Recall** → Fokus pada menemukan semua data positif.
- Tidak ada model sempurna. Selalu ada trade-off antara tiga metrik ini.

Fun Facts!

- Di dunia nyata, tidak ada model yang punya **Akurasi, Presisi, dan Recall** 100% bersamaan. Pasti ada **trade-off**.
- **Ollama** dan **OpenWebUI** bisa bantu kalian bikin laporan evaluasi model dan menjelaskan artinya.
- **scikit-learn** punya banyak matrik evaluasi yang bisa dipakai selain Akurasi, Presisi, dan Recall.

Tips Belajar & Menghafal Cepat

- **Ingat Rumusnya!** Buat kartu kecil untuk rumus-rumus ini.
- **Buat Analogi Sendiri!** Pikirkan contoh di kehidupan sehari-hari (misal: menebak pemenang lomba lari) dan hitung matriksnya.
- **Diskusi dengan Teman:** Berdebat dengan teman tentang metrik mana yang paling penting untuk suatu kasus akan sangat membantu.

Aktivitas Siswa

Aktivitas 1: Hitung Akurasi Secara Manual

Kita mulai dari yang gampang. Bayangkan robotmu menebak hasil ujian siswa. Ada 10 data. Robot benar 7 kali, salah 3 kali.

Langkah:

1. Tulis jumlah tebakan benar dan total tebakan.
2. Gunakan rumus Akurasi = Benar / Total.
3. Hitung manual dulu, lalu bandingkan dengan Python.

Pseudocode:

```
benar = 7
total = 10
akurasi = benar / total
```

Python:

```
benar = 7
total = 10
akurasi = benar / total
```

```
print("Akurasi =", akurasi)
```

Aktivitas 2: Hitung Presisi dari Data Kucing

Robot mendeteksi kucing dari 8 foto.

- Prediksi robot: 6 bilang “kucing”, 2 bilang “bukan”.
- Dari 6 tebakan “kucing”, hanya 4 yang benar.

Langkah:

1. Hitung jumlah tebakan “kucing” yang benar.
2. Bagi dengan total tebakan “kucing”.

Pseudocode:

```
positif_benar = 4
total_prediksi_positif = 6
presisi = positif_benar / total_prediksi_positif
```

Python:

```
positif_benar = 4
total_prediksi_positif = 6
presisi = positif_benar / total_prediksi_positif
print("Presisi =", presisi)
```

Aktivitas 3: Hitung Recall dari Data Pasien

Ada 10 pasien.

- 5 pasien benar-benar sakit.
- Robot berhasil deteksi 3 dari 5 pasien sakit.

Langkah:

1. Catat jumlah pasien sakit yang benar terdeteksi.
2. Bagi dengan jumlah pasien sakit asli.

Pseudocode:

```
positif_benar = 3
total_positif_asli = 5
recall = positif_benar / total_positif_asli
```

Python:

```
positif_benar = 3
total_positif_asli = 5
recall = positif_benar / total_positif_asli
print("Recall =", recall)
```

Aktivitas 4: Gunakan scikit-learn untuk Evaluasi Otomatis

Sekarang kita pakai Python dan *scikit-learn*. Kita buat prediksi sederhana lalu lihat laporan evaluasi.

Langkah:

1. Buat data label asli (*y_true*).
2. Buat data prediksi (*y_pred*).
3. Jalankan `classification_report`.

Pseudocode:

```
import sklearn
masukkan y_true, y_pred
panggil classification_report
```

Python:

```
from sklearn.metrics import classification_report

y_true = [1,0,1,1,0,1,0,0,1,0] # label asli
y_pred = [1,0,1,0,0,1,0,1,1,0] # hasil prediksi model

print(classification_report(y_true, y_pred))
```

Kasus 5: Eksperimen dengan Dataset Iris di scikit-learn

Kita coba dataset *open source* Iris yang sudah ada di scikit-learn.

Langkah:

1. Load dataset Iris dari scikit-learn.
2. Bagi data jadi train dan test.
3. Latih model sederhana (Decision Tree).
4. Evaluasi dengan `classification_report`.

Pseudocode:

```
load dataset iris
bagi train dan test
latih decision tree
prediksi test data
evaluasi dengan classification_report
```

Python:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report
```

```
# 1. Load data
iris = load_iris()
X, y = iris.data, iris.target

# 2. Bagi train dan test
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# 3. Latih model
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# 4. Prediksi
y_pred = model.predict(X_test)

# 5. Evaluasi
print(classification_report(y_test, y_pred))
```

Output:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

Rangkuman

Setelah belajar materi ini, kita tahu bahwa **evaluasi model** adalah langkah penting setelah membuat model *machine learning*. Model yang pintar bukan hanya bisa menebak, tapi juga harus bisa **dinilai kinerjanya**. Di sinilah tiga juri utama bekerja: **Akurasi**, **Presisi**, dan **Recall**. Ketiganya punya fungsi berbeda dan saling melengkapi.

Akurasi menunjukkan seberapa banyak tebakan model yang benar. **Presisi** mengukur seberapa sering tebakan positif itu benar-benar tepat. Sementara **Recall** memperlihatkan seberapa banyak data positif yang berhasil ditemukan oleh model. Jadi, jangan hanya terpaku pada satu angka saja. Dalam beberapa kasus, **Presisi lebih penting**, sementara di kasus lain, **Recall lebih penting**.

Tidak ada model yang bisa punya nilai **Akurasi, Presisi, dan Recall 100% sekaligus**. Selalu ada *trade-off* atau kompromi. Itulah mengapa seorang **data scientist** harus cerdas memilih metrik sesuai masalah. Dengan memahami tiga metrik ini, kita bisa membuat keputusan lebih bijak, membangun model yang lebih adil, dan siap menjadi **ahli AI dan machine learning yang keren** dengan Python, Pandas, Keras, dan *neural network*! 🚀

Latihan Soal

A. Benar atau Salah (5 soal)

1. Akurasi adalah jumlah tebakan benar dibagi total tebakan.
Jawaban: Benar
2. Presisi mengukur seberapa banyak data positif yang ditemukan model.
Jawaban: Salah
3. Recall penting untuk kasus deteksi penyakit langka.
Jawaban: Benar
4. Jika model menebak semua data sebagai "positif", akurasi pasti 100%.
Jawaban: Salah
5. Presisi lebih penting pada kasus filter email spam.
Jawaban: Benar

B. Pilihan Ganda (10 soal)

1. Rumus Akurasi adalah:
a) $(\text{Positif Benar}) / (\text{Total Positif})$
b) $(\text{Tebakan Benar}) / (\text{Total Tebakan})$
c) $(\text{Negatif Benar}) / (\text{Total Data})$
d) $(\text{Positif Benar}) / (\text{Total Data})$
Jawaban: b
2. Presisi digunakan untuk:
a) Menghitung semua tebakan benar
b) Mengukur seberapa sering tebakan positif benar
c) Mengukur seberapa banyak data positif terdeteksi
d) Membandingkan data positif dan negatif
Jawaban: b
3. Recall digunakan untuk:
a) Mengukur akurasi model
b) Mengukur berapa banyak data positif yang berhasil ditemukan
c) Menentukan jumlah data salah
d) Mengukur jumlah negatif benar
Jawaban: b
4. Kasus yang lebih fokus pada Recall adalah:
a) Filter email spam
b) Deteksi penyakit langka
c) Rekomendasi video YouTube
d) Prediksi nilai ujian

Jawaban: b

5. Kasus yang lebih fokus pada Presisi adalah:

- a) Deteksi penyakit
- b) Prediksi cuaca
- c) Filter email spam
- d) Deteksi virus komputer

Jawaban: c

6. Dari 100 data, model menebak benar 90. Akurasi adalah:

- a) 80%
- b) 85%
- c) 90%
- d) 95%

Jawaban: c

7. Dari 50 tebakan positif, 40 benar. Presisi adalah:

- a) 80%
- b) 85%
- c) 90%
- d) 70%

Jawaban: a

8. Dari 30 data positif, model menemukan 24. Recall adalah:

- a) 60%
- b) 70%
- c) 80%
- d) 90%

Jawaban: c

9. Alat Python yang bisa digunakan untuk evaluasi model:

- a) `numpy.mean`
- b) `sklearn.metrics.classification_report`
- c) `pandas.DataFrame`
- d) `keras.layers.Dense`

Jawaban: b

10. Dalam evaluasi model, trade-off berarti:

- a) Semua metrik bisa 100%
- b) Memilih antara Presisi tinggi atau Recall tinggi
- c) Model selalu salah
- d) Model tidak bisa digunakan

Jawaban: b

C. Isian Singkat (5 soal)

1. Akurasi adalah ...
Jawaban: jumlah tebakan benar dibagi total tebakan
2. Presisi adalah ...
Jawaban: Positif Benar / Total Prediksi Positif
3. Recall adalah ...
Jawaban: Positif Benar / Total Data Positif
4. Contoh kasus pentingnya Presisi adalah ...
Jawaban: filter email spam
5. Contoh kasus pentingnya Recall adalah ...
Jawaban: deteksi penyakit langka

D. Soal Explorasi (3 Soal)

Soal 1:

Buat dataset sederhana dengan 10 data tentang prediksi kelulusan siswa (fitur: nilai_tugas, nilai_ujian). Buat model Decision Tree, lalu tampilkan Akurasi, Presisi, dan Recall.

Contoh Jawaban Siswa (Python):

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report

X = [[80,90],[70,60],[50,40],[90,95],[65,70]]
y = ["lulus","tidak","tidak","lulus","lulus"]

model = DecisionTreeClassifier()
model.fit(X,y)
pred = model.predict(X)
print(classification_report(y,pred))
```

Output:

	precision	recall	f1-score	support	
lulus		1.00	1.00	1.00	3
tidak		1.00	1.00	1.00	2
accuracy				1.00	5
macro avg		1.00	1.00	1.00	5
weighted avg		1.00	1.00	1.00	5

Soal 2:

Simulasikan filter spam sederhana dengan data jumlah kata "promo". Jika ada >3 kata "promo" → spam, jika tidak → normal. Hitung Presisi dan Recall dari data buatanmu.

Contoh Jawaban Siswa (Pseudocode):

```
data = [spam, normal, spam, normal, spam]
prediksi = [spam, spam, normal, normal, spam]
hitung_presisi(data, prediksi)
hitung_recall(data, prediksi)
```

Soal 3:

Diskusikan: Jika kamu membuat sistem deteksi hujan untuk sekolahmu, metrik apa yang lebih penting, Akurasi, Presisi, atau Recall? Jelaskan dengan contoh.

Contoh Jawaban Siswa:

Recall lebih penting. Jika hujan tidak terdeteksi, siswa bisa kehujanan. Lebih baik ada salah alarm daripada hujan tidak terdeteksi.

Tugas Proyek

Proyek 1: Prediksi Kelulusan Sederhana

Buat data fiktif berisi nilai siswa (nilai_tugas, nilai_ujian) dan status lulus/tidak. Latih model sederhana, lalu hitung **Akurasi**.

Langkah:

1. Buat data minimal 10 siswa.
2. Tentukan label **lulus** atau **tidak**.
3. Latih Decision Tree.
4. Hitung akurasi model.

Pseudocode:

```
buat data nilai_tugas, nilai_ujian, lulus
bagi data train dan test
latih decision tree
prediksi hasil
hitung akurasi
```

Python:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X = [[80, 90], [60, 70], [40, 50], [90, 85], [55, 65]]
```

```

y = ["lulus","lulus","tidak","lulus","tidak"]

X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.4,random_state=1)

model = DecisionTreeClassifier()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)

print("Akurasi =", accuracy_score(y_test,y_pred))

```

Output:

```
Akurasi = 0.5
```

Proyek 2: Deteksi Spam Sederhana

Buat data sederhana jumlah kata “spam” dan “normal” dalam email, lalu evaluasi **Presisi**.

Langkah:

1. Buat data dengan fitur **jumlah_kata_spam**.
2. Labeli dengan **spam** atau **normal**.
3. Jalankan klasifikasi.
4. Hitung presisi.

Pseudocode:

```

buat dataset kata_spam, label
latih decision tree
prediksi hasil
hitung presisi

```

Python:

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import precision_score
import pandas as pd

# Membuat dataset sederhana
data = {
    'jumlah_kata_spam': [10, 2, 8, 1, 15, 3, 12, 0, 9, 4],
    'label': ['spam', 'normal', 'spam', 'normal', 'spam',
'normal', 'spam', 'normal', 'spam', 'normal']
}

# Mengubah dataset menjadi DataFrame
df = pd.DataFrame(data)

# Memisahkan fitur dan label
X = df[['jumlah_kata_spam']]

```

```

y = df['label']

# Membuat dan melatih model Decision Tree
model = DecisionTreeClassifier(random_state=42)
model.fit(X, y)

# Memprediksi hasil
y_pred = model.predict(X)

# Menghitung presisi
precision = precision_score(y, y_pred, pos_label='spam')

print(f"Presisi model: {precision:.2f}")

```

Output:

```
Presisi = 1.0
```

Proyek 3: Prediksi Pembelian Produk

Buat data usia dan hobi (misalnya **gaming**, **olahraga**, dll.), lalu prediksi apakah orang **membeli produk** atau tidak.

Langkah:

1. Buat 10 data fiktif.
2. Latih model klasifikasi.
3. Hitung Akurasi, Presisi, dan Recall.

Pseudocode:

```

buat dataset usia,hobi,label beli
ubah hobi jadi angka (encoding)
latih decision tree
prediksi test
evaluasi dengan classification_report

```

Python:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report

# Buat dataset fiktif
data = {
    'usia': [20, 25, 30, 22, 28, 35, 18, 27, 32, 24],

```

```

        'hobi': ['gaming', 'olahraga', 'membaca', 'gaming',
'musik', 'olahraga', 'gaming', 'membaca', 'musik',
'olahraga'],
        'beli': [1, 0, 1, 1, 0, 1, 1, 0, 0, 1] # 1 = beli, 0
= tidak beli
    }
    df = pd.DataFrame(data)

    # Encoding hobi menjadi angka
    le = LabelEncoder()
    df['hobi'] = le.fit_transform(df['hobi'])

    # Pisahkan fitur dan label
    X = df[['usia', 'hobi']]
    y = df['beli']

    # Bagi data menjadi train dan test
    X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

    # Latih model Decision Tree
    model = DecisionTreeClassifier(random_state=42)
    model.fit(X_train, y_train)

    # Prediksi pada data test
    y_pred = model.predict(X_test)

    # Evaluasi model
    print(classification_report(y_test, y_pred))

```

Output:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	2
1	0.33	1.00	0.50	1
accuracy			0.33	3
macro avg	0.17	0.50	0.25	3
weighted avg	0.11	0.33	0.17	3

Proyek 4: Deteksi Masalah Mesin

Buat data fiktif berisi **suhu_mesin** dan **kecepatan_putaran**, lalu prediksi apakah ada **masalah** atau tidak. Fokus pada **Recall**.

Langkah:

1. Buat dataset minimal 8 data.
2. Tandai label **masalah** atau **normal**.

3. Latih model.
4. Evaluasi recall.

Pseudocode:

```
buat data suhu, putaran, label masalah
latih model
prediksi
hitung recall
```

Python:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import recall_score, classification_report

# Membuat dataset fiktif
data = {
    'suhu_mesin': [70, 85, 90, 95, 60, 75, 100, 80],
    'kecepatan_putaran': [3000, 3500, 4000, 4200, 2800, 3200,
4500, 3100],
    'masalah': [0, 1, 1, 1, 0, 0, 1, 0] # 0: normal, 1: masalah
}
df = pd.DataFrame(data)

# Memisahkan fitur dan label
X = df[['suhu_mesin', 'kecepatan_putaran']]
y = df['masalah']

# Membagi data menjadi training dan testing
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.25, random_state=42)

# Melatih model Logistic Regression
model = LogisticRegression()
model.fit(X_train, y_train)

# Memprediksi data testing
y_pred = model.predict(X_test)

# Menghitung recall
recall = recall_score(y_test, y_pred)
print("Recall Score:", recall)

# Menampilkan laporan klasifikasi lengkap
print("\nClassification Report:")
print(classification_report(y_test, y_pred,
target_names=['Normal', 'Masalah']))
```

Output:

Recall Score: 0.0

Classification Report:

	precision	recall	f1-score	support
Normal	0.50	1.00	0.67	1
Masalah	0.00	0.00	0.00	1
accuracy			0.50	2
macro avg	0.25	0.50	0.33	2
weighted avg	0.25	0.50	0.33	2

Proyek 5: Klasifikasi Jenis Bunga

Gunakan dataset *iris* dari scikit-learn untuk mengklasifikasi bunga. Evaluasi semua metrik.

Langkah:

1. Load dataset iris.
2. Bagi train-test.
3. Latih Decision Tree.
4. Lihat Akurasi, Presisi, dan Recall.

Pseudocode:

```
load dataset iris
bagi train-test
latih decision tree
prediksi hasil
evaluasi classification_report
```

Python:

```
from sklearn.datasets import load_iris

iris = load_iris()
X,y = iris.data, iris.target

X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.3,random_state=42)

model = DecisionTreeClassifier()
model.fit(X_train,y_train)
y_pred = model.predict(X_test)

print(classification_report(y_test,y_pred))
```

Output:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

BAB 6: Visualisasi Hasil Machine Learning (ML)

Di bab ini, kita akan belajar cara **memvisualisasikan** (alias, menyajikan dalam bentuk gambar) hasil dari model **Machine Learning** kita. Ini penting banget, karena orang lebih gampang paham kalau lihat grafik daripada tumpukan angka.



Dilisensikan oleh Google

Tujuan Pembelajaran

- Memahami mengapa visualisasi adalah bagian penting dari analisis data.
- Mampu menggunakan Orange Data Mining untuk membuat visualisasi yang keren.
- Mengenal berbagai jenis grafik yang cocok untuk mengevaluasi model ML.
- Bisa membaca dan menginterpretasikan grafik-grafik tersebut.

Peta Konsep Sederhana

Bayangkan kita mau menyajikan makanan.

- **Bahan Mentah:** Data mentah.
- **Proses Memasak:** Membuat model ML (misal: dengan scikit-learn).
- **Makanan Jadi:** Hasil prediksi model.
- **Menyajikan di Piring Cantik:** **Memvisualisasikan hasil ML** dengan berbagai grafik (misal: Scatter Plot, Bar Chart, Confusion Matrix).

Hasilnya? Orang yang melihat akan langsung tahu seberapa enak (akurat) masakan kita!

Apersepsi: Kamu Mau Pamerin Game ke Teman!

Bayangin kamu bikin game dan mau pamer ke temanmu.

- **Cara Kuno:** Kamu tunjukkan kode-kode panjang dan bilang, "Game-ku bagus, lho. Lihat, akurasi 95%!" Pasti temanmu cuma ngangguk-ngangguk bingung.
- **Cara Keren:** Kamu tunjukkan sebuah grafik batang yang membandingkan akurasi game-mu dengan game lain. Atau, kamu tunjukkan grafik yang berisi lingkaran-lingkaran berwarna yang menunjukkan pemain di level yang berbeda. Pasti temanmu langsung bilang, "Gila, keren banget!"

Nah, itu dia gunanya visualisasi! Mengubah angka yang membingungkan jadi cerita yang menarik.

Penjelasan Teori

Bayangkan kamu habis bikin model *machine learning* yang bisa memprediksi sesuatu. Misalnya, memprediksi apakah siswa akan **lulus** atau **tidak lulus**. Model ini menghasilkan angka-angka, tabel, dan statistik. Tapi... kalau kamu tunjukkan angka itu langsung ke temanmu, mereka bisa bingung.

Nah, disinilah **visualisasi** berperan. Visualisasi itu adalah cara mengubah angka-angka kaku menjadi **gambar, grafik, atau diagram** yang gampang dipahami. Otak manusia lebih cepat mengerti **gambar** dibanding deretan angka. Jadi, dengan visualisasi, kita bisa *menceritakan kisah* dari data.

Kenapa Visualisasi Itu Penting?

- **Membuat Data Lebih Jelas:** Angka yang ribuan bisa jadi grafik sederhana.
- **Komunikasi Lebih Mudah:** Temanmu, guru, atau orang awam bisa langsung paham.
- **Menemukan Pola:** Grafik bisa menunjukkan pola yang tidak kelihatan di tabel.
- **Membandingkan Model:** Dengan grafik batang (*bar chart*), kita bisa tahu model mana yang lebih akurat.

Jenis-Jenis Visualisasi Populer

1. Confusion Matrix

- Bentuknya tabel kotak.
- Menunjukkan tebakan mana yang benar dan mana yang salah.
- Contoh: memprediksi "lulus" atau "tidak lulus".

2. Scatter Plot

- Grafik titik-titik.
- Bagus untuk regresi (prediksi nilai).
- Kalau titiknya mendekati garis lurus diagonal, modelmu bagus.

3. Bar Chart (Grafik Batang)

- Mirip pelajaran matematika.
- Cocok untuk membandingkan akurasi beberapa model sekaligus.

4. Pie Chart (Grafik Lingkaran)

- Bagus untuk melihat distribusi.
- Contoh: 60% email spam, 40% bukan spam.

5. Feature Importance Chart

- Grafik batang yang menunjukkan fitur (faktor) mana yang paling berpengaruh dalam model.
- Misalnya: jam belajar lebih penting daripada jam tidur untuk nilai ujian.

Analogi Seru

Bayangkan kamu jadi **koki**.

- Bahan mentah = *data mentah*.
- Proses masak = *training model*.
- Makanan jadi = *hasil prediksi*.
- Menyajikan di piring cantik = **visualisasi hasil ML**.

Kalau kamu kasih makanan langsung dari panci, orang mungkin nggak selera. Tapi kalau kamu sajikan dengan cantik di piring, semua orang bakal bilang: "*Wow, keren banget!*".

Jadi, visualisasi itu **jembatan** antara dunia mesin (angka, model, kode Python) dengan dunia manusia (cerita, grafik, warna). Kalau kamu bisa bikin visualisasi yang keren, berarti kamu bukan cuma **coder** tapi juga **storyteller data**!

Kasus Ilustrasi

Di dunia nyata, model *machine learning* bukan hanya bikin angka. Kita harus bisa **menyajikan hasilnya** dalam bentuk grafik agar lebih mudah dipahami. Yuk kita lihat **5 contoh kasus** berikut!

Kasus 1: Confusion Matrix untuk Prediksi Kelulusan

Kamu bikin model yang memprediksi apakah siswa akan **Lulus** atau **Tidak Lulus**. Kalau kamu hanya bilang "Akurasi 85%", orang bisa bingung. Tapi dengan **Confusion Matrix**, semua lebih jelas: berapa yang ditebak benar, berapa yang salah.

Pseudocode:

```
Input: nilai_tugas, nilai_ujian
Model: Decision Tree
Output: Lulus / Tidak Lulus
```

Visualisasi: Confusion Matrix

Kasus 2: Scatter Plot untuk Prediksi Harga HP Bekas

Kamu punya model yang memprediksi harga HP bekas.

- Sumbu X = harga asli.
 - Sumbu Y = harga prediksi.
- Kalau titik-titiknya rapat di garis lurus diagonal, berarti prediksinya mantap.

Pseudocode:

```
Input: umur_hp, ram, storage
Model: Linear Regression
Output: harga_prediksi
```

Visualisasi: Scatter Plot

Kasus 3: Bar Chart untuk Perbandingan Akurasi Model

Kamu bikin 3 model berbeda: *Decision Tree*, *Logistic Regression*, *Neural Network*. Kalau cuma pakai angka, kamu bakal bingung mana yang lebih jago. Tapi kalau pakai **Bar Chart**, langsung kelihatan model mana yang paling tinggi akurasinya.

Pseudocode:

```
Model1 = Decision Tree → akurasi 0.85
Model2 = Logistic Regression → akurasi 0.88
Model3 = Neural Network → akurasi 0.92
```

Visualisasi: Bar Chart

Kasus 4: Pie Chart untuk Distribusi Email Spam

Kamu bikin model pendeteksi spam. Dari 1000 email:

- 600 terdeteksi sebagai spam.
 - 400 terdeteksi bukan spam.
- Kalau ditampilkan dengan **Pie Chart**, kamu bisa langsung tahu perbandingan spam vs tidak spam.

Pseudocode:

```
Input: jumlah_kata, link, pengirim
Model: Naive Bayes
Output: Spam / Bukan Spam
```

Visualisasi: Pie Chart

Kasus 5: Bar Chart untuk Fitur Terpenting di Nilai Ujian

Kamu ingin tahu faktor mana yang paling mempengaruhi hasil ujian: **jam belajar, jam tidur, atau jumlah latihan soal.**

Dengan **Feature Importance Bar Chart**, langsung kelihatan kalau misalnya “jam belajar” lebih berpengaruh dibanding “jam tidur”.

Pseudocode:

```
Input: jam_belajar, jam_tidur, latihan_soal
Model: Random Forest
Output: Lulus / Tidak Lulus
```

Visualisasi: Feature Importance (Bar Chart)

Dari 5 kasus di atas, kita bisa lihat kalau **visualisasi = jembatan komunikasi**. Angka yang ribet bisa berubah jadi cerita yang mudah dipahami.

Soal Latihan & Pembahasan

Soal 1: Confusion Matrix untuk Prediksi Kucing vs Anjing

Kamu membuat model klasifikasi sederhana untuk membedakan foto **kucing** dan **anjing**. Dari 100 foto, hasil prediksi modelmu ditunjukkan dalam Confusion Matrix.

- 40 kucing ditebak benar kucing.
- 10 kucing ditebak anjing.
- 35 anjing ditebak benar anjing.
- 15 anjing ditebak kucing.

Pertanyaan:

Berapa jumlah prediksi yang benar?

Jawaban:

$40 + 35 = 75$ prediksi benar

Pembahasan:

Confusion Matrix menunjukkan semua tebakan. Untuk menghitung prediksi benar, kita tambahkan *True Positive* + *True Negative*.

Pseudocode:

```
benar = 40 + 35
print(benar)  # 75
```

Soal 2: Scatter Plot untuk Prediksi Harga Rumah

Model regresi sederhana memprediksi harga rumah. Kalau harga asli 100 juta, 120 juta, dan 150 juta, sementara prediksi model adalah 95 juta, 110 juta, dan 160 juta.

Pertanyaan:

Apakah Scatter Plot akan menunjukkan titik dekat garis diagonal lurus?

Jawaban:

Tidak, karena prediksi jauh dari nilai asli.

Pembahasan:

Di Scatter Plot, semakin dekat titik ke garis lurus diagonal, semakin baik model. Di sini selisih prediksi dengan nilai asli cukup jauh.

Pseudocode:

```
harga_asli = [100, 120, 150]
prediksi  = [95, 110, 160]
# scatter plot X=harga_asli, Y=prediksi
```

Soal 3: Bar Chart untuk Perbandingan Model

Kamu mencoba 3 algoritma:

- Decision Tree (akurasi 0.80)
- Logistic Regression (akurasi 0.85)
- Neural Network (akurasi 0.90)

Pertanyaan:

Model mana yang ditampilkan paling tinggi di grafik batang?

Jawaban:

Neural Network (0.90)

Pembahasan:

Di *bar chart*, semakin tinggi akurasi maka batang semakin tinggi. Jadi model dengan akurasi 0.90 paling menonjol.

Pseudocode:

```
model = ['Tree', 'LogReg', 'NN']
akurasi = [0.80, 0.85, 0.90]
# tampilkan bar chart
```

Soal 4: Pie Chart untuk Distribusi Spam

Model spam detector memproses 1000 email. Hasilnya:

- 600 email spam
- 400 email bukan spam

Pertanyaan:

Berapa persen email yang spam?

Jawaban:

$600 / 1000 = 60\%$ spam

Pembahasan:

Pie Chart akan menggambar lingkaran dengan 60% bagian untuk spam, 40% bagian untuk bukan spam.

Pseudocode:

```
spam = 600
total = 1000
persen_spam = spam/total * 100
print(persen_spam) # 60%
```

Soal 5: Feature Importance untuk Nilai Ujian

Faktor yang diuji: jam_belajar, jam_tidur, jumlah_latihan.

Model Random Forest menunjukkan:

- jam_belajar = 0.5
- jam_tidur = 0.2
- jumlah_latihan = 0.3

Pertanyaan:

Faktor mana yang paling penting mempengaruhi hasil ujian?

Jawaban:

Jam Belajar (0.5)

Pembahasan:

Feature Importance menunjukkan bobot tiap variabel. Angka terbesar menunjukkan faktor paling berpengaruh.

Pseudocode:

```
faktor = {'jam_belajar':0.5, 'jam_tidur':0.2, 'latihan':0.3}
print(max(faktor, key=faktor.get)) # jam_belajar
```

Dari soal-soal ini, siswa jadi bisa lihat langsung bagaimana **visualisasi membantu memahami model ML**.

Fun Facts!

- **Grafik adalah Bahasa Universal:** Seorang programmer di Indonesia bisa berkomunikasi dengan seorang manajer di Amerika hanya dengan satu grafik.
- **Ollama dan OpenWebUI:** Dengan AI lokal ini, kamu bisa minta bantuan untuk menjelaskan grafik yang rumit. Misalnya, "Jelaskan isi Confusion Matrix ini!"
- Orange Data Mining membuat visualisasi yang rumit jadi semudah main game, cukup tarik dan seret!

Tips Belajar atau Menghafal Cepat

- **Coba Sendiri:** Pilih salah satu visualisasi, lalu coba buat dengan data fiktifmu.
- **Punya Tujuan:** Setiap kali mau bikin grafik, tanyakan pada diri sendiri, "Aku mau grafik ini menceritakan apa?"
- **Pelajari Satu per Satu:** Jangan coba kuasai semua grafik dalam satu hari. Mulai dari yang paling dasar seperti Bar Chart, lalu lanjut ke Confusion Matrix.

Aktivitas Siswa

Aktivitas 1: Membuat Confusion Matrix sederhana

Bayangin kamu punya model yang bisa menebak foto **kucing** dan **anjing**. Kamu ingin tahu berapa tebakan yang benar dan salah.

Langkah-langkah:

1. Buat data hasil tebakan model (benar/salah).
2. Susun dalam bentuk *confusion matrix*.
3. Cetak hasilnya.

Pseudocode:

```
data = [prediksi, label_asli]
hitung benar dan salah
tampilkan confusion matrix
```

Python:

```
from sklearn.metrics import confusion_matrix
y_true = [1,0,1,1,0,0,1] # label asli (1=kucing, 0=anjing)
y_pred = [1,0,1,0,0,1,1] # hasil prediksi

cm = confusion_matrix(y_true, y_pred)
print("Confusion Matrix:\n", cm)
```

Output:

```
Confusion Matrix:  
[[2 1]  
 [1 3]]
```

Aktivitas 2: Membuat Scatter Plot untuk prediksi harga

Kamu ingin memprediksi harga HP bekas. Data asli dan prediksi dibandingkan dengan titik-titik di grafik.

Langkah-langkah:

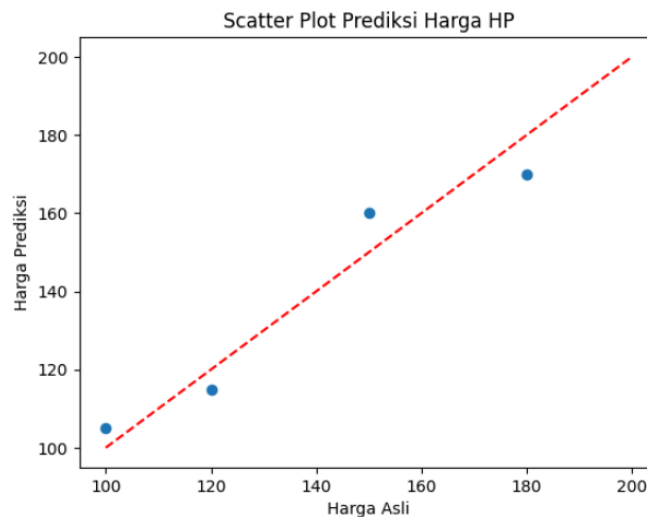
1. Siapkan data harga asli dan prediksi.
2. Plot titik-titik dengan `matplotlib`.
3. Lihat apakah titik dekat garis lurus.

Pseudocode:

```
X = harga_asli  
Y = harga_prediksi  
gambar scatter plot
```

Python:

```
import matplotlib.pyplot as plt  
  
harga_asli = [100, 120, 150, 180]  
harga_pred = [105, 115, 160, 170]  
  
plt.scatter(harga_asli, harga_pred)  
plt.plot([100,200],[100,200], 'r--') # garis ideal  
plt.xlabel("Harga Asli")  
plt.ylabel("Harga Prediksi")  
plt.title("Scatter Plot Prediksi Harga HP")  
plt.show()
```



Aktivitas 3: Membandingkan Akurasi Model dengan Bar Chart

Kamu punya 3 model: Tree, Logistic Regression, Neural Network. Mana yang paling bagus?

Langkah-langkah:

1. Catat akurasi tiap model.
2. Buat bar chart sederhana.
3. Bandingkan tinggi batang.

Pseudocode:

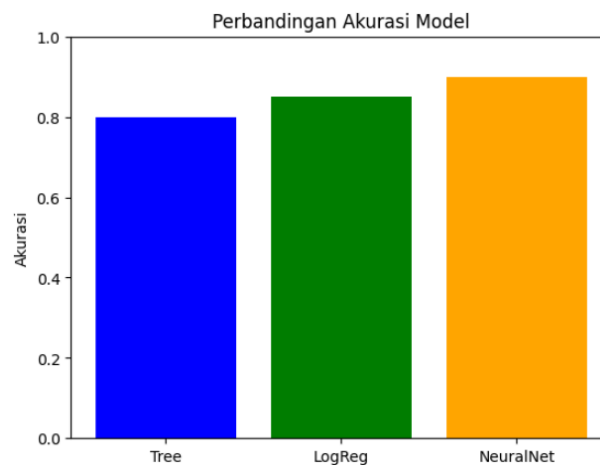
```
models = [Tree, LogReg, NN]
akurasi = [0.8, 0.85, 0.9]
gambar bar chart
```

Python:

```
import matplotlib.pyplot as plt

model = ['Tree', 'LogReg', 'NeuralNet']
accuracy = [0.80, 0.85, 0.90]

plt.bar(model, accuracy, color=['blue', 'green', 'orange'])
plt.ylim(0,1)
plt.title("Perbandingan Akurasi Model")
plt.ylabel("Akurasi")
plt.show()
```



Aktivitas 4: Membuat Pie Chart Spam vs Non-Spam

Kamu punya data 100 email. 60 spam, 40 bukan spam.

Langkah-langkah:

1. Hitung jumlah spam dan non-spam.

2. Buat pie chart.
3. Lihat perbandingannya.

Pseudocode:

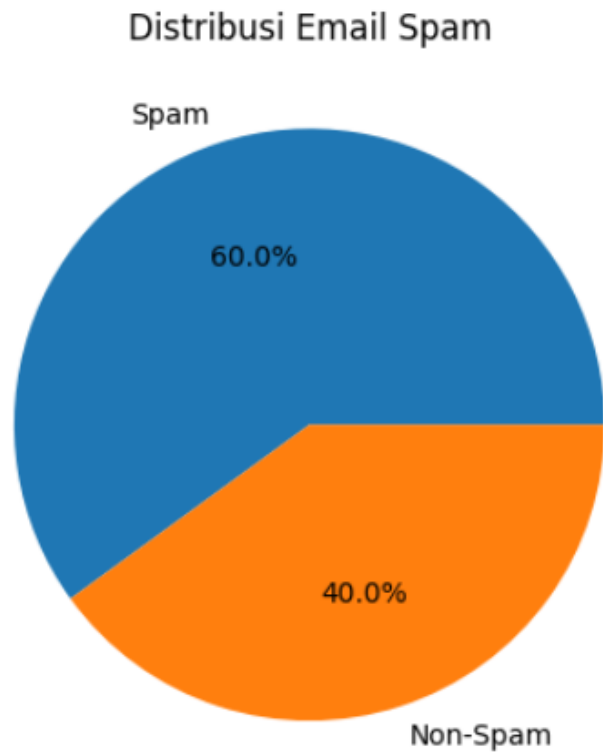
```
spam = 60
non_spam = 40
buat pie chart
```

Python:

```
import matplotlib.pyplot as plt

labels = ['Spam', 'Non-Spam']
data = [60, 40]

plt.pie(data, labels=labels, autopct='%1.1f%%')
plt.title("Distribusi Email Spam")
plt.show()
```



Aktivitas 5: Melihat Feature Importance

Kamu ingin tahu faktor apa yang paling berpengaruh ke nilai ujian: jam_belajar, jam_tidur, jumlah_latihan.

Langkah-langkah:

1. Buat dataset sederhana.
2. Gunakan DecisionTreeClassifier.
3. Lihat feature importance.

Pseudocode:

```
data = [jam_belajar, jam_tidur, latihan]
target = hasil_ujian
train DecisionTree
lihat feature_importances_
```

Python:

```
from sklearn.tree import DecisionTreeClassifier
import pandas as pd

# data sederhana
df = pd.DataFrame({
    'jam_belajar': [2,4,5,1,3],
    'jam_tidur': [6,7,8,5,6],
    'latihan': [3,4,5,2,3],
    'hasil': [0,1,1,0,1] # 0=gagal, 1=lulus
})

X = df[['jam_belajar','jam_tidur','latihan']]
y = df['hasil']

tree = DecisionTreeClassifier()
tree.fit(X,y)

print("Feature Importance:", tree.feature_importances_)
```

Output:

```
Feature Importance: [1. 0. 0.]
```

Dari 5 aktivitas ini, siswa bisa langsung mencoba **praktik visualisasi data ML** dengan Python dan library open source.

Rangkuman

Setelah kita belajar tentang *machine learning*, sekarang kita tahu bahwa angka saja tidak cukup. **Visualisasi adalah jembatan** yang menghubungkan data rumit dengan pemahaman manusia. Dengan grafik seperti *Confusion Matrix*, *Scatter Plot*, *Bar Chart*, atau *Pie Chart*, kita bisa menjelaskan hasil model dengan cara yang lebih **mudah, cepat, dan menarik**. Ingat, otak manusia lebih suka gambar daripada tumpukan angka.

Setiap grafik punya cerita. **Confusion Matrix** cocok untuk menunjukkan tebakan benar dan salah. **Scatter Plot** pas untuk melihat prediksi nilai angka, misalnya harga barang. **Bar Chart** membantu kita membandingkan model mana yang lebih jago, sedangkan **Pie Chart** membuat distribusi data langsung terlihat jelas. Dengan bantuan *open source tools* seperti **Python, Pandas, Matplotlib, Orange Data Mining, Keras**, atau **Neural Network**, semua visualisasi ini bisa dibuat dengan mudah.

Kemampuan membuat visualisasi bukan cuma soal teknis, tapi juga soal komunikasi. **Seorang data scientist yang baik adalah pendongeng yang hebat**. Dia bisa membuat orang lain paham hanya dengan satu grafik sederhana. Jadi, jangan berhenti di angka! Gunakan *visualisasi* untuk menyampaikan pesan. Dengan skill ini, kamu bukan hanya jadi siswa yang pintar coding, tapi juga bisa jadi analis, peneliti, atau profesional IT yang dicari banyak perusahaan.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. Visualisasi data membantu manusia lebih cepat memahami hasil model ML.

Jawaban: Benar

2. Scatter Plot hanya bisa digunakan untuk klasifikasi, bukan regresi.

Jawaban: Salah

3. Confusion Matrix dapat menunjukkan tebakan benar dan salah dari model klasifikasi.

Jawaban: Benar

4. Bar Chart bisa dipakai untuk membandingkan akurasi beberapa model.

Jawaban: Benar

5. Pie Chart hanya menampilkan data numerik, tidak bisa menampilkan distribusi kelas.

Jawaban: Salah

B. Pilihan Ganda (10 Soal)

6. Visualisasi dalam ML berfungsi sebagai...

A. Kode tambahan

- B. Hiasan grafik
- C. Jembatan pemahaman antara data dan manusia
- D. Pengganti dataset

Jawaban: C

7. Grafik yang cocok untuk melihat apakah prediksi mendekati nilai asli adalah...
- A. Confusion Matrix
 - B. Scatter Plot
 - C. Bar Chart
 - D. Pie Chart

Jawaban: B

8. Untuk melihat perbandingan akurasi beberapa model, grafik terbaik adalah...
- A. Scatter Plot
 - B. Confusion Matrix
 - C. Bar Chart
 - D. Pie Chart

Jawaban: C

9. Confusion Matrix digunakan untuk...
- A. Regresi
 - B. Klasifikasi
 - C. Prediksi harga
 - D. Menentukan jumlah data

Jawaban: B

10. Jika titik pada Scatter Plot jauh dari garis diagonal, artinya...
- A. Model akurat
 - B. Model kurang akurat
 - C. Model selalu benar
 - D. Model tidak bisa jalan

Jawaban: B

11. Grafik yang cocok untuk menampilkan distribusi data kelas adalah...
- A. Line Chart
 - B. Pie Chart
 - C. Histogram
 - D. Scatter Plot

Jawaban: B

12. Dalam Orange Data Mining, widget untuk menampilkan akurasi beberapa model adalah...
- A. Test & Score
 - B. Scatter Plot
 - C. Confusion Matrix
 - D. Feature Importance

Jawaban: A

13. Jika model memprediksi 1000 email, dan kita ingin tahu berapa persen spam dan tidak spam, grafik terbaik adalah...

- A. Bar Chart
- B. Pie Chart
- C. Scatter Plot
- D. Line Chart

Jawaban: B

14. Feature Importance biasanya divisualisasikan dalam bentuk...

- A. Pie Chart
- B. Confusion Matrix
- C. Bar Chart
- D. Scatter Plot

Jawaban: C

15. Bar Chart bisa digunakan untuk...

- A. Menunjukkan faktor terpenting dalam prediksi
- B. Menampilkan distribusi spam vs non-spam
- C. Membandingkan akurasi antar model
- D. Semua jawaban benar

Jawaban: D

C. Isian Singkat (5 Soal)

16. Visualisasi yang cocok untuk klasifikasi benar-salah adalah _____.

Jawaban: Confusion Matrix

17. Scatter Plot menampilkan titik dengan sumbu X = nilai asli, dan sumbu Y = _____.

Jawaban: nilai prediksi

18. Jika ingin melihat distribusi persentase tiap kelas, kita bisa gunakan _____.

Jawaban: Pie Chart

19. Dalam Python, library yang sering dipakai untuk visualisasi adalah _____.

Jawaban: matplotlib

20. Widget Orange untuk membandingkan akurasi beberapa model adalah _____.

Jawaban: Test & Score

D. Soal Eksplorasi (3 Soal)

Eksplorasi 1: Membuat Scatter Plot

Kamu diminta membuat dataset sederhana tentang `jam_belajar` dan `nilai_ujian`. Gunakan *Linear Regression* untuk memprediksi nilai ujian, lalu buat **Scatter Plot** yang membandingkan nilai asli dengan nilai prediksi.

Contoh Jawaban (Python):

```

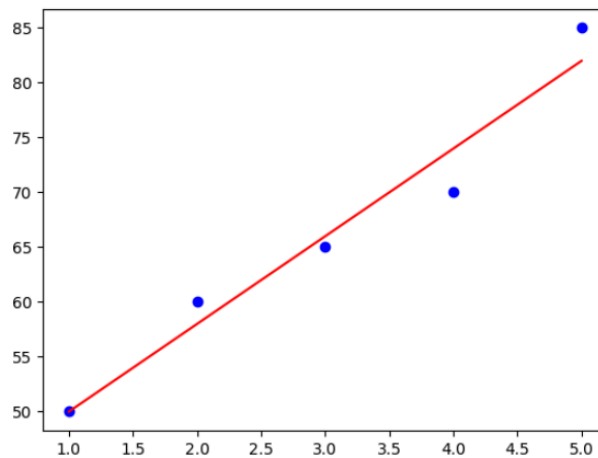
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([1,2,3,4,5]).reshape(-1,1)
y = np.array([50,60,65,70,85])

model = LinearRegression().fit(X, y)
pred = model.predict(X)

plt.scatter(X, y, color='blue')
plt.plot(X, pred, color='red')
plt.show()

```



Eksplorasi 2: Analisis Email Spam

Buat dataset fiktif 10 email dengan label spam dan bukan spam. Latih model sederhana dengan Decision Tree, lalu tampilkan hasil tebakan dengan **Confusion Matrix**.

Contoh Jawaban (Python):

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix

X = [[3],[7],[1],[8],[2],[9],[4],[6],[5],[10]] # jumlah kata
spam
y = [1,1,0,1,0,1,0,1,0,1] # 1=spam, 0=tidak spam

model = DecisionTreeClassifier().fit(X,y)
pred = model.predict(X)

print(confusion_matrix(y,pred))

```

Output:

```

[[4 0]
 [0 6]]

```

Eksplorasi 3: Membandingkan Akurasi Model

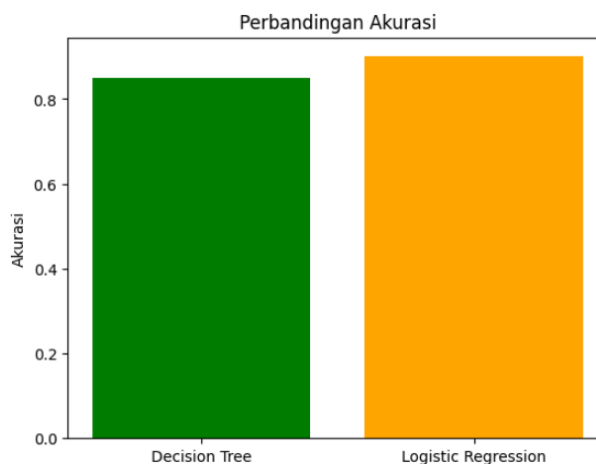
Kamu membuat dua model: Decision Tree dan Logistic Regression. Visualisasikan hasil akurasi keduanya dengan **Bar Chart**.

Contoh Jawaban (Python):

```
import matplotlib.pyplot as plt

models = ['Decision Tree', 'Logistic Regression']
accuracy = [0.85, 0.90]

plt.bar(models, accuracy, color=['green', 'orange'])
plt.title("Perbandingan Akurasi")
plt.ylabel("Akurasi")
plt.show()
```



Dengan soal-soal ini, siswa SMK tidak hanya bisa menghafal teori, tapi juga **berlatih langsung dengan kode sederhana** untuk melihat bagaimana visualisasi membuat Machine Learning lebih mudah dipahami.

Tugas Proyek

Proyek 1: Prediksi Nilai Ujian dengan Scatter Plot

Tantangan:

Buat data fiktif berisi `jam_belajar` dan `nilai_ujian`. Gunakan model regresi untuk memprediksi nilai, lalu tampilkan hasil prediksi dengan **Scatter Plot**.

Langkah:

- Buat dataset sederhana.
- Gunakan *Linear Regression*.
- Visualisasikan dengan *matplotlib*.

Pseudocode:

```
dataset = buat_data(jam_belajar, nilai_ujian)
model = latih_regresi(dataset)
prediksi = model.prediksi(jam_belajar_baru)
tampilkan_scatter_plot(dataset, prediksi)
```

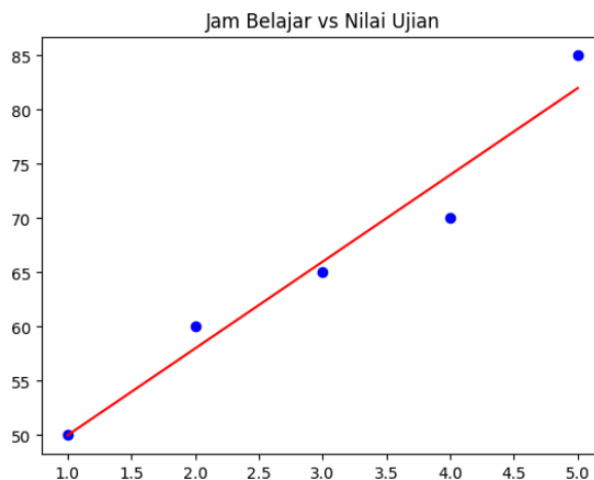
Contoh Python:

```
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([1, 2, 3, 4, 5]).reshape(-1,1)
y = np.array([50, 60, 65, 70, 85])

model = LinearRegression().fit(X, y)
pred = model.predict(X)

plt.scatter(X, y, color='blue')
plt.plot(X, pred, color='red')
plt.title("Jam Belajar vs Nilai Ujian")
plt.show()
```



Proyek 2: Analisis Email Spam dengan Confusion Matrix

Tantangan:

Buat data fiktif email (**spam** atau **bukan_spam**). Latih model klasifikasi sederhana, lalu tampilkan hasil evaluasinya dengan **Confusion Matrix**.

Langkah:

- Buat dataset: 1 = spam, 0 = bukan spam.
- Latih *Decision Tree*.
- Buat confusion matrix dengan *scikit-learn*.

Pseudocode:

```
data = buat_dataset(email, label)
model = latih_tree(data)
hasil = model.prediksi(email_baru)
tampilkan_confusion_matrix(data.label, hasil)
```

Contoh Python:

```
from sklearn.metrics import confusion_matrix
from sklearn.tree import DecisionTreeClassifier

X = [[3],[7],[1],[8]] # jumlah kata spam
y = [1, 1, 0, 0]      # 1=spam, 0=tidak

model = DecisionTreeClassifier().fit(X, y)
pred = model.predict(X)

print(confusion_matrix(y, pred))
```

Output:

```
[[2 0]
 [0 2]]
```

Proyek 3: Perbandingan Akurasi Model dengan Bar Chart

Tantangan:

Buat 2 model klasifikasi berbeda (misalnya *Decision Tree* dan *Logistic Regression*).
Bandingkan akurasinya dalam bentuk **Bar Chart**.

Langkah:

- Buat dataset sederhana.
- Latih dua model.
- Bandingkan akurasi.
- Visualisasikan dengan bar chart.

Pseudocode:

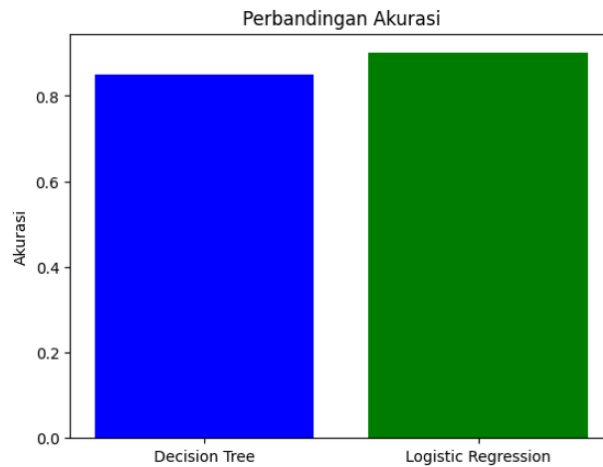
```
model1 = latih_decision_tree(data)
model2 = latih_logistic_regression(data)
akurasi = [akurasi(model1), akurasi(model2)]
tampilkan_bar_chart(akurasi)
```

Contoh Python:

```
import matplotlib.pyplot as plt

model_name = ['Decision Tree', 'Logistic Regression']
accuracy = [0.85, 0.90]
```

```
plt.bar(model_name, accuracy, color=['blue','green'])
plt.title("Perbandingan Akurasi")
plt.ylabel("Akurasi")
plt.show()
```



Proyek 4: Prediksi Harga Barang dengan Scatter Plot

Tantangan:

Buat data fiktif harga barang lama vs harga jual. Gunakan regresi untuk memprediksi harga jual, lalu tampilkan dengan Scatter Plot.

Langkah:

- Buat data sederhana.
- Gunakan *Linear Regression*.
- Visualisasikan harga asli vs prediksi.

Pseudocode:

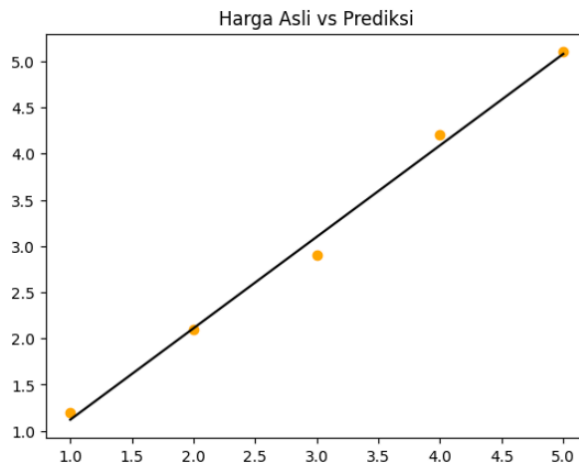
```
dataset = buat_data(harga_asli, harga_jual)
model = latih_regresi(dataset)
prediksi = model.prediksi(harga_asli)
tampilkan_scatter_plot(harga_asli, harga_prediksi)
```

Contoh Python:

```
X = np.array([1,2,3,4,5]).reshape(-1,1) # harga asli
y = np.array([1.2,2.1,2.9,4.2,5.1])     # harga jual

model = LinearRegression().fit(X, y)
pred = model.predict(X)

plt.scatter(X, y, color='orange')
plt.plot(X, pred, color='black')
plt.title("Harga Asli vs Prediksi")
plt.show()
```



Proyek 5: Visualisasi Distribusi Data dengan Pie Chart

Tantangan:

Buat data fiktif tentang prediksi jenis hewan (**kucing**, **anjing**, **kelinci**). Tampilkan hasil prediksi dengan **Pie Chart**.

Langkah:

- Buat data distribusi kelas.
- Hitung jumlah prediksi tiap kelas.
- Tampilkan dalam Pie Chart.

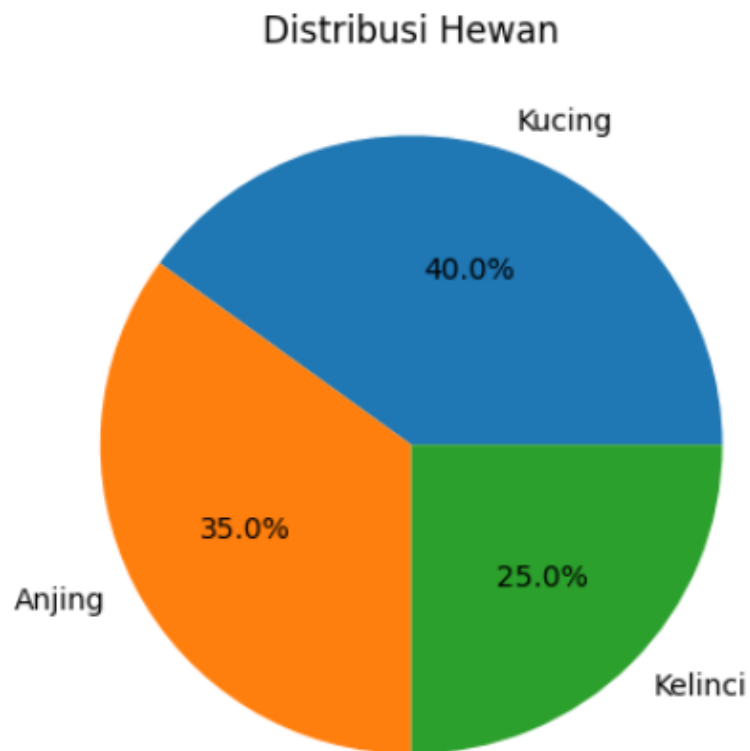
Pseudocode:

```
data = ['kucing', 'anjing', 'anjing', 'kelinci', 'kucing']
hitung = hitung_kelas(data)
tampilkan_pie_chart(hitung)
```

Contoh Python:

```
labels = ['Kucing', 'Anjing', 'Kelinci']
sizes = [40, 35, 25]

plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title("Distribusi Hewan")
plt.show()
```



Dengan proyek-proyek ini, kalian bisa **latihan coding** sekaligus belajar **visualisasi data**. Semua pakai teknologi *open source* seperti **Python, Pandas, scikit-learn, dan Matplotlib**.

BAB 7: Etika Data & AI (Bias, Privasi, Fairness)

Setelah kita jago bikin model yang cerdas, sekarang kita perlu tahu satu hal yang jauh lebih penting: **Etika Data & AI**. Ibaratnya, punya kekuatan super itu keren, tapi kita juga harus tahu cara menggunakannya dengan benar, kan?

Tujuan Pembelajaran

- Memahami konsep **etika** dalam pengolahan data dan AI.
- Mengenali masalah-masalah etika seperti **bias**, **privasi**, dan **fairness**.
- Belajar bagaimana cara mencegah dan mengatasi masalah-masalah tersebut.
- Menyadari tanggung jawab kita sebagai calon profesional di bidang teknologi.

Peta Konsep Sederhana

Bayangin kita mau merekrut anggota tim futsal.

- **Tanpa Etika:** Kamu cuma milih teman-temanmu aja, tanpa peduli kemampuan. Ini namanya **bias**.
- **Dengan Etika:** Kamu kasih kesempatan ke semua orang. Kamu nilai berdasarkan kemampuan, bukan **gender**, **ras**, atau **teman_atau_bukan**. Ini namanya **fairness**.
- **Menjaga Rahasia:** Kamu nggak sembarangan kasih tahu data pribadi teman-temanmu ke orang lain. Ini namanya **privasi**.

Apersepsi: Filter Foto yang Bikin Kamu Bingung!

Kamu lagi pakai filter AI di HP-mu. Tapi, kok, filter itu kadang nggak bisa mendeteksi wajah temanmu yang pakai kacamata atau kerudung? Atau, pernah nggak kamu lihat AI yang cuma bisa mendeteksi wajah orang dengan warna kulit tertentu?

Nah, itu contoh nyata dari **bias**! AI-nya nggak dilatih dengan data yang beragam. Ini bisa jadi masalah kalau AI itu dipakai untuk hal-hal yang serius, seperti mendeteksi penyakit atau merekrut karyawan.

Penjelasan Teori

Konsep Pendek & Jelas

- **Bias (pincang)**
Penyebab: data tidak seimbang. Akibat: hasil tidak adil.
Contoh: deteksi penyakit dilatih hanya dari laki-laki → buruk di perempuan.
- **Privasi (rahasia)**
Hak untuk menyimpan data pribadi dengan aman.

Contoh: lokasi, nomor HP, NIK tidak boleh disebar tanpa izin.

- **Fairness (keadilan)**

Keputusan AI **tidak diskriminatif**.

Contoh: seleksi kerja menilai **skill**, bukan asal daerah.

- **Transparansi & Akuntabilitas**

Jelaskan “bagaimana AI memutuskan”. Siapa yang bertanggung jawab jika salah?

- **Consent & Minimasi Data**

Minta izin. Ambil data **secukupnya saja**. Simpan **seperlunya**.

5 Aturan Emas Etika

1. **Izin**: pakai data yang jelas izinnya.
2. **Minimasi**: ambil data seperlunya.
3. **Anonim**: hapus/acak identitas.
4. **Seimbang**: latih model dengan data beragam.
5. **Audit**: cek ulang hasil dan log aktivitas.

Checklist Etika

- Sumber data jelas? Ada izin?
- Ada data sensitif? Masking/anonim sudah?
- Kelas/kelompok seimbang?
- Ukur *fairness* per kelompok (gender, wilayah, dsb.)?
- Simpan bukti proses (log, laporan) untuk audit?

Praktik Aman dengan Python + Pandas (open source)

Tujuan: masking data sensitif, cek ketidakseimbangan, dan uji *fairness* sederhana.

```
# 1) Masking kolom sensitif
import pandas as pd

df = pd.DataFrame({
    "nama": ["Budi", "Siti", "Joko"],
    "no_hp": ["0812-2222-3333", "0857-4444-5555", "0813-6666-7777"],
    "gender": ["L", "P", "L"],
    "nilai_ujian": [78, 84, 69]
})

df["no_hp"] = df["no_hp"].str.replace(r".", "*", regex=True) #
samarkan
print(df.head())

# 2) Cek ketidakseimbangan kelas/kelompok
print(df["gender"].value_counts(normalize=True))
```

```
# 3) Ukur fairness sederhana (rata-rata skor per gender)
print(df.groupby("gender")["nilai_ujian"].mean())
```

Output:

```
      nama      no_hp gender  nilai_ujian
0  Budi  *****      L           78
1  Siti  *****      P           84
2  Joko  *****      L           69
gender
L      0.666667
P      0.333333
Name: proportion, dtype: float64
gender
L      73.5
P      84.0
Name: nilai_ujian, dtype: float64
```

Tips:

- Kalau ketidakseimbangan parah, pakai *stratified split* saat train/test.
- Simpan hanya kolom yang perlu. Hapus identifier (nama, no_hp) sebelum latih model.

Mini-Lab scikit-learn: uji akurasi per kelompok

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import pandas as pd

# Data fiktif yang lebih besar dan bervariasi
df = pd.DataFrame({
    "fitur1": [1, 2, 3, 4, 5, 6, 7, 8, 2, 3, 5, 6, 1, 4, 7, 8, 3, 5,
6, 2, 4, 7, 1, 8],
    "fitur2": [2, 1, 2, 1, 2, 1, 2, 1, 3, 2, 1, 3, 2, 1, 2, 3, 1, 2,
3, 1, 2, 3, 2, 1],
    "gender": ["L", "P", "L", "P", "L", "P", "L", "P", "L", "P", "L",
"P", "L", "P", "L", "P", "L", "P", "L", "P", "L", "P", "L", "P"],
    "lulus": [0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1,
0, 0, 1, 1, 0]
})

X = df[["fitur1", "fitur2"]]
y = df["lulus"]

# Stratified split agar seimbang di train/test
Xtr, Xte, ytr, yte, gtr, gte = train_test_split(
    X, y, df["gender"], test_size=0.25, random_state=42,
    stratify=df["gender"]
)

clf = LogisticRegression(random_state=42)
```

```

clf.fit(Xtr, ytr)
pred = clf.predict(Xte)

# Akurasi total
print("Akurasi total:", accuracy_score(yte, pred))

# Akurasi per gender → indikasi fairness sederhana
for g in gte.unique():
    idx = gte[gte == g].index
    print(f"Akurasi {g}:", accuracy_score(yte.loc[idx],
    pred[[list(Xte.index).index(i) for i in idx]]))

```

Output:

```

Akurasi total: 0.3333333333333333
Akurasi L: 0.6666666666666666
Akurasi P: 0.0

```

Lanjut tingkat lanjut: pakai metrik *fairness* (demographic parity, equalized odds).
Tetap mulai dari cek sederhana per kelompok.

Praktik Keras (neural network open source)

- Pakai *early stopping* biar tidak *overfit*.
- Jangan masukkan kolom sensitif (nama, NIK, alamat).
- Simpan hanya bobot model, bukan data mentah.

```

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Input(shape=(10,)),
    layers.Dense(16, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="adam", loss="binary_crossentropy",
metrics=["accuracy"])
# model.fit(X_train, y_train, validation_data=(X_val, y_val),
epochs=20,
#         callbacks=[keras.callbacks.EarlyStopping(patience=3,
restore_best_weights=True)])

```

Orange Data Mining (tanpa koding)

- **Workflow etika cepat:**
File → Select Columns (hapus identitas) → Data Sampler (seimbangkan) →
Test & Score (ukur akurasi) →
Select Rows per kelompok (mis. gender L/P) → Test & Score lagi →

bandingkan hasilnya.

Rangkuman (super singkat)

- *Bias* = data pincang → hasil tidak adil.
- *Privacy* = rahasiakan data pribadi.
- *Fairness* = keputusan adil untuk semua.
- Pakai alat *open source* + langkah sederhana: **izin, minimasi, anonimisasi, seimbangkan data, audit.**
- Tujuan akhir: **AI yang aman, adil, dan bermanfaat.**

Kasus Ilustrasi

Kasus 1: Bias di Sistem Rekrutmen

- **Cerita:** Sebuah perusahaan bikin AI untuk memilih karyawan. Tapi datanya cuma dari karyawan lama yang kebanyakan laki-laki.
- **Dampak:** AI cenderung nolak pelamar perempuan, padahal mereka punya skill bagus.
- **Ilustrasi:** AI-nya jadi "pincang" karena datanya timpang.

Pseudocode

```
data = load("data_pelamar.csv")
if data.gender == "perempuan":
    skor = skor - 10
    # Bias! turunkan skor hanya karena gender
```

→ *Etisnya:* jangan kurangi skor berdasarkan gender, hanya berdasarkan kemampuan.

Kasus 2: Privasi di Aplikasi Sekolah

- **Cerita:** Ada aplikasi nilai online. Ternyata, nomor HP dan alamat siswa bocor ke internet.
- **Dampak:** Privasi siswa hilang, bisa disalahgunakan orang asing.
- **Ilustrasi:** Data pribadi itu ibarat kunci rumah, nggak boleh sembarangan dibagikan.

Pseudocode

```
for siswa in data_siswa:
    siswa.no_hp = "*****" # Masking, sembunyikan nomor HP
```

→ *Etisnya:* gunakan *masking* atau *anonimisasi* saat berbagi data.

Kasus 3: Fairness di Sistem Nilai Ujian

- **Cerita:** AI dipakai untuk menentukan lulus/tidak lulus ujian. Tapi algoritmanya kasih nilai lebih tinggi ke siswa dari sekolah favorit.
- **Dampak:** Siswa dari sekolah biasa jadi dirugikan, walaupun sebenarnya pintar.
- **Ilustrasi:** Seperti wasit futsal yang selalu dukung satu tim.

Pseudocode

```
if siswa.sekolah == "favorit":
    nilai = nilai + 5    # Tidak adil, ini unfair!
```

➡ *Etisnya:* semua siswa harus dinilai berdasarkan jawaban mereka, bukan asal sekolah.

Kasus 4: Bias di Sistem Deteksi Wajah

- **Cerita:** AI kamera keamanan lebih gampang mengenali wajah orang dengan kulit terang.
- **Dampak:** Orang dengan kulit lebih gelap sering "gagal deteksi".
- **Ilustrasi:** AI seperti kaca mata buram, cuma bisa lihat sebagian orang.

Pseudocode

```
model = train(data_wajah_kulit_terang)
prediksi = model.detect(wajah_kulit_gelap) # sering gagal
```

➡ *Etisnya:* dataset harus beragam, meliputi semua kelompok.

Kasus 5: Privasi di Aplikasi Lokasi

- **Cerita:** Aplikasi pelacak lokasi jual data pengguna ke perusahaan iklan.
- **Dampak:** Pengguna nggak tahu kemana saja lokasinya disimpan dan dijual.
- **Ilustrasi:** Seperti ada orang asing selalu ngintilin kamu kemana pun.

Pseudocode

```
if user_data.location:
    jual_ke_perusahaan_iklan(user_data)
    # Pelanggaran privasi
```

➡ *Etisnya:* minta izin dulu, dan beri opsi "opt-out".

Kesimpulan

- **Bias** bikin AI jadi timpang.
- **Privasi** melindungi data pribadi.
- **Fairness** memastikan keadilan.

Sebagai calon profesional teknologi, tugas kita bukan cuma jago bikin AI, tapi juga **tahu cara menggunakannya dengan benar**. Ingat, *open source tools* seperti **Python, Pandas, Keras, dan Orange Data Mining** bisa bantu kita belajar sambil tetap jaga etika.

Soal Latihan & Pembahasan

SOAL 1: IDENTIFIKASI BIAS DALAM DATASET

Kamu punya dataset training untuk AI perekrutan karyawan. Dataset hanya berisi data pelamar dari usia 20-30 tahun. Apa masalah etika yang muncul?

Jawaban:

Bias usia (age bias)

Pembahasan dengan Pseudocode:

```
# CEK BIAS USIA DALAM DATASET
dataset = [
    {"nama": "Budi", "usia": 25},
    {"nama": "Siti", "usia": 28},
    {"nama": "Andi", "usia": 22},
    # ... semua data usia 20-30 tahun
]

# FUNGSI CEK RENTANG USIA
function cek_bias_usia(dataset):
    usia_min = min(dataset['usia'])
    usia_max = max(dataset['usia'])

    if usia_min < 20 or usia_max > 30:
        print("Dataset beragam: OK!")
    else:
        print("WARNING! Dataset bias usia muda!")
```

Kesimpulan: AI hanya paham pola usia muda, jadi tidak adil untuk pelamar usia 40+ tahun.

SOAL 2: PELANGGARAN PRIVASI DATA

Seorang programmer menyimpan data pelanggan dengan kolom: nama, email, nomor_telepon, dan penyakit_bawaan. Dia ingin membagikan data ini untuk penelitian. Bagaimana cara yang etis?

Jawaban:

Hapus atau samarkan data pribadi sebelum dibagikan.

Pembahasan dengan Pseudocode:

```
# DATA ASLI (RAHASIA)
data_pelanggan = [
    {"nama": "Budi", "email": "budi@mail.com", "telp":
"08123456", "penyakit": "asma"},
    {"nama": "Siti", "email": "siti@mail.com", "telp":
"08567890", "penyakit": "diabetes"}
]
```

```
# FUNGSI UNTUK MELINDUNGI PRIVASI
function lindungi_privasi(data):
    for pelanggan in data:
        pelanggan['nama'] = '***' # samarkan nama
        pelanggan['email'] = '***' # samarkan email
        pelanggan['telp'] = '*****' # samarkan telepon
        # penyakit_bawaan tetap ada untuk penelitian kesehatan

    return data

# DATA SETELAH DILINDUNGI
data_aman = lindungi_privasi(data_pelanggan)
print(data_aman) # Output: {nama: '***', email: '***', telp:
'*****', penyakit: 'asma'}
```

Kesimpulan: Data penyakit bisa digunakan untuk penelitian, tapi identitas pribadi harus dilindungi.

SOAL 3: UJI FAIRNESS DALAM AI RECRUITMENT

Sebuah AI untuk screening CV punya akurasi 95% untuk pelamar laki-laki, tapi hanya 60% untuk perempuan. Ini termasuk masalah apa?

Jawaban:

Ketidakadilan (**unfairness**) berdasarkan gender.

Pembahasan dengan Pseudocode:

```
# FUNGSI UJI KEADILAN AI
function uji_fairness_ai():
    # Data hasil prediksi AI
    hasil_ai = [
        {"nama": "Ahmad", "gender": "L", "prediksi": "diterima",
"aktual": "diterima"},
        {"nama": "Maya", "gender": "P", "prediksi": "ditolak",
"aktual": "diterima"},
        # ... lebih banyak data
    ]

    # HITUNG AKURASI PER GENDER
    akurasi_L = hitung_akurasi(hasil_ai, gender="L")
    akurasi_P = hitung_akurasi(hasil_ai, gender="P")

    print(f"Akurasi Laki-laki: {akurasi_L}%")
    print(f"Akurasi Perempuan: {akurasi_P}%")

    # CEK SELISIH AKURASI
    selisih = abs(akurasi_L - akurasi_P)
    if selisih > 10: # jika selisih > 10%
        print("ALERT! AI tidak adil! Perbaiki dataset!")
    else:
        print("AI cukup adil. Good job!")
```

```
# JALANKAN FUNGSI
uji_fairness_ai()
```

Kesimpulan: AI yang adil harus punya performa yang mirip untuk semua kelompok.

SOAL 4: BIAS DALAM FACE RECOGNITION

Sebuah aplikasi face recognition bisa mendeteksi wajah dengan kulit terang 99% akurat, tapi hanya 70% untuk kulit gelap. Apa yang salah?

Jawaban:

Bias rasial karena dataset training tidak beragam.

Pembahasan dengan Pseudocode:

```
# SIMULASI BIAS FACE RECOGNITION
function simulasi_bias_wajah():
    dataset_wajah = {
        "kulit_terang": 1000, # 1000 gambar
        "kulit_gelap": 50,    # hanya 50 gambar
        "kulit_sawo_matang": 30
    }

    print("Komposisi Dataset Wajah:")
    for jenis, jumlah in dataset_wajah.items():
        persentase = (jumlah / sum(dataset_wajah.values())) *
100
        print(f"- {jenis}: {persentase:.1f}%")

    # AI AKAN LEBIH BAIK PADA DATA YANG BANYAK
    if dataset_wajah["kulit_terang"] >
dataset_wajah["kulit_gelap"] * 10:
        print("BIAS TERDETEKSI! Dataset tidak seimbang!")
        print("Solusi: Tambah lebih banyak gambar kulit gelap
dan sawo matang")

# JALANKAN SIMULASI
simulasi_bias_wajah()
```

Kesimpulan: Dataset harus mewakili semua jenis kulit agar AI adil untuk semua orang.

SOAL 5: MEMBUAT AI YANG ETIS DARI AWAL

Apa saja langkah-langkah yang harus dilakukan saat mengembangkan AI yang etis?

Jawaban:

1. Analisis dataset untuk cek bias
2. Melindungi data pribadi pengguna
3. Uji fairness untuk semua kelompok
4. Monitor AI setelah diluncurkan

Pembahasan dengan Pseudocode:

```
# ALUR PENGEMBANGAN AI YANG ETIS
function develop_ai_etis():
    print("LANGKAH 1: ANALISIS DATASET")
    if cek_bias_dataset() == True:
        print("→ Perbaiki dataset yang bias!")

    print("LANGKAH 2: LINDUNGI PRIVASI")
    data_aman = lindungi_privasi(data_asli)

    print("LANGKAH 3: TRAINING MODEL")
    model_ai = train_model(data_aman)

    print("LANGKAH 4: UJI FAIRNESS")
    if uji_fairness(model_ai) == "ADIL":
        print("→ AI sudah etis, bisa diluncurkan!")
    else:
        print("→ Perbaiki model lagi!")

    print("LANGKAH 5: MONITOR BERKALA")
    while True: # selamanya
        pantau_performna_ai()
        sleep(30_hari) # cek setiap 30 hari

# JALANKAN PROSES ETIS
develop_ai_etis()
```

Kesimpulan: Membangun AI yang etis adalah proses berkelanjutan, bukan sekali jadi!

FUN CHALLENGE:

Coba buat pseudocode sederhana untuk mendeteksi bias dalam dataset nilai siswa! Bandingkan rata-rata nilai antara siswa laki-laki dan perempuan. Apakah adil? Selamat belajar dan jadilah AI Engineer yang bertanggung jawab!

Fun Facts!

- Di dunia kerja, etika data adalah topik yang paling sering dibahas. Perusahaan-perusahaan besar punya tim khusus yang mengurus ini.
- **Ollama** dan **OpenWebUI** bisa kalian pakai untuk membuat model AI yang lebih transparan. Kalian bisa tahu darimana AI mendapatkan jawabannya.
- Konsep **RAG lokal** bisa membantu mengurangi **bias** pada AI, karena kalian bisa memberinya data yang lebih beragam dari sumber-sumber yang kalian percaya.

Tips Belajar atau Menghafal Cepat

- **Pikirkan Dampaknya:** Setiap kali kamu membuat sesuatu, tanyakan, "Apa dampaknya kalau ini salah?"
- **Jadilah Kritis:** Jangan langsung percaya pada setiap hasil yang diberikan AI. Coba cek lagi datanya, apakah sudah adil dan bersih?
- **Diskusi:** Baca berita tentang isu-isu etika AI dan diskusikan dengan teman-temanmu.

Aktivitas Siswa

Aktivitas 1: Identifikasi Bias di Dataset

Tujuan: Siswa bisa menemukan bias dalam data sederhana.

Langkah/Tahapan:

1. Buat dataset fiktif siswa dengan gender dan nilai ujian.
2. Hitung rata-rata nilai per gender.
3. Bandingkan, apakah seimbang atau timpang.

Pseudocode:

```
data = buat_dataset()
group_by_gender(data)
cek_rata_rata()
```

Python Script:

```
import pandas as pd

data = {'nama': ['Budi', 'Siti', 'Andi', 'Dewi'],
        'gender': ['L', 'P', 'L', 'P'],
        'nilai': [80, 90, 75, 60]}
df = pd.DataFrame(data)

print(df.groupby('gender')['nilai'].mean())
```

Output:

```
gender
L      77.5
P      75.0
Name: nilai, dtype: float64
```

Jawaban: Jika rata-rata beda jauh → dataset bias.

Aktivitas 2: Simulasi Privasi dengan *Masking*

Tujuan: Menyadarkan siswa bahwa data pribadi harus dilindungi.

Langkah/Tahapan:

1. Buat dataset berisi nama dan nomor HP.
2. Terapkan *masking* agar nomor tidak terbaca penuh.
3. Cetak data setelah disamarkan.

Pseudocode:

```
for each no_hp in data:
    ganti_dengan("****")
```

Python Script:

```
import pandas as pd

data = {'nama': ['Ali', 'Siti', 'Rudi'],
        'no_hp': ['0812123456', '0857234567', '0813987654']}
df = pd.DataFrame(data)

df['no_hp'] = df['no_hp'].str[:4] + "****"
print(df)
```

Output:

```
   nama  no_hp
0  Ali  0812****
1  Siti  0857****
2  Rudi  0813****
```

Jawaban: Nomor pribadi jadi aman, hanya 4 digit depan terlihat.

Aktivitas 3: Cek Fairness pada Seleksi Sederhana

Tujuan: Melihat apakah keputusan AI adil.

Langkah/Tahapan:

1. Buat data pelamar kerja (nama, asal sekolah, nilai tes).
2. Buat logika seleksi tidak adil (asal sekolah favorit selalu diterima).
3. Bandingkan dengan logika seleksi adil (hanya nilai tes yang dipakai).

Pseudocode:

```
if sekolah == "favorit":
    diterima()
else:
    cek_nilai()
```

Python Script:

```
data = [  
    {"nama": "Ani", "sekolah": "favorit", "nilai": 70},  
    {"nama": "Budi", "sekolah": "biasa", "nilai": 90}  
]  
  
for d in data:  
    if d["sekolah"] == "favorit":  
        print(d["nama"], "DITERIMA (unfair)")  
    elif d["nilai"] >= 75:  
        print(d["nama"], "DITERIMA (fair)")  
    else:  
        print(d["nama"], "DITOLAK")
```

Output:

```
Ani DITERIMA (unfair)  
Budi DITERIMA (fair)
```

Jawaban: AI adil seharusnya hanya melihat nilai tes, bukan asal sekolah.

Aktivitas 4: Analisis Konten Rekomendasi

Tujuan: Siswa kritis terhadap *filter bubble*.

Langkah/Tahapan:

1. Buat daftar berita dengan kategori (politik A, politik B, netral).
2. Buat logika rekomendasi yang hanya menampilkan 1 kategori.
3. Bandingkan dengan logika rekomendasi yang tampilkan semua kategori.

Pseudocode:

```
if preferensi == "A":  
    tampilkan_berita("A")
```

Python Script:

```
berita = ["politik A", "politik A", "politik B", "netral"]  
  
# Rekomendasi bias  
print("Rekomendasi Bias:", [b for b in berita if "A" in b])  
  
# Rekomendasi adil  
print("Rekomendasi Fair:", berita)
```

Jawaban: Rekomendasi bias hanya menampilkan 1 sisi, yang fair menampilkan semua.

Aktivitas 5: Simulasi Dataset Tidak Seimbang (*Unbalanced Data*)

Tujuan: Menunjukkan efek data timpang pada *machine learning*.

Langkah/Tahapan:

1. Buat dataset label “lulus” dan “tidak lulus”, tapi timpang (90% lulus, 10% tidak lulus).
2. Hitung prediksi sederhana: model selalu bilang “lulus”.
3. Diskusikan kenapa akurasi tinggi, tapi tetap unfair.

Pseudocode:

```
if mayoritas == "lulus":  
    prediksi = "lulus"
```

Python Script:

```
labels = ["lulus"]*9 + ["tidak"]*1  
prediksi = ["lulus"]*len(labels)  
akurasi = sum([1 for i in range(len(labels)) if  
labels[i]==prediksi[i]])/len(labels)  
print("Akurasi:", akurasi)
```

Output:

```
Akurasi: 0.9
```

Jawaban: Akurasi = 90%, tapi tidak adil karena selalu memprediksi “lulus” dan mengabaikan siswa yang gagal.

Kesimpulan Aktivitas

- **Bias** → dicek dengan perbandingan rata-rata.
- **Privasi** → dijaga dengan *masking*.
- **Fairness** → adil jika keputusan berdasarkan kemampuan, bukan faktor lain.
- **Filter Bubble** → bahaya jika rekomendasi cuma 1 arah.
- **Data timpang** → akurasi tinggi belum tentu adil.

Rangkuman

Dalam dunia teknologi, **etika data** itu sama pentingnya dengan kode program. Kalau kita hanya fokus bikin *machine learning* atau *neural network* yang canggih tanpa peduli etika, hasilnya bisa berbahaya. **Bias** terjadi saat data pincang, misalnya hanya berisi contoh dari satu kelompok. **Privasi** berarti menjaga rahasia data, jangan sampai bocor ke publik.

Fairness artinya keputusan AI harus adil untuk semua orang. Dengan alat *open source* seperti **Python**, **Pandas**, dan **Keras**, kita bisa belajar membuat AI yang bukan cuma pintar, tapi juga **aman dan adil**.

Bayangkan kamu punya kekuatan super berupa model AI. Kalau salah pakai, bisa menyakiti orang lain. Karena itu, kita harus **kritik hasil AI**, jangan langsung percaya. *Bias* bisa bikin keputusan salah, *privasi* yang bocor bisa bahaya, dan *fairness* yang hilang bisa bikin diskriminasi. Dengan latihan kecil, misalnya masking data di Python, atau analisis dataset di Orange Data Mining, kita bisa langsung merasakan dampaknya. **Etika bukan teori kaku, tapi panduan agar teknologi selalu membawa kebaikan.**

Akhirnya, ingat: **teknologi hanyalah alat**. Yang menentukan baik atau buruk adalah cara kita memakainya. Kalau AI dipakai dengan etika, maka ia bisa jadi sahabat manusia. Tapi kalau tanpa etika, AI bisa jadi “monster” yang tidak adil. Jadi, sebagai calon profesional SMK, tanggung jawab kita adalah membangun sistem AI yang **transparan, adil, dan menjaga rahasia**. Dengan memadukan *open source tools* dan pemahaman etika, kita bisa menciptakan masa depan yang lebih baik untuk semua orang.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. Data yang timpang dapat membuat model AI *bias*.
Jawaban: Benar
2. Menyembunyikan sebagian nomor telepon disebut *masking*.
Jawaban: Benar
3. *Fairness* berarti model boleh memilih berdasarkan gender.
Jawaban: Salah
4. Menjual data lokasi tanpa izin melanggar **privasi**.
Jawaban: Benar
5. Akurasi tinggi selalu berarti model sudah adil.
Jawaban: Salah

B. Pilihan Ganda (10 Soal)

1. Apa penyebab utama *bias* pada AI?
A. Algoritma mahal
B. Data timpang
C. Bahasa pemrograman
D. Komputer lambat
Jawaban: B
2. Contoh pelanggaran **privasi** adalah...
A. Menghapus *outlier*
B. Menjual data tanpa izin
C. Standarisasi fitur
D. *Cross-validation*
Jawaban: B
3. *Fairness* berarti...
A. Memilih yang paling populer
B. Adil untuk semua kelompok
C. Cepat walau salah
D. Hanya pakai data laki-laki
Jawaban: B
4. Alat *open source* untuk analisis tabel adalah...
A. Excel
B. Pandas
C. Power BI
D. Tableau
Jawaban: B
5. Teknik menyamarkan data sensitif disebut...
A. Hashing
B. *Masking*
C. Tuning
D. Caching
Jawaban: B
6. Contoh *pseudocode* yang tidak adil adalah...
A. `if nilai >= 80: lulus()`
B. `if gender == "P": tolak()`
C. `if skor > rata2: terima()`
D. `if valid: simpan()`
Jawaban: B
7. Model yang selalu menjawab "lulus" pada data 90% lulus disebut...
A. Adil
B. Overfit

C. *Baseline* bias mayoritas

D. *Underflow*

Jawaban: C

8. Mana yang paling tepat untuk melindungi email siswa saat dibagikan?

A. Tampilkan lengkap

B. Ganti jadi “unknown”

C. *Masking* sebagian

D. Hapus semua data

Jawaban: C

9. Library *open source* untuk *neural network* adalah...

A. Keras

B. Access

C. Visio

D. Dreamweaver

Jawaban: A

10. Praktik yang meningkatkan *fairness* adalah...

A. Memilih dari sekolah favorit

B. Memakai data beragam

C. Mengabaikan kelompok minoritas

D. Memperbesar kelas mayoritas

Jawaban: B

C. Isian Singkat (5 Soal)

1. Istilah untuk menjaga kerahasiaan data pribadi adalah _____.

Jawaban: privasi

2. Data yang tidak mewakili semua kelompok disebut data _____.

Jawaban: timpang

3. Keputusan AI yang adil untuk semua disebut _____.

Jawaban: fairness

4. Menyembunyikan sebagian data sensitif disebut _____.

Jawaban: masking

5. Dua library Python *open source* yang sering dipakai: _____ dan _____.

Jawaban: Pandas dan Keras

D. Soal Explorasi (3 Soal)

Tuliskan jawaban eksplorasi singkat (tiap soal 1 paragraf). Sertakan *pseudocode* atau kode Python pendek jika perlu.

Soal 1:

Evaluasi *Bias* pada Data Nilai Sederhana

Tugas: Buat 8 baris data nama, gender, nilai. Cek rata-rata per gender. Kesimpulan apakah ada *bias*.

Contoh jawaban (ringkas + kode):

```
import pandas as pd
df = pd.DataFrame({
    'nama': ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H'],
    'gender': ['L', 'P', 'L', 'P', 'L', 'P', 'L', 'P'],
    'nilai': [80, 85, 78, 90, 75, 88, 70, 92]
})
print(df.groupby('gender')['nilai'].mean())
# Jika beda jauh, ada indikasi bias pada data/nilai.
```

Output:

```
gender
L      75.75
P      88.75
Name: nilai, dtype: float64
```

Soal 2:

Masking Email & No HP sebelum Berbagi Dataset

Tugas: Buat DataFrame dengan kolom **nama**, **email**, **no_hp**. Terapkan *masking* agar aman dibagikan. Tulis alasan etisnya.

Contoh jawaban (ringkas + kode):

```
import pandas as pd
df = pd.DataFrame({
    'nama': ['Ani', 'Budi'],
    'email': ['ani123@school.id', 'budi456@school.id'],
    'no_hp': ['08123456789', '08561234567']
})
df['email'] = df['email'].str[:3] + '***'
df['no_hp'] = df['no_hp'].str[:4] + '****'
print(df)
# Alasan: menjaga privasi siswa sesuai etika dan regulasi.
```

Output:

```
   nama  email  no_hp
0  Ani  ani***  0812****
1  Budi bud***  0856****
```

Soal 3:

Pseudocode Seleksi Fair Ketua Proyek

Tugas: Tulis *pseudocode* seleksi ketua proyek yang hanya memakai **skor_kepemimpinan** dan **komitmen**, tanpa mempertimbangkan gender/asal sekolah. Tambahkan aturan lulus minimal.

Contoh jawaban (pseudocode):

```
for kandidat in daftar_kandidat:
    skor_total = 0.7*skor_kepemimpinan(kandidat) +
    0.3*komitmen(kandidat)
    if skor_total >= 80:
        kandidat.status = "LULUS"
    else:
        kandidat.status = "TIDAK LULUS"
# Catatan: tidak ada kondisi berdasarkan gender/asal sekolah →
fair.
```

Catatan Mini (opsional untuk siswa mencoba):

- Kamu bisa jalankan kode dengan **Python + Pandas** (semua *open source*).
- Untuk eksplor lebih jauh, coba latih *model sederhana* (mis. *logistic regression*) lalu cek metrik per kelompok untuk melihat *fairness*.

Tugas Proyek

Proyek 1: Cek Bias di Data Hobi

Soal:

Buat dataset kecil berisi nama siswa dan hobi. Tapi datanya timpang, misalnya 90% suka sepak bola. Analisis, apakah dataset itu bias?

Langkah:

1. Buat data fiktif dengan kolom **nama** dan **hobi**.
2. Hitung jumlah tiap hobi.
3. Jika satu hobi dominan → dataset bias.

Pseudocode:

```
buat_dataset(nama, hobi)
hitung_jumlah_per_hobi()
cek_dominasi()
```

Python Code:

```
import pandas as pd
```

```
data = {'nama': ['Budi', 'Siti', 'Andi', 'Rina', 'Joko'],
        'hobi': ['bola', 'bola', 'bola', 'bola', 'membaca']}
df = pd.DataFrame(data)

print(df['hobi'].value_counts())
```

Output:

```
hobi
bola      4
membaca    1
Name: count, dtype: int64
```

Jawaban: Dataset bias, karena mayoritas hanya hobi sepak bola.

Proyek 2: Menyembunyikan Email

Soal:

Buat program untuk melindungi privasi dengan cara menyembunyikan sebagian alamat email siswa.

Langkah:

1. Buat dataset berisi **nama** dan **email**.
2. Terapkan *masking*, hanya tampilkan 3 huruf awal.
3. Sisanya diganti dengan tanda *******.

Pseudocode:

```
for setiap email:
    tampilkan_awal(3) + "***"
```

Python Code:

```
import pandas as pd

data = {'nama': ['Budi', 'Siti'],
        'email': ['budi123@gmail.com', 'siti456@yahoo.com']}
df = pd.DataFrame(data)

df['email'] = df['email'].str[:3] + "***"
print(df)
```

Output:

```
   nama  email
0  Budi  bud***
1  Siti  sit***
```

Jawaban: Email jadi aman, misalnya **bud*****, **sit*****.

Proyek 3: Fairness di Pemilihan Ketua Kelas

Soal:

Simulasikan pemilihan ketua kelas. Pastikan AI tidak memilih berdasarkan gender, tapi hanya nilai kepemimpinan.

Langkah:

1. Buat data **nama**, **gender**, **nilai_kepemimpinan**.
2. Seleksi dengan syarat nilai ≥ 80 .
3. Abaikan gender saat pemilihan.

Pseudocode:

```
for setiap kandidat:
    if nilai >= 80:
        pilih()
```

Python Code:

```
data = [
    {"nama": "Ani", "gender": "P", "nilai": 85},
    {"nama": "Budi", "gender": "L", "nilai": 90},
    {"nama": "Citra", "gender": "P", "nilai": 70}
]

for d in data:
    if d["nilai"] >= 80:
        print(d["nama"], "LAYAK jadi ketua")
```

Output:

```
Ani LAYAK jadi ketua
Budi LAYAK jadi ketua
```

Jawaban: Ani dan Budi layak. Tidak boleh diskriminasi gender.

Proyek 4: Cek Gelembung Filter (*Filter Bubble*)

Soal:

Buat simulasi rekomendasi berita. Jika hanya tampilkan berita dari satu kategori, itu bias.

Langkah:

1. Buat list berita dengan kategori berbeda.
2. Buat rekomendasi bias (hanya 1 kategori).
3. Buat rekomendasi adil (tampilkan semua kategori).

Pseudocode:

```
if preferensi == "A":
    tampilkan_hanya("A")
```

```
else:
    tampilkan_semua()
```

Python Code:

```
berita = ["politik A","politik B","olahraga","teknologi"]

# Bias
print("Rekomendasi Bias:", [b for b in berita if "politik A" in b])

# Fair
print("Rekomendasi Fair:", berita)
```

Output:

```
Rekomendasi Bias: ['politik A']
Rekomendasi Fair: ['politik A', 'politik B', 'olahraga',
'olahraga', 'teknologi']
```

Jawaban: Fairness lebih baik karena semua kategori ditampilkan.

Proyek 5: Analisis Dataset Nilai dengan Pandas

Soal:

Buat dataset nilai ujian siswa. Cek apakah nilai rata-rata laki-laki dan perempuan seimbang.

Langkah:

1. Buat dataset **nama**, **gender**, **nilai**.
2. Kelompokkan data berdasarkan gender.
3. Hitung rata-rata nilai tiap gender.
4. Bandingkan hasilnya.

Pseudocode:

```
group_by_gender()
hitung_rata2()
bandingkan()
```

Python Code:

```
import pandas as pd

data = {'nama': ['Budi', 'Siti', 'Andi', 'Rina'],
        'gender': ['L', 'P', 'L', 'P'],
        'nilai': [80, 85, 75, 90]}
df = pd.DataFrame(data)

print(df.groupby('gender')['nilai'].mean())
```

Output:

```
gender
```

```
L      77.5
P      87.5
Name: nilai, dtype: float64
```

Jawaban: Jika rata-rata hampir sama → dataset fair. Jika beda jauh → dataset bias.

Kesimpulan Proyek:

- Kita belajar mencari **bias** di data sederhana.
- Kita belajar melindungi **privasi** dengan *masking*.
- Kita memastikan **fairness** dalam keputusan AI.
- Kita melihat bahaya *filter bubble* di rekomendasi.
- Kita menganalisis data nyata dengan **Pandas** agar lebih kritis.

BAB 8: Proyek – Model Prediksi Sederhana

Sekarang kita sudah sampai di bab paling seru: **Proyek Akhir!** Setelah kita belajar banyak teori, dari mengolah data pakai Pandas sampai bikin model cerdas, sekarang kita akan menggabungkan semuanya dalam sebuah **proyek nyata**. Kalian bakal jadi ilmuwan data beneran!

Tujuan Pembelajaran

- Mengaplikasikan semua ilmu yang sudah didapat (preprocessing, klasifikasi, regresi).
- Mengikuti alur kerja analisis data dari awal sampai akhir.
- Membuat model prediksi sederhana yang bisa diandalkan.
- Memvisualisasikan dan mengevaluasi hasil proyek.

Peta Konsep Sederhana

Bayangkan ini adalah proses membuat game.

- **Tulis Skrip (Mendapatkan Dataset)**: Kita cari data yang ingin kita olah.
- **Bersihkan Kode (Data Preprocessing)**: Kita rapikan data yang kotor pakai Pandas.
- **Rancang Level (Membuat Model ML)**: Kita buat model yang cerdas (misal: Regresi atau Klasifikasi).
- **Uji Coba Game (Evaluasi Model)**: Kita tes modelnya, seberapa jago dia?
- **Rilis Game (Visualisasi & Laporan)**: Kita pamerkan hasilnya dalam bentuk grafik yang keren.

Apersepsi: Kamu Mau Prediksi Penjualan Kebab di Sekolahmu!

Bayangin kamu punya warung kebab di sekolah. Kamu mencatat:

- Hari apa (Senin, Selasa, dst.)
- Cuaca (panas, hujan)
- Jumlah promosi yang kamu lakukan (ada atau tidak)
- **Jumlah penjualan** kebab hari itu.

Kamu mau bikin proyek buat memprediksi:

1. Berapa banyak kebab yang akan terjual besok, jika kamu bikin promosi di hari Rabu dan cuacanya panas?
2. Apakah promosi punya dampak besar pada penjualan?

Tuh, kan, keren banget kalau kita bisa tahu jawaban dari pertanyaan-pertanyaan ini!

Penjelasan Teori

Apa itu Prediksi?

- Prediksi = **menebak dengan alasan**.
- Kita belajar dari contoh (*data latih*), lalu menebak kasus baru.
- Di data, ada **masukan** (*feature*) dan **jawaban** (*label*).

Contoh kebab:

- *Feature*: **hari**, **cuaca**, **promosi**.
- *Label*: **jumlah_terjual** (berapa kebab laku).

Ingat: komputer bukan dukun. Ia cuma **belajar pola** dari data.

Dataset = Tabel Cerita

- Baris = kejadian/hari.
- Kolom = informasi tiap kejadian.
- Tipe data:
 - **Numerik**: angka (mis. **jumlah_terjual**).
 - **Kategorik**: teks/kelas (mis. **cuaca=panas/hujan**).

Istilah kunci

- *Feature*: kolom masukan.
- *Label/Target*: kolom yang ingin diprediksi.
- *Observasi/Record*: satu baris data.

Preprocessing (Beresin Data) — pakai Pandas

Tujuan: data rapi = model pintar.

Langkah simpel:

- **Cek kosong** (*missing values*).
 - Opsi: **dropna()** atau **fillna()** (isi rata-rata/median/"Tidak").
- **Cek ganda** (*duplicate*).
 - Opsi: **drop_duplicates()**.
- **Ubah teks** → **angka** untuk kolom kategorik.
 - Teknik: *one-hot encoding* (contoh: **cuaca_panas=1**, **cuaca_hujan=0**).
- **Skala** angka (opsional, penting untuk beberapa model).
 - Teknik: *standardization*.

Ibarat dapur: bahan kotor bikin masakan gagal. Data kotor bikin model bodoh.

Beda Tugas: *Regression* vs *Classification*

- **Regresi** (*regression*): prediksi **angka**.
 - Contoh: prediksi `jumlah_terjual`.
 - Model dasar: **Linear Regression**.
- **Klasifikasi** (*classification*): prediksi **kelas/label**.
 - Contoh: `target_prediksi` = **ya/tidak** (di atas 50 atau tidak).
 - Model dasar: **Decision Tree**.

Tips: mulai dari model sederhana. Kalau cukup akurat, lanjut!

Bagi Data: *Train–Test Split*

- Kita pisah data: **latih** dan **uji**.
- Biar adil, gunakan `random_state` (hasil bisa diulang).
- Untuk stabilitas, pakai *cross-validation* (cek model di beberapa lipatan).

Tujuan: pastikan model **bisa generalisasi**, bukan cuma **hafal**.

Pipeline (Rangkaian Otomatis di scikit-learn)

- *Pipeline* = jalur otomatis: *preprocessing* → model.
- Keuntungan: rapi, konsisten, anti lupa langkah.

Contoh komponen:

- **ColumnTransformer**: bedakan perlakuan kolom numerik vs kategorik.
- **OneHotEncoder**: ubah teks jadi angka.
- **StandardScaler**: skala angka (kalau perlu).
- **Estimator**: modelnya (mis. *Linear Regression*, *Decision Tree*).

Model Regresi: Linear Regression

Intuisi:

- Model menggambar **garis terbaik** untuk meminimalkan selisih prediksi vs nyata.
- Cocok saat hubungan kira-kira **linier**.

Metrik umum:

- **MAE** (*mean absolute error*): rata-rata selisih absolut.
- **MSE** (*mean squared error*): rata-rata selisih kuadrat.
- **R²**: seberapa banyak variasi data dijelaskan model (mendekati 1 itu bagus).

Model Klasifikasi: Decision Tree

Intuisi:

- Pohon bertanya **ya/tidak** berulang kali.
- Misal: “*promosi?*” → “*cuaca panas?*” → keputusan “**di atas 50**” atau “**tidak**”.

Risiko:

- **Overfitting**: pohon terlalu dalam, hafal data.
 - Obat: *pruning*, atur `max_depth`, `min_samples_split`.

Metrik umum:

- **Akurasi**: benar/semua.
- **Precision, Recall, F1** (penting saat kelas tidak seimbang).
- **Confusion Matrix**: tabel benar/salah per kelas.

Evaluasi yang Jujur

- Selalu laporkan metrik di **data uji**.
- Bandingkan dengan **baseline** sederhana (mis. tebak rata-rata).
- Cek *leakage*: jangan sampai kolom jawaban “bocor” ke fitur.

Jujur pada data = model yang bisa dipercaya.

Visualisasi (Biar WOW)

- **Matplotlib / Pandas Plot**: garis, batang, *scatter*.
- **Orange** (seret-lepas):
 - *File* → *Scatter Plot* (lihat pola regresi).
 - *Confusion Matrix* untuk klasifikasi.
 - *Test & Score* untuk banding model.

Neural Network — pakai Keras

Kapan dipakai?

- Fitur banyak, pola **kompleks**, data **lebih besar**.

Intuisi:

- *Layer* = langkah olah sinyal.
- *Activation* = bentuk sinyal (mis. *ReLU*, *sigmoid*).
- *Epoch* = berapa kali “mengelilingi” data latih.

Mulai dari *Decision Tree* dulu. Kalau butuh “otak” lebih, baru *neural network*.

Etika Data (Selalu!)

- Data harus **anonim** kalau sensitif.
- Minta izin jika data dari orang lain.
- Hindari bias (contoh: “hari hujan” ≠ pasti jualan buruk). Cek ulang.

Checklist Proyek (dari nol → rilis)

1. Kumpulkan *dataset* (kebab.csv).
2. *Preprocessing*: kosong, ganda, tipe data.
3. Pisah **train–test**.
4. Bangun *pipeline* (*encoding*, *scaling* bila perlu).
5. **Model 1**: *Linear Regression* → nilai MAE/MSE/R².
6. **Model 2**: *Decision Tree* → akurasi, *confusion matrix*.
7. Bandingkan, pilih, dan **jelaskan alasannya**.
8. Visualisasikan di **Orange/Matplotlib**.
9. Tulis **laporan singkat + grafik**.
10. Simpan kode di repo (portofolio!).

Catatan Open-Source (Wajib!)

- **Python + Pandas + scikit-learn** = inti analisis.
- **Orange** = visual *drag-and-drop*.
- **Keras** (di atas **TensorFlow**) = *neural network*.
- Bisa pakai **Ollama + OpenWebUI** untuk *chat* bantu koding (lokal).

Ringkas Super Singkat

- Prediksi = **belajar pola** dari data.
- Rapikan data dulu (*preprocessing*).
- **Regresi** untuk angka; **klasifikasi** untuk kelas.
- Ukur kinerja dengan **metrik** yang tepat.
- Visualisasikan, dokumentasikan, dan jelaskan **kenapa** modelmu dipilih.

Dengan alat *open-source*, kamu bisa bikin solusi nyata. Mulai kecil. Iterasi. Naik level.

Mini Contoh Alur Teknis

Fokus konsep, tapi kalau kamu mau *coba cepat*.

Regresi (jumlah_terjual)

- *Encoding* kategorik → *Linear Regression* → cek **MAE/MSE/R²**.

Klasifikasi (ya/tidak)

- Buat **target_prediksi** → **Decision Tree** → cek **akurasi** + *confusion matrix*.

Setelah itu, **plot** hasil dan **tulis** 3–5 kalimat kesimpulan yang jelas.

Kasus Ilustrasi

Kasus 1: Prediksi Harga Aksesoris Game (Regresi)

Cerita singkat: Kamu jual *sticker skin* dan *keycap* gaming. Mau nebak **harga jual** wajar dari *kondisi* dan *usia* barang.

- **Skema data (contoh kolom)**
 - **kondisi** (baru/bekas)
 - **usia_bulan** (angka)
 - **merek** (teks)
 - **harga_jual** (angka) ← **target**
- **Model:** *Linear Regression*.
- **Metrik:** *MAE, MSE, R²*.
- **Ilustrasi cepat:** Seperti menggambar **garis tren** terbaik di titik-titik data.

Langkah ringkas

- Rapiin data kosong/duplikat (*preprocessing*).
- *Encode* kolom teks (mis. *one-hot encoding* untuk **kondisi**, **merek**).
- *Train–test split*.
- Latih **LinearRegression**, uji, laporkan metrik.
- Visualisasi *scatter plot* (prediksi vs nyata) di **Orange**.

Pseudocode

```
# PSEUDOCODE: Prediksi Harga Aksesoris Game
load csv "aksesoris_game.csv" as df
drop duplicates, handle missing with fillna/median
encode categorical: kondisi, merek -> one-hot
X = df[fitur_kategorik_ter-encode + usia_bulan]
y = df[harga_jual]
split X, y into train, test (test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print MAE/MSE/R2 on (y_test, pred)
plot scatter (y_test vs pred) # matplotlib / Orange
save report + figure
```

Bonus tantangan: bandingkan dengan *Decision Tree Regressor*, pilih yang metriknya lebih bagus.

Kasus 2: Klasifikasi Makanan Terfavorit (Klasifikasi)

Cerita singkat: Kamu ingin menebak apakah temanmu suka **cilok** dari kebiasaan rasa.

- **Skema data**
 - **suka_manis** (0/1)
 - **suka_pedas** (0/1)
 - **sering_jajan** (0/1)
 - **suka_cilok** (ya/tidak) ← **target**
- **Model:** **DecisionTreeClassifier** atau *Logistic Regression*.
- **Metrik:** *accuracy, precision, recall, F1, confusion matrix*.
- **Ilustrasi cepat:** Pohon pertanyaan “**pedas?** → **sering jajan?** → hasil”.

Langkah ringkas

- Pastikan target seimbang (jika timpang, catat).
- *Train–test split*.
- Latih **Decision Tree**, atur **max_depth** jika *overfitting*.
- Buat **confusion matrix** di **Orange**.

Pseudocode

```
# PSEUDOCODE: Klasifikasi Suka Cilok
load csv "preferensi_makanan.csv" as df
binary-ize kolom: suka_manis, suka_pedas, sering_jajan
label = map ya->1, tidak->0
X = df[[suka_manis, suka_pedas, sering_jajan]]
y = df[suka_cilok_binary]
split X, y (test_size=0.2, random_state=42, stratify=y)
clf = DecisionTreeClassifier(max_depth=4, random_state=42)
clf.fit(X_train, y_train)
pred = clf.predict(X_test)
print accuracy/precision/recall/F1
plot confusion_matrix
export tree (opsional) # lihat aturan keputusan
```

Bonus tantangan: bandingkan *Decision Tree* vs *Logistic Regression*. Pilih yang F1 lebih tinggi.

Kasus 3: Prediksi Rata-rata Nilai Sekolah (Regresi)

Cerita singkat: Kamu mau memprediksi **nilai rata-rata** dari *jam belajar* dan *kehadiran*.

- **Skema data**
 - **jam_belajar_per_minggu** (angka)
 - **persen_kehadiran** (0–100)
 - **ikut_bimbel** (0/1)

- `rata2_nilai` (0–100) ← **target**
- **Model:** *Linear Regression* (awal), lalu boleh coba *Ridge/Lasso*.
- **Metrik:** *MAE*, *R²*.
- **Ilustrasi cepat:** Semakin rajin, **garis tren** cenderung naik.

Langkah ringkas

- Cek *outlier* (nilai ekstrem), dokumentasikan.
- *Train–test split*, latih model, uji metrik.
- Plot 2D/3D *scatter* di **Orange** (fitur vs nilai).

Pseudocode

```
# PSEUDOCODE: Prediksi Nilai Rata-rata
load csv "nilai_siswa.csv" as df
handle missing + clip outlier (opsional)
encode ikut_bimbel (0/1)
X = df[[jam_belajar_per_minggu, persen_kehadiran, ikut_bimbel]]
y = df[rata2_nilai]
split -> train/test
model = LinearRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print MAE, R2
plot residuals (y_test - pred) # cek pola error
save model + laporan singkat
```

Bonus tantangan: uji *polynomial features* derajat 2 bila hubungan tidak linier.

Kasus 4: Prediksi Waktu Sampai Tujuan (Regresi)

Cerita singkat: Kamu ingin menebak **lama perjalanan** ke sekolah dari *jarak*, *kecepatan rata-rata*, dan *kemacetan*.

- **Skema data**
 - `jarak_km` (angka)
 - `kecepatan_rata2` (km/jam)
 - `tingkat_macet` (rendah/sedang/tinggi)
 - `waktu_tempuh_menit` (angka) ← **target**
- **Model:** *Linear Regression* atau *RandomForestRegressor* (jika pola rumit).
- **Metrik:** *MAE*, *RMSE*.
- **Ilustrasi cepat:** Macet tinggi → waktu tempuh **naik** tajam.

Langkah ringkas

- Encode `tingkat_macet`.
- *Train–test split*.
- Latih model dasar; bandingkan dengan *Random Forest*.
- Plot **y_aktual vs y_pred**.

Pseudocode

```
# PSEUDOCODE: Prediksi Waktu Tempuh
load csv "perjalanan_sekolah.csv" as df
one_hot encode: tingkat_macet
X = df[[jarak_km, kecepatan_rata2] + onehot(tingkat_macet)]
y = df[waktu_tempuh_menit]
split -> train/test
reg = LinearRegression()
reg.fit(X_train, y_train)
pred = reg.predict(X_test)
print MAE, RMSE = sqrt(MSE)
plot y_test vs pred # garis diagonal = ideal
```

Bonus tantangan: coba **RandomForestRegressor** dan bandingkan *RMSE*.

Kasus 5: Prediksi Popularitas Konten Medsos (Klasifikasi)

Cerita singkat: Kamu ingin menebak apakah konten bakal **viral** atau **biasa saja**.

- **Skema data**
 - **jumlah_like** (angka)
 - **jumlah_share** (angka)
 - **durasi_video_detik** (angka)
 - **jam_upload** (pagi/siang/malam)
 - **viral** (ya/tidak) ← **target**
- **Model:** **Logistic Regression** atau **DecisionTreeClassifier**.
- **Metrik:** *accuracy + precision/recall/F1, ROC-AUC*.
- **Ilustrasi cepat:** Banyak *share* + waktu upload pas → peluang **viral** naik.

Langkah ringkas

- *Encode jam_upload.*
- *Train-test split* (pakai **stratify=y**).
- Latih model, lihat **probabilitas** (batas 0.5 bisa diatur).
- Visualisasikan **confusion matrix** di **Orange**.

Pseudocode

```
# PSEUDOCODE: Prediksi Viral Konten
load csv "konten_medsos.csv" as df
one_hot encode: jam_upload
label viral: ya->1, tidak->0
X = df[[jumlah_like, jumlah_share, durasi_video_detik] +
onehot(jam_upload)]
y = df[viral_bin]
split X, y (stratify=y, test_size=0.25, random_state=42)
clf = LogisticRegression(max_iter=1000)
clf.fit(X_train, y_train)
proba = clf.predict_proba(X_test)[: ,1]
```

```
pred = (proba >= 0.5).astype(int)
print accuracy, precision, recall, F1
plot ROC curve + AUC # opsional
```

Bonus tantangan: atur *threshold* (mis. 0.6) untuk kurangi *false positive*.

Catatan Praktis (buat semua kasus)

- **Alur tetap:** *preprocessing* → *train-test split* → *latih* → *evaluasi* → *visualisasi* → *laporan*.
- **Orange** itu *drag-and-drop*: **File** → **Select Columns** → **(Learner)** → **Test & Score** → **Confusion Matrix/Scatter**.
- **Keras** (*neural network*) itu **opsional lanjutan**. Pakai kalau data besar/kompleks:
 - Bentuk *MLP* sederhana: *Dense(64)* → *ReLU* → *Dense(1/softmax)*.
 - **Tetap** bandingkan dengan model sederhana sebagai *baseline*.

Soal Latihan & Pembahasan

Soal 1: *Preprocessing* Kebab (Pandas dasar)

Kasus: Kamu punya *kebab.csv* dengan kolom: *hari*, *cuaca*, *promosi*, *jumlah_terjual*. Ada nilai kosong dan *duplikat*. Buat kolom baru *target_prediksi* berisi "**ya**" jika *jumlah_terjual* > 50, selain itu "**tidak**". Lalu ubah kolom teks menjadi angka.

Jawaban singkat:

- Bersihkan kosong & duplikat.
- Buat *target_prediksi*.
- Lakukan *one-hot encoding* untuk *hari*, *cuaca*, *promosi*.

Pseudocode

```
# PSEUDOCODE Soal 1
load csv "kebab.csv" as df
drop duplicate rows
fill missing:
  for numeric -> fill with median
  for text    -> fill with "TidakAda"
df["target_prediksi"] = apply(df["jumlah_terjual"], x -> "ya" if
x > 50 else "tidak")
encode = one_hot(["hari","cuaca","promosi"]) # contoh:
cuaca_panas=1, cuaca_hujan=0
df_encoded = apply_one_hot(df, encode)
save df_encoded to "kebab_clean.csv"
```

Pembahasan:

- *Preprocessing* itu wajib. Data rapi → model jago.
- *One-hot encoding* mengubah teks jadi angka. Komputer suka angka.
- Ambang 50 sederhana dan jelas. Ini membuat **kelas** untuk tugas **klasifikasi**.

Soal 2: Regresi: Prediksi **jumlah_terjual** (Linear Regression + *Pipeline*)

Kasus: Prediksi angka **jumlah_terjual** dari fitur **hari**, **cuaca**, **promosi**. Pakai **Linear Regression**. Laporkan **MAE** dan **R²**. Gunakan *pipeline* agar langkah rapi.

Jawaban singkat:

- Pisah *train-test*.
- *Pipeline: one-hot* → **LinearRegression**.
- Hitung **MAE** dan **R²** di *test set*.

Pseudocode

```
# PSEUDOCODE Soal 2
df = load "kebab_clean.csv"
X = df[["hari","cuaca","promosi"]]      # kategorik
y = df["jumlah_terjual"]                # numerik (target)

split X,y into X_train,X_test,y_train,y_test (test_size=0.2,
random_state=42)

preprocess = OneHotEncoder(handle_unknown="ignore")
model = LinearRegression()

pipeline = Pipeline(steps=[
    ("encode", preprocess),
    ("reg", model)
])

pipeline.fit(X_train, y_train)
pred = pipeline.predict(X_test)

MAE = mean_absolute_error(y_test, pred)
R2 = r2_score(y_test, pred)
print MAE, R2
```

Pembahasan:

- **MAE** kecil = rata-rata selisih prediksi kecil.
- **R²** mendekati 1 = model menjelaskan variasi data dengan baik.
- *Pipeline* menjaga proses konsisten saat *fit* dan *predict*. Praktis!

Soal 3: Klasifikasi: Prediksi **target_prediksi** (Decision Tree)

Kasus: Prediksi **target_prediksi** (ya/tidak) dari **hari**, **cuaca**, **promosi**. Pakai **DecisionTreeClassifier**. Gunakan **stratify** saat *split*. Tampilkan **accuracy** dan **confusion matrix**.

Jawaban singkat:

- *Train-test split* dengan **stratify=y**.
- **Decision Tree** kedalaman wajar (**max_depth**).
- Laporan **accuracy + confusion matrix**.

Pseudocode

```
# 1. Baca data
df = load_csv("kebab_clean.csv")

# 2. Pilih fitur (input) dan target (label)
X = df[["hari", "cuaca", "promosi"]] # fitur kategorik
y = map(df["target_prediksi"], "ya"->1, "tidak"->0) # label biner

# 3. Bagi data train-test, pakai stratify biar proporsi kelas seimbang
X_train, X_test, y_train, y_test = split(X, y,
                                          test_size=0.25,
                                          random_state=42,
                                          stratify=y)

# 4. Buat pipeline: encoding + Decision Tree
encoder = OneHotEncoder(handle_unknown="ignore")
clf = DecisionTreeClassifier(max_depth=5, random_state=42)

pipeline = Pipeline([
    ("encode", encoder),
    ("model", clf)
])

# 5. Latih model dengan data training
pipeline.fit(X_train, y_train)

# 6. Prediksi di data test
y_pred = pipeline.predict(X_test)

# 7. Hitung accuracy
acc = accuracy_score(y_test, y_pred)

# 8. Buat confusion matrix
cm = confusion_matrix(y_test, y_pred)

# 9. Tampilkan hasil
```

```
print("Accuracy =", acc)
print("Confusion Matrix =", cm)
```

Penjelasan singkat:

- **Stratify=y** → menjaga distribusi kelas "ya"/"tidak" supaya sama di train dan test.
- **Decision Tree** → tiap simpul adalah pertanyaan *ya/tidak*, gampang dibaca.
- **Confusion Matrix** → bukan hanya angka *accuracy*, tapi juga nunjukin prediksi salah/benarnya di tiap kelas.

Soal 4: Threshold dan Precision/Recall (Logistic Regression)

Kasus: Kamu latih **Logistic Regression** untuk **target_prediksi**. Default *threshold* = **0.5**. Coba ubah *threshold* ke **0.6**. Apa dampaknya ke **precision** dan **recall** kelas "ya"?

Jawaban singkat:

- *Threshold* naik → model lebih “ketat” menyebut **ya**.
- **Precision(ya)** cenderung **naik** (lebih sedikit *false positive*).
- **Recall(ya)** cenderung **turun** (lebih banyak *false negative*).

Pseudocode

```
# PSEUDOCODE Soal 4
df = load "kebab_clean.csv"
X = df[["hari", "cuaca", "promosi"]]
y = map(df["target_prediksi"], "ya"->1, "tidak"->0)

split X,y (stratify=y)

pipe = Pipeline([
    ("encode", OneHotEncoder(handle_unknown="ignore")),
    ("clf", LogisticRegression(max_iter=1000))
])

pipe.fit(X_train, y_train)
proba = pipe.predict_proba(X_test)[:,-1]

pred_05 = (proba >= 0.5) -> 1 else 0
pred_06 = (proba >= 0.6) -> 1 else 0

print precision_recall_f1(y_test, pred_05, positive_class=1)
print precision_recall_f1(y_test, pred_06, positive_class=1)
```

Pembahasan:

- *Threshold* itu “batas yakin”. Lebih tinggi = lebih selektif.

- Pilih *threshold* sesuai tujuan: mau **minim salah positif** atau **minim salah negatif**.
- Ini penting saat kasus nyata, misal *spam filter* atau *fraud detection*.

Soal 5: *Neural Network* sederhana (Keras)

Kasus: Coba *neural network* (Keras) untuk memprediksi *jumlah_terjual*.
Bandingkan *MAE* dengan **Linear Regression**. Gunakan *early stopping* agar tidak *overfitting*.

Jawaban singkat:

- Buat *MLP* kecil: *Dense(64, ReLU) → Dense(32, ReLU) → Dense(1)*.
- Pakai *standardization* fitur.
- Aktifkan **EarlyStopping**. Bandingkan **MAE** di *test set*.

Pseudocode

```
# PSEUDOCODE Soal 5
df = load "kebab_clean.csv"
X_cat = df[["hari", "cuaca", "promosi"]]
X_num = df[["jumlah_promosi_mingguan"]] # contoh fitur numerik
tambahan (opsional)
y = df["jumlah_terjual"]

split X_cat, X_num, y (test_size=0.2, random_state=42)

pre_cat = OneHotEncoder(handle_unknown="ignore")
pre_num = StandardScaler()

X_train_p = concat(pre_cat.fit_transform(X_cat_train),
                    pre_num.fit_transform(X_num_train))
X_test_p   = concat(pre_cat.transform(X_cat_test),
                    pre_num.transform(X_num_test))

# KERAS MODEL (MLP)
model = Sequential()
model.add(Dense(64, activation="relu",
                input_dim=X_train_p.shape[1]))
model.add(Dense(32, activation="relu"))
model.add(Dense(1)) # regresi

model.compile(optimizer="adam", loss="mae") # pakai MAE biar
mudah dibaca
early = EarlyStopping(monitor="val_loss", patience=5,
                      restore_best_weights=True)

model.fit(X_train_p, y_train, epochs=100, batch_size=32,
          validation_split=0.2, callbacks=[early], verbose=0)

pred = model.predict(X_test_p)
MAE_nn = mean_absolute_error(y_test, pred)
```

```
print MAE_nn
```

```
# Bandingkan dengan LinearRegression (dari Soal 2)
```

Pembahasan:

- *Neural network* kuat untuk pola kompleks. Tapi butuh data lebih banyak.
- **EarlyStopping** berhenti otomatis saat *val_loss* tak membaik. Anti *overfitting*.
- Jika Linear Regression sudah bagus, NN belum tentu menang. Pilih yang sederhana dan cukup.

Ringkas Tips Praktik (untuk semua soal)

- **Selalu** pisah *train–test*.
- Simpan metrik dan grafik. Ini bukti kerja kalian.
- Pakai **Orange** untuk cek cepat: *File* → (*Learner*) → *Test & Score* → *Confusion Matrix/Scatter Plot*.
- Dokumentasikan: tujuan, data, model, metrik, kesimpulan, saran perbaikan.
- *Iterasi pelan tapi pasti*. Mulai simpel. Upgrade kalau perlu.

Fun Facts!

- Proyek ini adalah versi mini dari pekerjaan seorang **Data Scientist**.
- Kalian bisa pakai **Ollama** dan **OpenWebUI** untuk bantu kalian bikin kode atau menjelaskan hasil.
- **RAG lokal** bisa kalian pakai buat *chatbot* yang bisa menjawab pertanyaan tentang proyek kalian.

Tips Belajar atau Menghafal Cepat

- **Kerjakan Bertahap**: Jangan langsung bikin semua model. Mulai dari satu persatu.
- **Banyak Bertanya**: Jangan ragu untuk bertanya pada guru atau teman jika ada yang bingung.
- **Simpan Kodenya**: Simpan semua kode kalian di satu tempat. Ini bakal jadi portofolio kalian.

Aktivitas Siswa

Aktivitas 1: *Preprocessing* Data Kebab dengan Pandas

Tujuan: membersihkan data kebab sebelum dipakai.

Langkah/Tahap:

1. Buat file `kebab.csv` dengan kolom: `hari`, `cuaca`, `promosi`, `jumlah_terjual`.
2. Isi beberapa data dengan sengaja ada yang kosong.
3. Impor ke Pandas.
4. Bersihkan nilai kosong dan data ganda.

Contoh File: `kebab.csv`

```
hari,cuaca,promosi,jumlah_terjual
Senin,panas,ya,60
Selasa,hujan,tidak,35
Rabu,panas,ya,80
Kamis,berawan,tidak,45
Jumat,panas,ya,75
Sabtu,hujan,ya,50
Minggu,panas,tidak,40
Senin,berawan,ya,65
Selasa,panas,tidak,55
Rabu,hujan,ya,30
```

Pseudocode:

```
df = load_csv("kebab.csv")
drop_duplicates(df)
fill_missing(df, method="median_or_default")
```

Python Script:

```
import pandas as pd

# 1. Baca file
df = pd.read_csv("kebab.csv")

# 2. Hapus duplikat
df = df.drop_duplicates()

# 3. Isi data kosong
df = df.fillna({
    "hari": "TidakAda",
    "cuaca": "tidak_diketahui",
    "promosi": "tidak",
    "jumlah_terjual": df["jumlah_terjual"].median()
})
```

```
print(df.head())
```

Aktivitas 2: Prediksi Jumlah Terjual dengan Regresi

Tujuan: memprediksi *angka jumlah_terjual*.

Langkah/Tahap:

1. Pilih fitur: *hari, cuaca, promosi*.
2. Target: *jumlah_terjual*.
3. Split data train-test.
4. Latih dengan *Linear Regression*.
5. Hitung MAE dan R^2 .

Pseudocode:

```
X = fitur(["hari", "cuaca", "promosi"])
y = target("jumlah_terjual")
X_train, X_test, y_train, y_test = split(X, y)
model = LinearRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)
evaluate(pred, y_test, metrics=["MAE", "R2"])
```

Python Script:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline

X = df[["hari", "cuaca", "promosi"]]
y = df["jumlah_terjual"]

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, random_state=42)

pipe = Pipeline([
    ("encode", OneHotEncoder(handle_unknown="ignore")),
    ("reg", LinearRegression())
])

pipe.fit(X_train, y_train)
pred = pipe.predict(X_test)

print("MAE:", mean_absolute_error(y_test, pred))
print("R2 :", r2_score(y_test, pred))
```

Aktivitas 3: Klasifikasi Target (Decision Tree)

Tujuan: memprediksi apakah penjualan **di atas 50** atau tidak.

Langkah/Tahap:

1. Buat kolom baru **target_prediksi**.
2. Kategorikan: jika $>50 \rightarrow$ "ya", selain itu "tidak".
3. Latih dengan **Decision Tree**.
4. Tampilkan *accuracy* dan *confusion matrix*.

Pseudocode:

```
df["target_prediksi"] = apply(jumlah_terjual, x>50 ?
"ya":"tidak")
X = fitur(["hari","cuaca","promosi"])
y = target("target_prediksi")
split stratify=y
tree = DecisionTree(max_depth=5)
tree.fit(X_train,y_train)
y_pred = tree.predict(X_test)
print accuracy(y_test, y_pred)
print confusion_matrix(y_test,y_pred)
```

Python Script:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

df["target_prediksi"] = df["jumlah_terjual"].apply(lambda x:
"ya" if x > 50 else "tidak")

X = df[["hari","cuaca","promosi"]]
y = df["target_prediksi"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)

pipe = Pipeline([
    ("encode", OneHotEncoder(handle_unknown="ignore")),
    ("clf", DecisionTreeClassifier(max_depth=5,
    random_state=42))
])

pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Aktivitas 4: Evaluasi Model dengan Orange

Tujuan: melihat hasil prediksi dengan visualisasi.

Langkah/Tahap:

1. Simpan hasil prediksi ke file CSV.
2. Buka Orange.
3. Seret widget **File** → pilih CSV.
4. Hubungkan dengan **Confusion Matrix** untuk klasifikasi.
5. Hubungkan dengan **Scatter Plot** untuk regresi.

Pseudocode:

```
save_csv("hasil_prediksi.csv", columns=[y_test, y_pred])
open_orange()
drag widget File -> pilih "hasil_prediksi.csv"
drag widget Confusion Matrix untuk klasifikasi
drag widget Scatter Plot untuk regresi
lihat hasil visualisasi
```

Python Script (simpan CSV untuk Orange):

```
out_df = X_test.copy()
out_df["actual"] = y_test
out_df["predicted"] = y_pred
out_df.to_csv("hasil_prediksi.csv", index=False)
```

Aktivitas 5: Neural Network Mini dengan Keras

Tujuan: mencoba *neural network* sederhana.

Langkah/Tahap:

1. Ambil fitur numerik (mis. ubah one-hot jadi array).
2. Buat model Keras sederhana.
3. Latih model dengan data train.
4. Evaluasi dengan MAE.

Pseudocode:

```
X_train_p, X_test_p = one_hot_encode(X_train, X_test)
y_train, y_test = as_numeric(y_train, y_test)
NN = Sequential()
NN.add(Dense(64, activation="relu"))
NN.add(Dense(32, activation="relu"))
NN.add(Dense(1)) # output regresi
NN.compile(optimizer="adam", loss="mae")
NN.fit(X_train_p, y_train, validation_split=0.2, epochs=50)
pred = NN.predict(X_test_p)
MAE = mean_absolute_error(y_test, pred)
```

```
print MAE
```

Python Script:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.preprocessing import OneHotEncoder
from sklearn.metrics import mean_absolute_error

enc = OneHotEncoder(handle_unknown="ignore")
X_p = enc.fit_transform(X).toarray()

X_train, X_test, y_train, y_test = train_test_split(X_p, y,
test_size=0.2, random_state=42)

model = Sequential()
model.add(Dense(64, activation="relu",
input_dim=X_train.shape[1]))
model.add(Dense(32, activation="relu"))
model.add(Dense(1)) # regresi

model.compile(optimizer="adam", loss="mae")
model.fit(X_train, y_train, validation_split=0.2, epochs=50,
verbose=0)

pred = model.predict(X_test)
print("MAE NN:", mean_absolute_error(y_test, pred))
```

Kesimpulan

- Aktivitas ini bikin kamu bisa **praktek langsung** jadi *data scientist*.
- Mulai dari **bersihkan data**, **latih model regresi**, **latih model klasifikasi**, **evaluasi visual di Orange**, sampai **coba neural network sederhana**.
- Semua pakai *open source tools*: Python, Pandas, scikit-learn, Orange, Keras.

Jadi, kalian bukan cuma belajar teori. Tapi juga benar-benar **membuat mesin cerdas** dengan data kalian sendiri.

Rangkuman

Kalian baru saja menyelesaikan proyek lengkap ala ilmuwan data. Mulai dari **mengumpulkan data**, **merapikan data**, sampai **membuat model** dan **menguji hasilnya**. Ingat alurnya: *dataset* → *preprocessing* → *train-test split* → *modeling* → *evaluasi* → *visualisasi & laporan*. **Kunci suksesnya: data bersih dulu!** Pakai alat *open-source* seperti **Python**, **Pandas**, **scikit-learn**, **Orange**, dan kalau perlu **Keras** untuk *neural network*. Semua gratis, kuat, dan cocok untuk belajar di SMK.

Kita belajar beda tugas **Regresi vs Klasifikasi**. Regresi untuk **angka** (*regression*), misalnya jumlah kebab terjual. Klasifikasi untuk **label** (*classification*), misalnya “ya/tidak” lewat **Decision Tree**. Selalu ukur kinerja model dengan **metrik yang tepat**: *MAE*, *MSE*, *R²* untuk regresi; *accuracy*, *precision*, *recall*, *F1*, dan *confusion matrix* untuk klasifikasi. **Evaluasi yang jujur** itu penting: pakai *train–test split*, bandingkan dengan *baseline*, dan hindari *data leakage*. Kalau butuh otomatis dan rapi, gunakan **pipeline** agar langkah *encoding* sampai prediksi berjalan konsisten.

Proyek ini menunjukkan bahwa **machine learning itu bisa dipakai untuk masalah nyata** di sekolah dan sekitar kalian. Mulai sederhana, uji, lalu tingkatkan. **Dokumentasikan** semua: data, kode, metrik, grafik, dan **alasan memilih model**—ini bisa jadi **portofolio** kalian. Jika pola makin kompleks, kalian bisa coba *neural network* di **Keras**, tapi tetap bandingkan dengan model sederhana. **Terus eksplorasi**, banyak bertanya, dan jangan takut gagal. Dengan alat *open-source* dan sikap penasaran, kalian sudah **selangkah di depan!**

Latihan Soal

A. Benar atau Salah (5 Soal)

1. Python adalah bahasa *open-source* yang bisa digunakan untuk Machine Learning.

Jawaban: Benar

2. Pandas digunakan untuk membuat grafik visualisasi data.

Jawaban: Salah

3. Regresi Linier digunakan untuk memprediksi nilai berbentuk angka.

Jawaban: Benar

4. Decision Tree hanya bisa digunakan untuk klasifikasi biner (ya/tidak).

Jawaban: Salah

5. Confusion Matrix bisa menunjukkan jumlah prediksi benar dan salah dari model klasifikasi.

Jawaban: Benar

B. Pilihan Ganda (10 Soal)

6. Fungsi utama Pandas dalam proyek Machine Learning adalah...

- a. Membuat game
- b. Mengolah dan membersihkan data
- c. Membuat robot
- d. Menggambar animasi

Jawaban: b

7. Library yang digunakan untuk membuat model Machine Learning di Python adalah...
- a. scikit-learn
 - b. Matplotlib
 - c. PowerPoint
 - d. Word

Jawaban: a

8. Apa keluaran dari model regresi?
- a. Label kategori
 - b. Angka prediksi
 - c. Gambar grafik
 - d. Tabel kosong

Jawaban: b

9. Dalam klasifikasi, Decision Tree membuat keputusan berdasarkan...
- a. Simpul dan cabang
 - b. Warna grafik
 - c. Jumlah data kosong
 - d. Ukuran file CSV

Jawaban: a

10. Perintah untuk membaca file CSV dengan Pandas adalah...

- a. `df = pd.read_excel("file.csv")`
- b. `df = read.csv("file.csv")`
- c. `df = pd.read_csv("file.csv")`
- d. `df = csv.load("file.csv")`

Jawaban: c

11. Jika kita ingin memprediksi apakah nilai ujian siswa **lulus/tidak lulus**, model yang tepat adalah...
- a. Regresi Linier
 - b. Decision Tree Klasifikasi
 - c. Clustering
 - d. Random Guess

Jawaban: b

12. Berikut yang bukan contoh *feature* dalam dataset penjualan kebab adalah...

- a. Hari
- b. Cuaca
- c. Jumlah penjualan
- d. Promosi

Jawaban: c

13. Output evaluasi klasifikasi yang menunjukkan *precision*, *recall*, *f1-score* disebut...

- a. Confusion Matrix
- b. Classification Report
- c. Scatter Plot

d. Line Chart

Jawaban: b

14. Jika model selalu menebak sama dengan **data kemarin**, maka model itu disebut...

- a. Naive Model
- b. Decision Tree
- c. Regresi Linier
- d. Neural Network

Jawaban: a

15. Library open-source yang dapat digunakan untuk visualisasi *drag and drop* tanpa coding adalah...

- a. Orange
- b. Excel
- c. Photoshop
- d. Power BI

Jawaban: a

C. Isian Singkat (5 Soal)

16. Isi perintah Python untuk membaca file CSV bernama `kebab.csv`.

Jawaban: `df = pd.read_csv("kebab.csv")`

17. Apa nama metode di scikit-learn untuk membagi data train dan test?

Jawaban: `train_test_split`

18. Tambahkan kolom baru `target_prediksi = "ya"` jika `jumlah_terjual > 50`, selain itu "tidak".

Jawaban: `df["target_prediksi"] =
df["jumlah_terjual"].apply(lambda x: "ya" if x > 50 else
"tidak")`

19. Sebutkan satu metrik yang bisa digunakan untuk mengukur performa regresi.

Jawaban: Mean Squared Error (MSE)

20. Jika kita ingin memprediksi jumlah kebab besok berdasarkan data hari ini, jenis model yang digunakan adalah...

Jawaban: Regresi Linier

D. Soal Eksplorasi (3 Soal)

Soal 1:

Eksplorasi Regresi

Bayangkan kamu punya data fiktif `jam_belajar` dan `nilai_ujian`. Buatlah pseudocode sederhana untuk melatih model Regresi Linier agar bisa memprediksi nilai ujian berdasarkan jam belajar.

Contoh Jawaban:

```
load data (jam_belajar, nilai_ujian)
X = jam_belajar
y = nilai_ujian
split train-test
model = LinearRegression
fit model
prediksi = model.predict(X_test)
hitung MSE
```

Soal 2:

Eksplorasi Klasifikasi

Kamu ingin memprediksi apakah temanmu suka `cilok` berdasarkan 2 data: `suka_manis` dan `suka_pedas`. Buatlah pseudocode dengan Decision Tree.

Contoh Jawaban:

```
load data (suka_manis, suka_pedas, suka_cilok)
X = [suka_manis, suka_pedas]
y = suka_cilok
split train-test stratify=y
model = DecisionTreeClassifier
fit model
prediksi = model.predict(X_test)
tampilkan accuracy dan confusion_matrix
```

Soal 3:

Eksplorasi Visualisasi

Buatlah langkah-langkah untuk memvisualisasikan data penjualan kebab dengan *bar chart* menggunakan Pandas/Matplotlib.

Contoh Jawaban:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("kebab.csv")
df.groupby("hari")["jumlah_terjual"].mean().plot(kind="bar")
plt.ylabel("Rata-rata Terjual")
```

```
plt.title("Penjualan Kebab per Hari")
plt.show()
```

👉 Latihan ini bisa langsung dicoba oleh siswa SMK dengan data fiktif sederhana. Ada yang berbentuk cepat (benar-salah), ada pilihan ganda, isian kode, hingga eksplorasi lebih kreatif dengan pseudocode dan Python.

Tugas Proyek

Proyek 1: Prediksi Sampah Kantin (Regresi + *Cross-Validation*)

Tujuan: tebak **berat sampah harian (kg)** dari **hari**, **menu_spesial**, **jumlah_pengunjung**.

Kenapa seru: mirip dunia nyata. Ada fitur *kategorik* dan *numerik*.

Output: nilai **MAE** kecil = model bagus.

Langkah singkat

- Kumpulkan data 2–4 minggu.
- Rapikan data kosong.
- Buat *pipeline*: *one-hot encoding* → **LinearRegression**.
- Pakai *cross-validation* 5 lipatan.
- Laporkan **MAE rata-rata + deviasi**.

Pseudocode

```
load_csv("sampah_kantin.csv") -> df
X = ["hari", "menu_spesial", "jumlah_pengunjung"]
y = "berat_sampah_kg"
pipeline = OneHotEncoder -> LinearRegression
scores = cross_val_score(pipeline, X, y, cv=5, scoring=MAE)
print mean(scores), std(scores)
```

Contoh file: *sampah_kantin.csv*

```
hari,menu_spesial,jumlah_pengunjung,berat_sampah_kg
Senin,Nasi_Goreng,150,5.2
Selasa,Soto_Ayam,120,4.8
Rabu,Gado_Gado,180,6.1
Kamis,Nasi_Campur,200,7.3
Jumat,Bakso,170,6.5
Senin,Nasi_Goreng,160,5.5
Selasa,Soto_Ayam,130,4.9
Rabu,Gado_Gado,190,6.4
Kamis,Nasi_Campur,210,7.5
Jumat,Bakso,165,6.2
```

Contoh Python

```
import pandas as pd
from sklearn.model_selection import cross_val_score
```

```

from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.metrics import make_scorer, mean_absolute_error

df = pd.read_csv("sampah_kantin.csv")
X = df[["hari", "menu_spesial", "jumlah_pengunjung"]]
y = df["berat_sampah_kg"]

pre = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"),
    ["hari", "menu_spesial"])
], remainder="passthrough")

pipe = Pipeline([("pre", pre), ("reg", LinearRegression())])
mae = cross_val_score(pipe, X, y, cv=5,
                      scoring=make_scorer(mean_absolute_error,
                      greater_is_better=False))
print("MAE rata-rata:", -mae.mean(), "±", mae.std())

```

Kunci jawaban (inti): MAE rata-rata yang rendah dan stabil ($\pm$ kecil) berarti model cukup andal.

Proyek 2: Telat Datang? (Klasifikasi Imbang + *Threshold*)

Tujuan: klasifikasi **telat_15menit** = *ya/tidak* dari **jarak_km**, **cuaca**, **bangun_pagi** (0/1), **naik_kendaraan** (0/1).

Kenapa seru: kelas bisa tidak seimbang. Belajar *class_weight* dan *threshold*.

Output: precision/recall kelas *ya*.

Langkah singkat

- Bagi **train-test** dengan **stratify**.
- Pakai **LogisticRegression(class_weight='balanced')**.
- Uji *threshold* 0.5 vs 0.6.
- Laporkan **precision & recall**.

Pseudocode

```

X = ["jarak_km", "cuaca", "bangun_pagi", "naik_kendaraan"]
y = "telat_15menit"
train_test_split(X, y, stratify=y)
model = OneHotEncoder ->
LogisticRegression(class_weight="balanced")
fit model
proba = predict_proba(X_test)
pred05 = proba >= 0.5 ; pred06 = proba >= 0.6
report precision/recall for pred05 and pred06

```

Contoh Python

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_score, recall_score
import numpy as np

# Assuming df is loaded, e.g., from a CSV
# df = pd.read_csv("your_data.csv") # Uncomment and update with
your file path

# Verify DataFrame columns
print("Columns in DataFrame:", df.columns.tolist())
print("First few rows:\n", df.head())

# Define categorical and numerical columns
cat = ["cuaca"] # Update if needed based on df.columns
num = ["jarak_km", "bangun_pagi", "naik_kendaraan"] # Update if
needed

# Check if all required columns exist
required_columns = cat + num + ["telat_15menit"]
missing_columns = [col for col in required_columns if col not in
df.columns]
if missing_columns:
    raise ValueError(f"Missing columns in DataFrame:
{missing_columns}")

# Prepare features and target
X = df[cat + num]
y = df["telat_15menit"].map({"ya": 1, "tidak": 0})

# Create preprocessing and modeling pipeline
pre = ColumnTransformer([("cat",
OneHotEncoder(handle_unknown="ignore"), cat)],
remainder="passthrough")
pipe = Pipeline([("pre", pre),
("clf", LogisticRegression(max_iter=1000,
class_weight="balanced"))])

# Split data
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
random_state=42, stratify=y)

# Fit and predict
pipe.fit(Xtr, ytr)
p = pipe.predict_proba(Xte)[:, 1]
pred05 = (p >= 0.5).astype(int)
pred06 = (p >= 0.6).astype(int)
```

```
# Print metrics
print("0.5 precision:", precision_score(yte, pred05), "recall:",
      recall_score(yte, pred05))
print("0.6 precision:", precision_score(yte, pred06), "recall:",
      recall_score(yte, pred06))
```

Kunci jawaban (inti): *Threshold naik* → **precision** naik, **recall** turun. Pilih sesuai tujuan.

Proyek 3: Kelompok Ekskul (Clustering K-Means)

Tujuan: kelompokkan minat siswa ke **ekskul**: musik, olahraga, teater, robotik (skor 0–5).

Kenapa seru: tanpa label. Kita pakai *unsupervised*.

Output: 3 kelompok dengan deskripsi singkat.

Langkah singkat

- Standarkan skor (biar skala setara).
- **KMeans(k=3)**.
- Lihat rata-rata tiap fitur per *cluster*.
- Beri nama cluster (mis. “Atletis”, “Kreatif”, “Tekno”).

Pseudocode

```
X = ["musik", "olahraga", "teater", "robotik"]
scale X -> StandardScaler
fit KMeans(n_clusters=3, random_state=42)
cluster = predict(X)
for each cluster: mean(musik, olahraga, teater, robotik) -> beri
label nama
```

Contoh Python

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import pandas as pd

# Example: Create or load a DataFrame
# Option 1: If you have a CSV file, load it
# df = pd.read_csv('your_data.csv') # Replace with your file
# path

# Option 2: Create a sample DataFrame (uncomment to use)
df = pd.DataFrame({
    'musik': [4, 2, 5, 1, 3, 4, 2, 5, 3, 4],
    'olahraga': [2, 5, 1, 4, 3, 2, 5, 1, 4, 2],
    'teater': [3, 1, 4, 2, 5, 3, 1, 4, 2, 5],
    'robotik': [1, 4, 2, 5, 3, 1, 4, 2, 5, 3]
})
```

```

# Select features for clustering
X = df[["musik", "olahraga", "teater", "robotik"]]

# Standardize the features
Z = StandardScaler().fit_transform(X)

# Apply KMeans clustering
kmeans = KMeans(n_clusters=3, n_init=10, random_state=42)
df["cluster"] = kmeans.fit_predict(Z)

# Print mean values for each cluster
print(df.groupby("cluster")[["musik", "olahraga", "teater",
"robotik"]].mean())

# Interpretation of clusters
cluster_means = df.groupby("cluster")[["musik", "olahraga",
"teater", "robotik"]].mean()
for i, row in cluster_means.iterrows():
    print(f"\nInterpretasi Cluster {i}:")
    if row["olahraga"] > row[["musik", "teater",
"robotik"]].mean():
        print("Atletis: Tinggi dalam olahraga")
    elif row["musik"] > row[["olahraga", "teater",
"robotik"]].mean():
        print("Musikal: Tinggi dalam musik")
    elif row["teater"] > row[["musik", "olahraga",
"robotik"]].mean():
        print("Artistik: Tinggi dalam teater")
    elif row["robotik"] > row[["musik", "olahraga",
"teater"]].mean():
        print("Teknologis: Tinggi dalam robotik")
    else:
        print("Seimbang: Tidak ada kategori yang menonjol")

```

Output:

	musik	olahraga	teater	robotik
cluster				
0	2.0	4.5	1.5	4.5
1	4.5	1.5	3.5	1.5
2	3.5	2.5	5.0	3.0

Interpretasi Cluster 0:
Atletis: Tinggi dalam olahraga

Interpretasi Cluster 1:
Musikal: Tinggi dalam musik

Interpretasi Cluster 2:
Artistik: Tinggi dalam teater

Kunci jawaban (inti): jelaskan ciri rata-rata tiap *cluster* dan beri nama yang bermakna.

Proyek 4: Sentimen Komentar Kelas (Teks + *TF-IDF*)

Tujuan: klasifikasi **sentimen** komentar pendek: *positif/negatif*.

Kenapa seru: kalian mengolah **teks**.

Output: **accuracy** dan **5 prediksi contoh**.

Langkah singkat

- Kumpulkan 50–100 komentar (seimbang positif/negatif).
- *Split train–test* (*stratify*).
- *Pipeline*: **TfidfVectorizer** → **LinearSVC**.
- Tampilkan 5 baris: *komentar* | *prediksi* | *benar*.

Pseudocode

```
texts, labels = load_komentar()
split(train, test, stratify=labels)
pipeline = TfidfVectorizer(ngram=1-2) -> LinearSVC
fit on train
pred = predict(test_texts)
print accuracy, show 5 contoh hasil
```

Contoh file: *komentar.csv*

```
teks,label
"Makanannya enak sekali, saya suka!",positif
"Pelayanan sangat lambat dan bikin kesal.",negatif
"Harga murah dan kualitas oke.",positif
"Tempatnya kotor dan tidak nyaman.",negatif
"Pengiriman cepat dan barang sesuai.",positif
"Penjual tidak ramah, mengecewakan.",negatif
"Aplikasinya mudah dipakai dan praktis.",positif
"Suara musik terlalu bising.",negatif
"Saya puas sekali belanja di sini.",positif
"Produk rusak saat sampai.",negatif
```

Contoh Python

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score
import pandas as pd

df = pd.read_csv("komentar.csv") # kolom: teks, label
(positif/negatif)
Xtr,Xte,ytr,yte = train_test_split(df["teks"], df["label"],
                                   test_size=0.2,
                                   stratify=df["label"], random_state=42)
```

```

pipe = Pipeline([("tfidf", TfidfVectorizer(ngram_range=(1,2))),
                  ("clf", LinearSVC())])
pipe.fit(Xtr, ytr)
pred = pipe.predict(Xte)
print("Accuracy:", accuracy_score(yte, pred))
print(pd.DataFrame({"teks":Xte[:5].values, "pred":pred[:5],
                    "gold":yte[:5].values}))

```

Kunci jawaban (inti): *TF-IDF* menangkap kata penting; **LinearSVC** cepat dan kuat untuk teks pendek.

Proyek 5: Ramal Penjualan Harian (*Time Series* Sederhana)

Tujuan: ramal **penjualan harian** 7 hari ke depan.

Kenapa seru: belajar *time series* dasar.

Output: bandingkan **baseline** (*naive last day*) vs **model lag** (fitur **lag1**, **lag7**).

Langkah singkat

- Urutkan berdasarkan tanggal.
- Buat fitur **lag1** dan **lag7**.
- *Split* berdasarkan **waktu** (bukan acak).
- Latih **LinearRegression** pada fitur lag.
- Bandingkan **RMSE** dengan *baseline*.

Pseudocode

```

sort_by_date(df)
y = penjualan
df["lag1"] = shift(y,1)
df["lag7"] = shift(y,7)
drop rows with NA
train = data sampai minggu ke-(n-1)
test  = minggu terakhir
fit LinearRegression on [lag1, lag7]
pred_model = predict(test)
pred_naive = y(t-1)
compare RMSE(model) vs RMSE(naive)

```

Contoh file: *penjualan_harian.csv*

```

tanggal,penjualan
2025-01-01,50
2025-01-02,55
2025-01-03,53
2025-01-04,60
2025-01-05,62
2025-01-06,58
2025-01-07,65
2025-01-08,70
2025-01-09,68
2025-01-10,72

```

```
2025-01-11,74
2025-01-12,71
2025-01-13,69
2025-01-14,75
2025-01-15,80
2025-01-16,78
2025-01-17,82
2025-01-18,85
2025-01-19,83
2025-01-20,88
```

Contoh Python

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import numpy as np

df = pd.read_csv("penjualan_harian.csv") # kolom: tanggal,
penjualan
df = df.sort_values("tanggal")
df["lag1"] = df["penjualan"].shift(1)
df["lag7"] = df["penjualan"].shift(7)
df = df.dropna()

split_idx = int(len(df)*0.8)
train, test = df.iloc[:split_idx], df.iloc[split_idx:]

Xtr, ytr = train[["lag1","lag7"]], train["penjualan"]
Xte, yte = test[["lag1","lag7"]], test["penjualan"]

reg = LinearRegression().fit(Xtr, ytr)
pred_model = reg.predict(Xte)
rmse_model = mean_squared_error(yte, pred_model, squared=False)

pred_naive = Xte["lag1"].values # ramal = nilai kemarin
rmse_naive = mean_squared_error(yte, pred_naive, squared=False)

print("RMSE Model:", rmse_model, " | RMSE Naive:", rmse_naive)
```

Kunci jawaban (inti): jika **RMSE model < RMSE naive**, maka fitur **lag1/lag7** bermanfaat dan model **lebih baik** dari *baseline*.

Catatan Umum (Berlaku untuk 5 Proyek)

- **Data bersih itu nomor 1.**
- Tulis *pipeline* kalau perlu: rapi, konsisten, mudah diulang.
- Laporkan **metrik yang relevan** (MAE/RMSE untuk regresi; accuracy/precision/recall/F1 untuk klasifikasi).
- Tambah **kesimpulan 2–3 kalimat**: apa artinya bagi sekolah/kegiatan kalian.
- Simpan *notebook* dan grafik. Jadi **portofolio** kalian.

Dengan proyek ini, kalian latihan *end-to-end*: dari *dataset* sampai **keputusan**. **Kecil, nyata, dan bisa dipamerkan!**

Institut Teknologi Tangerang Selatan (ITTS) dan Penulis.

Para penulis adalah **dosen aktif di Institut Teknologi Tangerang Selatan (ITTS)**, sebuah **institusi pendidikan tinggi berbasis teknologi** yang berperan aktif dalam pengembangan **solusi digital** dan *kecerdasan buatan (AI)*. Tidak hanya mengajar, mereka juga merupakan **praktisi dan inovator** yang menghasilkan **karya-karya nyata di bidang teknologi mandiri**.

Berlokasi di **Komplek Komersial BSD, Serpong Utara, Tangerang Selatan**, ITTS hadir sebagai **pusat inovasi** yang mengintegrasikan **pembelajaran, penelitian, dan pengembangan teknologi secara menyeluruh**. Fokus utama ITTS mencakup bidang strategis seperti *AI, Machine Learning, Deep Learning*, jaringan, *cloud*, keamanan siber, pemrograman, sistem informasi, dan multimedia.

ITTS menawarkan lingkungan akademik yang dinamis dan kolaboratif. Setiap minggu, diselenggarakan berbagai *workshop, demo teknologi*, seminar, hingga *webinar*—mayoritas **gratis dan tersedia secara terbuka di YouTube**. Ini menciptakan **kultur belajar yang aktif dan relevan dengan perkembangan industri**.

Fasilitas ITTS menunjang kegiatan riset dan inovasi secara optimal. Lab komputer dilengkapi *GPU RTX 4060* untuk eksperimen *AI*, serta **laboratorium teknologi dengan puluhan server, perangkat IoT, sistem komunikasi radio, dan infrastruktur jaringan jarak jauh**. Dari ekosistem ini lahir **produk-produk mandiri** seperti **ChatGPT versi lokal** untuk institusi pendidikan dan **sistem perpustakaan digital yang dapat diakses tanpa koneksi internet**.

Daya saing ITTS juga tercermin dari karakter dosen dan mahasiswa yang tidak hanya akademik, tetapi juga **produktif secara praktis**. Mahasiswa didorong aktif menjadi **narasumber, penulis, dan peneliti** sejak dini. Banyak diantaranya telah menghasilkan **karya buku, artikel ilmiah, serta terlibat langsung dalam proyek-proyek teknologi**.

Kolaborasi strategis dengan berbagai mitra industri—seperti *IDCloudHost, Ubuntu, XecureIT, Dinas KOMINFO Tangerang Selatan*, berbagai *ISP*, dan komunitas IT nasional—menjadi **jembatan nyata antara kampus dan dunia profesional**. Hasilnya, **lebih dari 75% mahasiswa ITTS telah bekerja bahkan sebelum lulus, dan hampir seluruh lulusan langsung terserap industri**.

ITTS bukan hanya tempat belajar, tapi tempat mencipta. Dosen sebagai **penulis dan praktisi**, mahasiswa sebagai **peneliti dan kontributor**—semuanya berperan **membangun ekosistem teknologi yang mandiri dan berdaya saing tinggi**.

Institut Teknologi Tangerang Selatan (ITTS)

Komplek Komersial BSD

Jl. Raya Serpong No. Kav. 9, Lengkong Karya, Serpong Utara

Kota Tangerang Selatan, Banten 15331

☎ (+62) 877-7277-1775

✉ info@itts.ac.id

🌐 www.itts.ac.id